



(19) 대한민국특허청(KR)
(12) 등록특허공보(B1)

(45) 공고일자 2016년05월27일
(11) 등록번호 10-1625324
(24) 등록일자 2016년05월23일

(51) 국제특허분류(Int. Cl.)
G06F 9/46 (2006.01) G06F 9/30 (2006.01)
G06F 9/38 (2006.01)
(21) 출원번호 10-2014-7031950
(22) 출원일자(국제) 2012년11월26일
심사청구일자 2014년11월26일
(85) 번역문제출일자 2014년11월14일
(65) 공개번호 10-2015-0004839
(43) 공개일자 2015년01월13일
(86) 국제출원번호 PCT/IB2012/056734
(87) 국제공개번호 WO 2013/186604
국제공개일자 2013년12월19일
(30) 우선권주장
13/524,788 2012년06월15일 미국(US)
(56) 선행기술조사문헌
US20110145637 A1
US6195744 B1
US20110320420 A1
US20070260608 A1

(73) 특허권자
인터내셔널 비즈니스 머신즈 코포레이션
미국 10504 뉴욕주 아몬크 뉴오차드 로드
(72) 발명자
그라이너, 덴
미국 캘리포니아 95141, 산 호세, 베일리 애비뉴
555, SVL/090/F374, 아이비엠 코포레이션
슬레겔, 티모시
미국 뉴욕 12601, 포킵시, 사우스 로드 2455, 엠
에스 피310, 아이비엠 코포레이션
자코비, 크리스찬
미국 뉴욕 12601, 포킵시, 사우스 로드 2455, 빌
딩 706/디10, 아이비엠 코포레이션
(74) 대리인
허정훈

전체 청구항 수 : 총 10 항

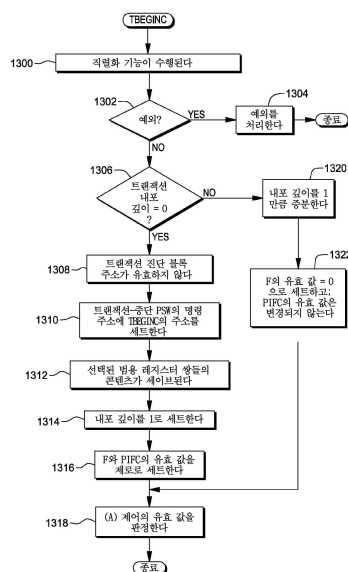
심사관 : 유진태

(54) 발명의 명칭 제약 트랜잭션 실행

(57) 요약

제약 트랜잭션 처리(constrained transactional processing)가 제공된다. 제약 트랜잭션은 제약 Transaction Begin 명령의 실행에 의해 개시된다. 상기 제약 트랜잭션은 그와 연관된 다수의 제한사항들을 갖는다. 제한사항의 위반이 없으면, 상기 제약 트랜잭션은 완료된다. 만일 중단 조건을 만나면, 상기 트랜잭션은 재실행되어 상기 Transaction Begin 명령에서 시작한다. 제한사항의 위반은 인터럽트를 일으킬 수 있다.

대표도 - 도13



명세서

청구범위

청구항 1

컴퓨팅 환경 내에서 트랜잭션들(transactions)을 처리하기 위한 방법에 있어서, 상기 방법은:

프로세서에 의해, 기계어 명령(machine instruction)을 획득하는 단계—상기 기계어 명령은 명시된 유형의 트랜잭션을 개시하고, 상기 명시된 유형은 상기 기계어 명령의 한 필드에 명시적으로 표시됨—;

상기 프로세서에 의해, 상기 기계어 명령에 의해 개시된 트랜잭션을 실행하는 단계—상기 트랜잭션은 선택된 트랜잭션이 완료될 때까지 메인 메모리에 대한 트랜잭션 저장을 커밋(commit)하는 것을 효과적으로 지연시킴—;

상기 트랜잭션을 실행하는 것에 기초하여, 중단 조건(an abort condition)이 발생하였는지 판정하는 단계;

상기 트랜잭션의 명시된 유형이 제약 트랜잭션(a constrained transaction)인지 검사하는 단계—상기 제약 트랜잭션은 그것과 연관된 하나 또는 그 이상의 제한사항들을 가지며, 상기 검사하는 단계는 상기 기계어 명령의 필드를 검사하여 상기 트랜잭션이 제약 트랜잭션인지 판정하는 단계를 포함함—; 및

상기 중단 조건에 따라서 그리고 상기 트랜잭션의 명시된 유형에 따라서 처리를 수행하는 단계—상기 검사하는 단계에서 상기 트랜잭션이 제약 트랜잭션이라고 표시하는 것에 기초하여 그리고 상기 중단 조건이 특정 유형의 중단 조건인 것에 기초하여, 상기 처리를 수행하는 단계는 상기 트랜잭션을 재실행하기 위해 상기 트랜잭션의 시작부(beginning)로 돌아가는 단계를 포함하고, 상기 트랜잭션의 시작부로 돌아가는 단계 이외의 패일 경로(fail path)가 상기 중단 조건을 처리하기 위해 제공되지는 않음—; 및

상기 트랜잭션을 재실행하는 단계를 포함하되, 상기 재실행하는 단계는 상기 트랜잭션의 완료를 용이하게 하기 위해 하나 또는 그 이상의 액션들을 수행하는 단계를 포함하는,

방법.

청구항 2

제1항에 있어서, 상기 기계어 명령은 트랜잭션 시작 명령을 포함하고, 상기 트랜잭션 시작 명령은 상기 트랜잭션이 제약된 것인지를 표시하는,

방법.

청구항 3

제2항에 있어서, 상기 트랜잭션의 시작부로 돌아가는 단계는 상기 트랜잭션 시작 명령으로 돌아가는 것을 포함하는,

방법.

청구항 4

삭제

청구항 5

제1항에 있어서, 상기 하나 또는 그 이상의 액션들은:

비순차적 실행(out-of-order execution)을 금지하는 것;

상기 트랜잭션과 충돌하는 메모리 위치들에 다른 프로세서들이 액세스하는 것을 금지하는 것; 또는

중단 처리시 임의의 지연(random delays)을 유도하는 것 중 적어도 하나를 포함하는,

방법.

청구항 6

제1항에 있어서, 상기 트랜잭션의 시작부로 돌아가는 단계는 상기 트랜잭션의 완료 또는 인터럽션 전에 한 번 또는 그 이상 수행되는,

방법.

청구항 7

제1항에 있어서, 상기 방법은:

상기 검사하는 단계에서 상기 트랜잭션이 제약 트랜잭션이라고 표시하는 것에 기초하여, 상기 하나 또는 그 이상의 제한사항들 중 하나의 제한 위반이 발생하였는지 판정하는 단계; 및

상기 위반이 발생하였다고 판정하는 것에 기초하여 인터럽트를 일으키는 단계를 더 포함하는,

방법.

청구항 8

제1항에 있어서, 상기 하나 또는 그 이상의 제한사항들은:

상기 트랜잭션 내에는 한정된 수의 명령들이 있음;

상기 트랜잭션 내의 명령들은 미리 정의된 메모리 구역 내에 있어야 함;

특정의 미리 명시된 명령들은 상기 트랜잭션 내에 포함될 수 있고, 다른 명령들은 제한됨;

상기 트랜잭션의 스토리지 오퍼랜드들에 의해 액세스에 한정이 있음;

상기 트랜잭션은 상기 미리 정의된 메모리 구역을 포함하는 메모리의 블록들 내의 스토리지 오퍼랜드들을 액세스하지 않음;

상기 트랜잭션은 동일 절대 주소에 매핑되는 다른 논리 주소들을 사용하여 명령들 또는 스토리지 오퍼랜드들을 액세스하지 않음; 및

상기 트랜잭션에 의해 만들어진 오퍼랜드 참조들은 명시된 사이즈 내에 있어야 함 중, 적어도 하나를 포함하는,

방법.

청구항 9

삭제

청구항 10

제1항에 있어서, 상기 트랜잭션은 다수의 비인접 메모리 위치들을 업데이트하는,

방법.

청구항 11

컴퓨팅 환경 내에서 트랜잭션들(transactions)을 처리하기 위한 방법을 수행하도록 구성된 컴퓨터 시스템으로서, 상기 컴퓨터 시스템은:

메모리; 및

상기 메모리와 통신하는 프로세서를 포함하고, 상기 방법은:

프로세서에 의해, 기계어 명령(machine instruction)을 획득하는 단계—상기 기계어 명령은 명시된 유형의 트랜잭션을 개시하고, 상기 명시된 유형은 상기 기계어 명령의 한 필드에 명시적으로 표시됨—;

상기 프로세서에 의해, 상기 기계어 명령에 의해 개시된 트랜잭션을 실행하는 단계—상기 트랜잭션은 선택된 트랜잭션이 완료될 때까지 메인 메모리에 대한 트랜잭션 저장을 커밋(commit)하는 것을 효과적으로 지연시킴—;

상기 트랜잭션을 실행하는 것에 기초하여, 중단 조건(an abort condition)이 발생하였는지 판정하는 단계;

상기 트랜잭션의 명시된 유형이 제약 트랜잭션(a constrained transaction)인지 검사하는 단계—상기 제약 트랜잭션은 그것과 연관된 하나 또는 그 이상의 제한사항들을 가지며, 상기 검사하는 단계는 상기 기계어 명령의 필드를 검사하여 상기 트랜잭션이 제약 트랜잭션인지 판정하는 단계를 포함함—; 및

상기 중단 조건에 따라서 그리고 상기 트랜잭션의 명시된 유형에 따라서 처리를 수행하는 단계—상기 검사하는 단계에서 상기 트랜잭션이 제약 트랜잭션이라고 표시하는 것에 기초하여 그리고 상기 중단 조건이 특정 유형의 중단 조건인 것에 기초하여, 상기 처리를 수행하는 단계는 상기 트랜잭션을 재실행하기 위해 상기 트랜잭션의 시작부(beginning)로 돌아가는 단계를 포함하고, 상기 트랜잭션의 시작부로 돌아가는 것 이외의 페일 경로(fail path)가 상기 중단 조건을 처리하기 위해 제공되지는 않음—; 및

상기 트랜잭션을 재실행하는 단계를 포함하되, 상기 재실행하는 단계는 상기 트랜잭션의 완료를 용이하게 하기 위해 하나 또는 그 이상의 액션들을 수행하는 단계를 포함하는,

컴퓨터 시스템.

청구항 12

프로세서에 의해서 판독 가능하고, 컴퓨팅 환경 내에서 트랜잭션들(transactions)을 처리하기 위한 방법을 수행하기 위해 상기 프로세서에 의해 실행하기 위한 명령들을 저장하는 비-일시적인 컴퓨터 판독가능 저장 매체로서, 상기 방법은:

상기 프로세서에 의해, 기계어 명령(machine instruction)을 획득하는 단계—상기 기계어 명령은 명시된 유형의 트랜잭션을 개시하고, 상기 명시된 유형은 상기 기계어 명령의 한 필드에 명시적으로 표시됨—;

상기 프로세서에 의해, 상기 기계어 명령에 의해 개시된 트랜잭션을 실행하는 단계—상기 트랜잭션은 선택된 트랜잭션이 완료될 때까지 메인 메모리에 대한 트랜잭션 저장을 커밋(commit)하는 것을 효과적으로 지연시킴—;

상기 트랜잭션을 실행하는 것에 기초하여, 중단 조건(an abort condition)이 발생하였는지 판정하는 단계;

상기 트랜잭션의 명시된 유형이 제약 트랜잭션(a constrained transaction)인지 검사하는 단계—상기 제약 트랜잭션은 그것과 연관된 하나 또는 그 이상의 제한사항들을 가지며, 상기 검사하는 단계는 상기 기계어 명령의 필드를 검사하여 상기 트랜잭션이 제약 트랜잭션인지 판정하는 단계를 포함함—; 및

상기 중단 조건에 따라서 그리고 상기 트랜잭션의 명시된 유형에 따라서 처리를 수행하는 단계—상기 검사하는 단계에서 상기 트랜잭션이 제약 트랜잭션이라고 표시하는 것에 기초하여 그리고 상기 중단 조건이 특정 유형의 중단 조건인 것에 기초하여, 상기 처리를 수행하는 단계는 상기 트랜잭션을 재실행하기 위해 상기 트랜잭션의 시작부(beginning)로 돌아가는 단계를 포함하고, 상기 트랜잭션의 시작부로 돌아가는 것 이외의 페일 경로(fail path)가 상기 중단 조건을 처리하기 위해 제공되지는 않음—; 및

상기 트랜잭션을 재실행하는 단계를 포함하되, 상기 재실행하는 단계는 상기 트랜잭션의 완료를 용이하게 하기 위해 하나 또는 그 이상의 액션들을 수행하는 단계를 포함하는,

비-일시적인 컴퓨터 판독가능 저장 매체.

발명의 설명

기술 분야

[0001] 하나 또는 그 이상의 특징들은 일반적으로 다중 처리 컴퓨팅 환경들에 관한 것이고, 구체적으로 그러한 컴퓨팅 환경들 내에서 트랜잭션 처리에 관한 것이다.

배경 기술

[0002] 멀티프로세서 프로그래밍에서 다중 중앙 처리 장치들(CPU들)에 의한 동일한 스토리지 위치에 대한 업데이트(updates)를 위한 도전이 끈질기게 이어지고 있다. AND 연산 같은 단순한 논리 연산들을 포함하여 스토리지 위치들을 업데이트하는 많은 명령들도 그 위치에 대한 다중 액세스에 대하여 도전하고 있다. 예를 들어, 우선, 스토리지 위치가 폐지되고, 그 다음에, 업데이트된 결과가 다시 저장된다.

[0003] 다중 CPU들이 동일한 스토리지 위치를 안전하게 업데이트하기 위하여, 그 위치에 대한 액세스는 직렬화된다(serialized). 인터내셔널 비지네스 머신즈 코퍼레이션에 의해 예전에 공급된 S/360 아키텍처와 함께 소개된 한 명령인 TEST AND SET(테스트 및 세트) 명령은 스토리지 위치의 인터로크 업데이트(interlocked update)를 제공하였다. 인터로크 업데이트가 의미하는 바는, 다른 CPU들과 입력/출력(I/O) 서브시스템(예를 들어, 채널 서브시스템)에 의해 관찰될 때, 상기 명령의 전체 스토리지 액세스가 원자적으로 발생하는 것으로 보인다는 것이다. 그 후에, 인터내셔널 비지네스 머신즈 코퍼레이션에 의해 공급된 S/370 아키텍처는 COMPARE AND SWAP(비교 및 교환) 명령 및 COMPARE DOUBLE AND SWAP(이중 비교 및 교환) 명령을 소개하였으며, 이 명령들은 인터로크 업데이트를 수행하기 위한 더 정교한 수단을 제공하고, 일반적으로 로크 워드(lock word)(또는 세마포어)로 알려진 것의 구현을 가능하게 한다. 최근에 추가된 명령들은 추가적인 인터로크 업데이트 능력들을 제공하는데, COMPARE AND SWAP AND PURGE(비교 및 교환 및 소거) 명령, 및 COMPARE AND SWAP AND STORE(비교 및 교환 및 저장) 명령이 포함된다. 하지만, 이 명령들 모두는 단지 단일 스토리지 위치에 대해서만 인터로킹(interlocking)을 제공한다.

[0004] 더 복잡한 프로그램 기술들은 이중 연결 목록에 엘리먼트를 추가할 때처럼 다중 스토리지 위치들의 인터로크 업데이트를 필요로할 수 있다. 그러한 연산에서, 다른 CPU들과 I/O 서브시스템에 의해 관찰될 때, 전방향 포인터와 역방향 포인터는 모두 동시에 업데이트되는 것으로 보인다. 그러한 다중 위치 업데이트를 실현하기 위해서, 프로그램은 로크 워드 같은 별개의 단일 직렬화 요소를 강제로 사용하게 된다. 하지만, 로크 워드들은 보증되는 것보다 훨씬 더 조악한 수준의 직렬화를 제공할 수 있는데, 예를 들면, 로크 워드들은 단지 두 엘리먼트만 업데이트되는데도 불구하고 전체 수백만 엘리먼트들의 큐(queue)를 직렬화할 수 있다. 프로그램은 아주 세밀한(finer-grained) 직렬화(예를 들어, 로크 지점들의 계층)를 사용하도록 데이터를 구조화하지만, 그것은 만일 상기 계층이 위반될 경우의 잠재적인 교착 상태 상황들(potential deadlock situations)과, 만일 상기 프로그램이 하나 또는 그 이상의 로크들을 보유하고 있는 중에 오류가 발생하거나 또는 로크가 획득될 수 없을 경우의 복구 이슈들(recovery issues) 같은 추가적인 문제들을 발생시킬 수 있다.

[0005] 진술한 것에 더하여, 프로그램이 일련의 명령들을 실행하고 그 결과가 예외 조건으로 나오거나 나오지 않을 수 있는 상황들은 많이 있다. 만일 예외 조건이 발생하지 않으면 프로그램은 계속되지만, 만일 예외가 인지되면 프로그램은 그 예외 조건을 제거하기 위해 교정 조치를 취할 수 있다. 한 예로서, 자바(Java)는 그러한 실행을 예를 들어 추론적 실행(speculative execution), 기능의 부분적 인라인링(partial in-lining of a function), 및/또는 포인터 널 검사의 순서 재정렬(re-sequencing of pointer null checking)에 활용할 수 있다.

[0006] 인터내셔널 비지네스 머신즈 코퍼레이션에 의해 공급되는 z/OS 및 그 이전 세대들 같은 고전적인 운영체제 환경들에서, 프로그램은 발생할 수 있는 어느 프로그램-예외 조건이든 인터셉트할 수 있는 복구 환경을 구축한다. 만일 프로그램이 예외를 인터셉트하지 않으면, 운영체제는 운영체제가 처리할 준비가 되어 있지 않은 예외들에 대하여 통상적으로 프로그램을 비정상적으로 종료시킨다. 그러한 환경을 구축하고 활용하는 것은 비용이 많이 들고 복잡하다.

발명의 내용

과제의 해결 수단

[0007] 컴퓨팅 환경 내에서 트랜잭션들을 처리하기 위한 컴퓨터 프로그램 제품의 제공을 통해서 선행 기술의 단점들이 극복되고 장점들이 제공된다. 상기 컴퓨터 프로그램 제품은, 처리 회로에 의해 판독 가능한 그리고 어떤 방법을 수행하기 위해 상기 처리 회로에 의해 실행할 명령들을 저장하는, 컴퓨터 판독 가능 스토리지 매체를 포함한다. 상기 방법은, 예를 들어, 프로세서에 의해 컴퓨팅 환경 내에서 트랜잭션을 실행하는 단계—상기 트랜잭션은 선택된 트랜잭션이 완료될 때까지 메인 메모리에 대한 트랜잭션 저장을 커밋(commit)하는 것을 효과적으로 지연시킴—; 상기 트랜잭션을 실행하는 것에 기초하여, 중단 조건이 발생했는지를 판정하는 단계; 상기 트랜잭션이 제약 트랜잭션인지를 검사하는 단계—상기 제약 트랜잭션은 그것과 연관된 하나 또는 그 이상의 제한사항들을 가짐—; 및 상기 중단 조건을 판정한 것에 기초하여 그리고 상기 검사하는 단계가 상기 트랜잭션이 제약 트랜잭션이라고 표시하는 것에 기초하여, 상기 트랜잭션의 시작부(beginning)로 돌아가서 상기 트랜잭션을 재실행하는 단계를 포함한다.

[0008] 하나 또는 그 이상의 실시 예들과 관련된 방법들과 시스템들이 또한 여기에서 기술되고 청구된다. 추가로, 하나 또는 그 이상의 실시 예들과 관련된 서비스들 또한 여기에서 기술되고 청구될 수 있다.

[0009] 추가적인 특징들과 장점들이 구현된다. 다른 실시 예들과 특징들이 여기에서 상세하게 기술되며 청구하는 발명

의 일부로 간주된다.

도면의 간단한 설명

[0010]

하나 또는 그 이상의 특징들이 구체적으로 언급되고 본 명세서의 끝 부분의 청구 범위에서 예시로서 분명하게 청구된다. 전술한 것과 다른 대상들, 특징들, 및 장점들은 다음과 같은 내용으로 첨부되는 도면들과 그 다음에 오는 발명을 실시하기 위한 구체적인 내용을 참조하면 분명해진다.

도 1은 컴퓨팅 환경의 한 실시 예를 도시한다.

도 2a는 Transaction Begin(트랜잭션 시작, TBEGIN) 명령의 한 예를 도시한다.

도 2b는 도 2a의 TBEGIN 명령의 한 필드의 더 상세한 사항의 한 실시 예를 도시한다.

도 3a는 제약 Transaction Begin(TBEGINC) 명령의 한 예를 도시한다.

도 3b는 도 3a의 TBEGINC 명령의 한 필드의 더 상세한 사항의 한 실시 예를 도시한다.

도 4는 Transaction End(트랜잭션 종료, TEND) 명령의 한 예를 도시한다.

도 5는 Transaction Abort(트랜잭션 중단, TABORT) 명령의 한 예를 도시한다.

도 6은 내포된(nested) 트랜잭션들의 한 예를 도시한다.

도 7은 NONTRANSACTIONAL STORE(비트랜잭션 저장, NTSTG) 명령의 한 예를 도시한다.

도 8은 EXTRACT TRANSACTION NESTING DEPTH(트랜잭션 내포 깊이 추출, ETND) 명령의 한 예를 도시한다.

도 9는 트랜잭션 진단 블록(transaction diagnostic block, TDB)의 한 예를 도시한다.

도 10은 중단의 예시 이유들을 연관된 중단 코드들 및 조건 코드들과 함께 도시한다.

도 11은 제약 또는 비제약 트랜잭션을 생성하는 것과 연관된 로직의 한 실시 예를 도시한다.

도 12는 트랜잭션을 실행하는 것과 연관된, 구체적으로 트랜잭션의 유형에 따른 중단 조건을 처리하는 것과 연관된 로직의 한 실시 예를 도시한다.

도 13은 TBEGINC 명령을 실행하는 것과 연관된 로직의 한 실시 예를 도시한다.

도 14는 제약 트랜잭션의 완료를 용이하게 하기 위해 액션을 취하는 것과 연관된 로직의 한 실시 예를 도시한다.

도 15a와 15b는 큐 엘리먼트를 큐 엘리먼트들의 이중 연결 목록으로 삽입하는 것의 일 예를 도시한다.

도 16은 컴퓨터 프로그램 제품의 한 실시 예를 도시한다.

도 17은 호스트 컴퓨터 시스템의 한 실시 예를 도시한다.

도 18은 컴퓨터 시스템의 추가 예를 도시한다.

도 19는 컴퓨터 네트워크를 포함하는 컴퓨터 시스템의 또 다른 예를 도시한다.

도 20은 컴퓨터 시스템의 여러 엘리먼트들의 한 실시 예를 도시한다.

도 21a는 도 20의 컴퓨터 시스템의 실행 유닛의 한 실시 예를 도시한다.

도 21b는 도 20의 컴퓨터 시스템의 분기 유닛의 한 실시 예를 도시한다.

도 21c는 도 20의 컴퓨터 시스템의 로드/저장 유닛의 한 실시 예를 도시한다.

도 22는 에뮬레이트된 호스트 컴퓨터 시스템의 한 실시 예를 도시한다.

발명을 실시하기 위한 구체적인 내용

[0011]

한 특징에 따라서, 트랜잭션 실행(TX) 퍼실리티가 제공된다. 이 퍼실리티는 명령들에 대한 트랜잭션 처리를 제공하고, 하나 또는 그 이상의 실시 예들에서, 아래에 기술되는 바와 같이 각각 다른 실행 모드를 제공하며, 트랜잭션 처리의 내포 레벨들 또한 제공한다.

- [0012] 이 트랜잭션 실행 퍼실리티는 트랜잭션 실행(TX) 모드라 불리는 CPU 상태(state)를 도입한다. CPU 리셋 후에, 그 CPU는 TX 모드에 있지 않는다. 상기 CPU는 TRANSACTION BEGIN(트랜잭션 시작) 명령에 의해 TX 모드로 진입한다. 상기 CPU는 (a) 최외부(outermost) TRANSACTION END(트랜잭션 종료) 명령(내부와 외부에 대한 더 상세한 사항은 이후에 나옴) 또는 (b) 상기 트랜잭션이 중단될 둘 중 하나에 의해 TX 모드에서 벗어난다. TX 모드에 있는 동안, 상기 CPU에 의한 스토리지 액세스는 다른 CPU들과 I/O 서브시스템에 의해 관찰될 때 블록-동시적(block-concurrent)으로 보인다. 상기 스토리지 액세스는 (a) 상기 최외부 트랜잭션이 중단 없이 종료될 때(즉, 예를 들어, 상기 CPU의 로컬 캐시 또는 버퍼에서 이루어진 업데이트들은 실제 메모리에 전파되고 저장되며 다른 CPU들에 보임) 스토리지에 커밋되거나 또는 (b) 만일 상기 트랜잭션이 중단되면 폐기된다.
- [0013] 트랜잭션들은 내포될 수 있다. 즉, 상기 CPU가 TX 모드에 있는 동안, 상기 CPU는 또 다른 TRANSACTION BEGIN(트랜잭션 시작) 명령을 실행할 수 있다. 상기 CPU가 TX 모드로 진입되게 하는 명령은 최외부 TRANSACTION BEGIN(트랜잭션 시작)이라 불리고, 유사하게 상기 프로그램은 최외부 트랜잭션에 있다고 말한다. TRANSACTION BEGIN(트랜잭션 시작)의 이후 실행들은 내부 명령들(inner instructions)이라 불리고, 상기 프로그램은 내부 트랜잭션을 실행한다. 상기 모델은 최소 내포 깊이와 모델-종속적인 최대 내포 깊이를 제공한다. EXTRACT TRANSACTION NESTING DEPTH(트랜잭션 내포 깊이 추출) 명령은 현재의 내포 깊이 값을 반환하며(return), 추가 실시 예에서는 최대 내포 깊이 값을 반환할 수도 있다. 이 기술은 "평탄화된 내포(flattened nesting)"라 불리는 모델을 사용하는데, 이 평탄화된 내포 모델에서 어느 내포 깊이에서든 중단 조건은 모든 레벨의 트랜잭션을 중단시키고, 제어는 최외부 TRANSACTION BEGIN(트랜잭션 시작) 다음에 오는 명령으로 반환된다.
- [0014] 트랜잭션의 처리 동안, 한 CPU에 의해 이루어지는 트랜잭션 액세스는 (a) 다른 CPU에 의해 이루어지는 트랜잭션 액세스 또는 비트랜잭션 액세스와 충돌한다고 하거나, 또는 (b) I/O 서브시스템에 의해 이루어지는 비트랜잭션 액세스와 충돌한다고 말한다(만일 두 액세스들이 동일한 캐시 라인 내의 임의 위치에 대한 것이고, 두 액세스 중 하나 또는 모두가 저장(store)일 경우). 다른 말로 하면, 트랜잭션 실행이 생산적(productive) 되도록 하기 위해, 상기 CPU는 커밋할 때까지 트랜잭션 액세스를 하는 것으로 관찰되지 않는다. 이 프로그래밍 모델은 어떤 환경들에서는 매우 효과적일 수 있는데, 예를 들면, 백만개의 엘리먼트들의 이중 연결 목록에서 두 지점을 업데이트 하는 경우를 들 수 있다. 하지만, 만일 트랜잭션 액세스되는 스토리지 위치들에 대하여 경쟁(contention)이 많으면, 덜 효과적일 수도 있다.
- [0015] 트랜잭션 실행의 한 모델(여기에서는 비계약 트랜잭션이라 불림)에서, 트랜잭션이 중단될 때, 프로그램은 그 중단 조건이 더이상 존재하지 않기를 희망하며 그 트랜잭션의 재구동(re-drive)을 시도하거나, 또는 프로그램은 등가의 비트랜잭션 경로(equivalent non-transactional path)로 "폴백(fall back)"할 수 있다. 트랜잭션 실행의 또 다른 모델(여기에서는 계약 트랜잭션이라 불림)에서, 중단된 트랜잭션은 CPU에 의해 자동으로 재구동되고, 제약 위반들(constraint violations)이 없을시에, 상기 제약 트랜잭션은 최종적인 완료(eventual completion)를 보장받는다.
- [0016] 트랜잭션을 개시(initiating)할 때, 프로그램은 여러 제어들을 명시할 수 있는데, 그 제어들의 예는 다음과 같다: (a) 만일 상기 트랜잭션이 중단되면 어느 범용 레지스터들이 자신들의 원래 콘텐츠로 복원될 것인가, (b) 상기 트랜잭션이 예를 들어 부동 소수점 레지스터들과 부동 소수점 제어 레지스터를 포함하여 부동-소수점-레지스터 컨텍스트(context)를 수정하도록 허가되었는지 여부, (c) 상기 트랜잭션이 액세스 레지스터들(AR들)을 수정하도록 허가되었는지 여부, 및 (d) 특정 프로그램-예외 조건들이 인터럽션을 일으키는 것으로부터 차단되는지 여부. 만일 비계약 트랜잭션이 중단되면, 여러 진단 정보가 제공될 수 있다. 예를 들어, 비계약 트랜잭션을 개시하는 최외부 TBEGIN 명령은 프로그램 명시 트랜잭션 진단 블록(program specified TDB)을 지정할 수 있다. 또한, 만일 트랜잭션이 프로그램 인터럽션으로 인해 또는 해석적 실행이 종료되도록 야기하는 조건으로 인해 각각 중단되면, CPU의 프리픽스 영역(prefix area)에 있거나 또는 호스트의 상태 묘사에 의해 지정되는 TDB가 또한 사용될 수 있다.
- [0017] 위에서는 레지스터들의 여러 유형들을 표시하고 있다. 여기에서 이들에 대해 더 상세하게 설명한다. 범용 레지스터(general registers)는 일반적인 산술 및 논리 연산들에서 누산기(accumulators)로서 사용될 수 있다. 한 실시 예에서, 각 레지스터는 64개 비트 위치들(bit positions)을 포함하며, 여기에는 16개의 범용 레지스터가 있다. 범용 레지스터들은 숫자 0~15에 의해 식별되고, 명령에서 4-비트의 R 필드에 의해 지정된다. 일부 명령들은 여러 R 필드들을 보유함으로써 다중 범용 레지스터들의 주소지정을 제공한다. 일부 명령들에서, 특정한 범용 레지스터의 사용은 그 명령의 R 필드에 의해 명시적으로 지정되기보다는 암시된다.
- [0018] 일반적인 산술 및 논리 연산들에서 누산기로서 사용하는 것에 더하여, 상기 16개 범용 레지스터들 중 15개는 주

소 생성에서 기준 주소 및 인덱스 레지스터(base address and index registers)로서 사용될 수도 있다. 이러한 경우에, 상기 레지스터들은 명령에서 4-비트의 B 필드 또는 X 필드에 의해 지정된다. 상기 B 또는 X 필드 내 제로 값은 기준(base) 또는 인덱스(index)가 적용되지 않는 것으로 명시하며, 따라서 범용 레지스터 0은 기준 주소 또는 인덱스를 포함하는 것으로 지정되지 않는다.

- [0019] 부동 소수점 명령들은 일 세트의 부동 소수점 레지스터들을 사용한다. 한 실시 예에서, CPU는 16개의 부동 소수점 레지스터들을 보유한다. 부동 소수점 레지스터들은 숫자 0~15에 의해 식별되고, 부동 소수점 명령들에서 4-비트의 R 필드에 의해 지정된다. 각 부동 소수점 레지스터는 64비트의 길이를 가지며 짧은(32비트) 또는 긴(64비트) 부동 소수점 오퍼랜드를 포함한다.
- [0020] 부동 소수점 제어(FPC) 레지스터는 마스크 비트들, 플래그 비트들, 데이터 예외 코드, 및 라운딩 모드 비트들을 포함하는 32비트 레지스터이고, 부동 소수점 연산들의 처리 동안 사용된다.
- [0021] 또한, 한 실시 예에서, CPU는 16개의 제어 레지스터들을 보유하고, 각각은 64개 비트 위치들을 보유한다. 상기 레지스터들 내 비트 위치들은 프로그램 이벤트 기록(Program Event Recording, PER) 같은 시스템 내 특정한 퍼실리티들에 할당되어, 연산이 발생할 수 있다고 명시하거나 또는 그 퍼실리티에 의해 요구되는 특별(special) 정보를 제공하는 데 사용된다. 한 실시 예에서, 트랜잭션 퍼실리티를 위해서, CR0(비트들 8과 9) 및 CR2(비트들 61~63)가 사용되며, 아래에 기술되는 바와 같다.
- [0022] CPU는 예를 들어 0~15로 번호가 붙은 16개의 액세스 레지스터들을 보유한다. 액세스 레지스터는 주소 공간 제어 엘리먼트(ASCE)의 간접 명세(indirect specification)를 포함하는 32개의 비트 위치들로 구성된다. 주소 공간 제어 엘리먼트는 동적 주소 변환(DAT) 메커니즘에 의해 레퍼런스들(references)을 대응 주소 공간으로 변환시키는 데 사용되는 매개변수이다. CPU가 액세스 레지스터 모드(프로그램 상태 워드(PSW) 내 비트들에 의해 제어됨)라 불리는 모드에 있을 때, 스토리지 오퍼랜드 참조를 위해 논리 주소를 명시하는 데 사용되는 명령의 B 필드가 액세스 레지스터를 지정하고, 그리고 그 액세스 레지스터에 의해 명시되는 주소 공간 제어 엘리먼트는 참조를 위해 DAT에 의해 사용된다. 일부 명령들에 있어서, R 필드가 B 필드 대신에 사용된다. 상기 액세스 레지스터들의 콘텐츠를 로드하고 저장하기 위한 그리고 한 액세스 레지스터의 콘텐츠를 또 다른 액세스 레지스터로 이동시키기 위한 명령들이 제공된다.
- [0023] 액세스 레지스터들 1~15의 각각은 어느 주소 공간이든 지정할 수 있다. 액세스 레지스터 0은 1차 명령 공간을 지정한다. 액세스 레지스터들 1~15 중 하나가 주소 공간을 지정하는 데 사용될 때, CPU는 상기 액세스 레지스터의 콘텐츠를 변환함으로써 어느 주소 공간이 지정될 것인지를 결정한다. 액세스 레지스터 0이 주소 공간을 지정하는 데 사용될 때, CPU는 1차 명령 공간을 지정하는 것으로서 상기 액세스 레지스터를 취급하며, CPU는 상기 액세스 레지스터의 실제 콘텐츠를 조사(examine)하지는 않는다. 그러므로, 상기 16개 액세스 레지스터는 임의의 시간에 1차 명령 공간과 최대 15개의 다른 공간들을 지정할 수 있다.
- [0024] 한 실시 예에서, 다수 유형의 주소 공간들이 있다. 주소 공간은 연속적인 시퀀스의 정수 숫자들(가상 주소들)로서, 그와 함께 각 숫자가 스토리지 내 바이트 위치와 연관되도록 해주는 특정 변환 매개변수들(specific transformation parameters)을 포함한다. 상기 시퀀스는 0에서 시작하며 좌측에서 우측으로 진행된다.
- [0025] 예를 들어 z/Architecture에서, 가상 주소가 CPU에 의해 메인 스토리지(메인 메모리라고도 알려짐)에 액세스하는 데 사용될 때, 그 가상 주소는 동적 주소 변환(DAT)를 통해서 처음에 실 주소로 변환되고, 그 다음에 프리픽싱(prefixing)을 통해서 절대 주소로 변환된다. DAT는 1레벨부터 5레벨까지의 테이블들(페이지, 세그먼트, 영역 제3, 영역 제2, 및 영역 제1)을 변환 매개변수들로 사용할 수 있다. 특정 주소 공간에 대한 최고-레벨 테이블의 지정(designation)(기점과 길이(origin and length))은 주소 공간 제어 엘리먼트라 불리며, 그것은 DAT에 의해 사용되기 위해 제어 레지스터에서 발견되거나 또는 액세스 레지스터에 의해 명시된 대로 발견된다. 이와는 달리, 주소 공간에 대한 주소 공간 제어 엘리먼트는 실제 공간 지정(real space designation)일 수 있는데, 이것은 DAT가 단순히 가상 주소를 실 주소로 취급하고 어느 테이블도 사용하지 않음으로써 가상 주소를 변환하는 것을 표시한다.
- [0026] DAT는 여러 경우에 각각 다른 제어 레지스터들에 있는 주소 공간 제어 엘리먼트들을 사용하거나 액세스 레지스터들에 의해 명시되는 주소 공간 제어 엘리먼트들을 사용한다. 선택은 현재 PSW에 명시되는 변환 모드에 의해 결정된다. 다음의 4개 변환 모드를 이용할 수 있다: 1차 공간 모드, 2차 공간 모드, 액세스 레지스터 모드 및 홈 공간 모드(home space mode). 변환 모드에 따라서 각각 다른 주소 공간들이 주소지정될 수 있다.
- [0027] 어느 때든지 CPU가 1차 공간 모드 또는 2차 공간 모드에 있을 때, CPU는 두 주소 공간(1차 주소 공간과 2차 주

소 공간)에 속하는 가상 주소들을 변환할 수 있다. 어느 때든지 CPU가 액세스 레지스터 모드에 있을 때, CPU는 최대 16개의 주소 공간들(1차 주소 공간과 최대 15개의 AR-명시 주소 공간들)의 가상 주소들을 변환할 수 있다. 어느 때든지 CPU가 홈 공간 모드에 있을 때, CPU는 홈 주소 공간의 가상 주소들을 변환할 수 있다.

[0028] 1차 주소 공간은 1차 가상 주소들—이것들은 1차 주소 공간 제어 엘리먼트(ASCE)를 통해서 변환됨—로 구성되기 때문에 그렇게 식별된다. 유사하게, 2차 주소 공간은 2차 ASCE를 통해서 변환되는 2차 가상 주소들로 구성되고, AR 명시 주소 공간들은 AR 명시 ASCE들을 통해서 변환되는 AR 명시 가상 주소들로 구성되고, 그리고 홈 주소 공간은 홈 ASCE를 통해서 변환되는 가상 주소들로 구성된다. 1차 ASCE와 2차 ASCE는 제어 레지스터 1과 제어 레지스터 7에 각각 있다. AR 명시 ASCE들은 액세스-레지스터 변환(ART)이라 불리는 프로세스를 통해서 제어 레지스터들 2, 5 및 8을 이용하여 위치가 확인되는 ASN-제2-테이블 엔트리들에 있다. 홈 ASCE는 제어 레지스터 13에 있다.

[0029] 도 1을 참조하여 여기에서 기술된 트랜잭션 퍼실리티의 하나 또는 그 이상의 특징들을 포함 및 사용하기 위한 컴퓨팅 환경의 한 실시 예를 기술한다.

[0030] 도 1을 참조하면, 한 예에서, 컴퓨팅 환경(100)은 미국 뉴욕주 아몬크 소재 인터내셔널 비지네스 머신즈 코퍼레이션(IBM®)에 의해 공급되는 z/Architecture에 기초한다. z/Architecture는 "z/Architecture - Principles of Operation,"(간행물 번호 SA22-7932-08, 9판, 2010년 8월)라는 제목의 IBM 간행물에 기술되어 있다.

[0031] Z/ARCHITECTURE, IBM, Z/OS 및 Z/VM(아래에서 참조됨)은 미국 뉴욕주 아몬크 소재 인터내셔널 비지네스 머신즈 코퍼레이션의 등록 상표이다. 여기에서 사용되는 다른 명칭들도 인터내셔널 비지네스 머신즈 코퍼레이션 또는 다른 회사들의 등록 상표, 상표 또는 제품 명칭일 수 있다.

[0032] 한 예로서, 컴퓨팅 환경(100)은 하나 또는 그 이상의 제어 유닛들(108)을 통해서 하나 또는 그 이상의 입력/출력(I/O) 디바이스들(106)에 결합된 중앙 프로세서 복합체(CPC)(102)를 포함한다. 중앙 프로세서 복합체(102)는 예를 들어 하나 또는 그 이상의 중앙 프로세서들(110), 하나 또는 그 이상의 파티션들(112)(예를 들어, 논리적 파티션들(LP)), 논리적 파티션 하이퍼바이저(114), 및 입력/출력 서브시스템(115)을 포함하며, 이들의 각각은 아래에서 기술한다.

[0033] 중앙 프로세서들(110)은 논리적 파티션들에 할당되는 물리적인 프로세서 자원들이다. 구체적으로, 각 논리적 파티션(112)은 하나 또는 그 이상의 논리적 프로세서들을 보유하며, 이들 각각은 상기 파티션들에 할당되는 물리적인 프로세서(110)의 전부 또는 일부분을 나타낸다. 특정한 파티션(112)의 논리적 프로세서들은 그 파티션에 전용이어서 그 기본 프로세서 자원(110)이 그 파티션을 위해 유보되거나, 또는 다른 파티션과 공유되어서 그 기본 프로세서 자원이 다른 파티션에도 잠재적으로 이용 가능할 수 있다.

[0034] 논리적 파티션은 별개의 시스템으로 기능하고 하나 또는 그 이상의 애플리케이션들을 보유하며, 선택에 따라 그 안에 상주 운영체제를 보유하는데 이것은 각 논리적 파티션마다 다를 수 있다. 한 실시 예에서, 운영체제는 미국 뉴욕주 아몬크에 있는 인터내셔널 비지네스 머신즈 코퍼레이션에 의해 공급되는 z/OS 운영체제, z/VM 운영체제, z/Linux 운영체제, 또는 TPF 운영체제이다. 논리적 파티션들(112)은 논리적 파티션 하이퍼바이저(114)에 의해 관리되고, 이것은 프로세서들(110) 상에서 실행되는 펌웨어에 의해 구현된다. 여기에서 사용할 때, 펌웨어(firmware)는 예를 들어 프로세서의 마이크로코드(microcode) 및/또는 밀리코드(millicode)를 포함한다. 예를 들어, 펌웨어는 상위 레벨(higher level) 머신 코드의 구현에 사용되는 하드웨어-레벨 명령들 및/또는 데이터 구조들을 포함한다. 한 실시 예에서, 펌웨어는 예를 들어 통상적으로 마이크로코드로 전달되는 사유권 있는 코드(proprietary code)를 포함하며 이 마이크로코드는 신뢰 소프트웨어(trusted software) 또는 기본 하드웨어에 특화된 마이크로코드를 포함하고 운영체제가 시스템 하드웨어에 액세스하는 것을 제어한다.

[0035] 논리적 파티션들 및 논리적 파티션 하이퍼바이저 각각은 중앙 프로세서들과 연관된 중앙 스토리지의 각각의 파티션들에 상주하는 하나 또는 그 이상의 프로그램들을 포함한다. 논리적 파티션 하이퍼바이저(114)의 한 예는 미국 뉴욕주 아몬크 소재 인터내셔널 비지네스 머신즈 코퍼레이션에서 공급하는 프로세서 자원/시스템 매니저(PR/SM)이다.

[0036] 입력/출력 서브시스템(115)은 입력/출력 디바이스들(106)과 메인 스토리지(메인 메모리라고도 알려짐) 사이의 정보의 흐름을 지시한다. 이것(입력/출력 서브시스템)은 상기 중앙 처리 복합체의 일부 또는 그와 별개일 수 있다는 점에서 상기 중앙 처리 복합체에 결합된다(coupled). 상기 I/O 서브시스템은 상기 중앙 프로세서들로 하여금 상기 입력/출력 디바이스들과 직접 통신해야 하는 태스크를 덜어주고 데이터 처리가 입력/출력 처리와 동시에(concurrently) 진행되도록 허용한다. 통신을 제공하기 위해, I/O 서브시스템은 I/O 통신 어댑터들을 채용한

다. 통신 어댑터들에는 예를 들어 채널(channels), I/O 어댑터, PCI 카드, 이더넷 카드, SCSI(Small Computer Storage Interface) 카드 등의 여러 유형이 있다. 여기에 기술된 구체적인 예에서, I/O 통신 어댑터는 채널이고, 그러므로 I/O 서브시스템은 여기에서 채널 서브시스템으로 불린다. 하지만 이것은 단지 예시일 뿐이다. 다른 유형의 I/O 서브시스템들도 사용될 수 있다.

- [0037] I/O 서브시스템은 입력/출력 디바이스들(106) 안팎으로 정보의 흐름을 관리하는 데 하나 또는 그 이상의 입력/출력 경로들을 통신 링크들로서 사용한다. 이 구체적인 예에서, 통신 어댑터들이 채널들이므로 이 경로들은 채널 경로(channel paths)라 불린다.
- [0038] 위에 기술된 컴퓨팅 환경은 사용될 수 있는 컴퓨팅 환경의 한 예시일 뿐이다. 파티션되지 않은 환경들을 포함한 (그러나 그에 한정되지 않음) 다른 환경들, 다른 파티션된 환경들 및/또는 에뮬레이트된 환경들이 사용될 수 있고, 실시 예들은 어느 한 환경에 한정되지 않는다.
- [0039] 하나 또는 그 이상의 특징들에 따라서, 트랜잭션 실행 퍼실리티는 CPU가 트랜잭션으로 알려진 일련의 명령들을 실행할 수 있는 수단을 제공하는 CPU를 향상시켜주는 것이며, 상기 명령들은 다중 스토리지 위치들을 업데이트 하는 것을 포함하여 그 위치들을 액세스할 수 있다. 다른 CPU들과 I/O 서브시스템에 의해 관찰될 때, 트랜잭션은 (a) 전체가 단일 원자적 연산으로서 완료되거나, 또는 (b) 중단되어 (여기에 기술된 특정 조건들을 제외하고) 그것이 실행되었다는 증거를 잠재적으로 남기지 않는다. 그러므로, 성공적으로 완료된 트랜잭션은 고전적인 다중 처리 모델에서 필요한 특수 로킹(locking) 없이 다중 스토리지 위치들을 업데이트할 수 있다.
- [0040] 트랜잭션 실행 퍼실리티는 예를 들어 하나 또는 그 이상의 제어들(controls), 하나 또는 그 이상의 명령들, 제약 및 비제약 실행을 포함한 트랜잭션 처리, 및 중단 처리를 포함하며, 이들 각각은 아래에서 더 기술된다.
- [0041] 한 실시 예에서, 트랜잭션 중단 프로그램 상태 워드(PSW), 트랜잭션 진단 블록(TDB) 주소, 및 트랜잭션 내포 깊이를 포함한 세 개의 특수 목적 제어들; 다섯 개의 제어 레지스터 비트들; 및 TRANSACTION BEGIN(제약 및 비제약), TRANSACTION END, EXTRACT TRANSACTION NESTING DEPTH, TRANSACTION ABORT, 및 NONTRANSACTIONAL STORE를 포함한 여섯 개의 범용 명령들이 트랜잭션 실행 퍼실리티를 제어하는 데 사용된다. 상기 퍼실리티가 설치될 때, 상기 퍼실리티는 예를 들어 상기 구성(configuration) 내 모든 CPU들에 설치된다. 한 구현 예에서 비트 73인 퍼실리티 인디케이션(facility indication)이 1일 때 트랜잭션 실행 퍼실리티가 설치되어 있음을 표시한다.
- [0042] 트랜잭션 실행 퍼실리티가 설치되어 있을 때, 상기 구성은 비제약 트랜잭션 실행 퍼실리티를 제공하고, 선택에 따라 제약 트랜잭션 실행 퍼실리티를 제공하며, 이들 각각은 아래에서 기술한다. 퍼실리티 인디케이션 50과 퍼실리티 인디케이션 70이 예시로서 모두 1일 때, 제약 트랜잭션 실행 퍼실리티가 설치된다. 두 퍼실리티 인디케이션은 모두 메모리 내의 명시된 위치들에 저장된다.
- [0043] 여기에서 사용할 때, 명령의 이름인 TRANSACTION BEGIN은 연상 기호(mnemonic) TBEGIN(비제약 트랜잭션에 대한 트랜잭션 시작)과 연상 기호 TBEGINC(제약 트랜잭션에 대한 트랜잭션 시작)를 갖는 명령들을 나타낸다. 특정 명령에 관련된 논의들은 명령의 이름과 그 뒤에 붙은 괄호안의 연상 기호로 표시되거나, 단순히 연상 기호로 표시된다.
- [0044] TRANSACTION BEGIN(TBEGIN) 명령의 포맷의 한 실시 예를 도 2a~2b에 도시한다. 한 예로서, TBEGIN 명령(200)은 비제약 트랜잭션 시작 연산(transaction begin nonconstrained operation)을 명시하는 오피코드를 포함하는 오피코드 필드(202); 베이스 필드(B_1)(204); 변위 필드(D_1)(206); 및 즉시 필드(I_2)(208)를 포함한다. B_1 필드가 비제로(nonzero)일 때, B_1 (204)에 의해 명시되는 범용 레지스터의 콘텐츠가 D_1 (206)에 더해져 제1 오퍼랜드 주소를 획득한다.
- [0045] B_1 필드가 비제로일 때, 다음이 적용된다:
- [0046] • 트랜잭션 내포 깊이가 최초에 제로일 때, 제1 오퍼랜드 주소는 TBEGIN-명시 TDB(아래에서 더 기술함)라 불리는 256바이트 트랜잭션 진단 블록의 위치를 지정하며, 만일 이 트랜잭션이 중단될 경우 이 TBEGIN-명시 TDB 안으로 여러 진단 정보가 저장될 수 있다. CPU가 1차 공간 모드 또는 액세스 레지스터 모드에 있을 때, 제1 오퍼랜드 주소는 1차 주소 공간 내 위치를 지정한다. CPU가 2차 공간 또는 홈 공간 모드에 있을 때, 제1 오퍼랜드 주소는 2차 또는 홈 주소 공간 내 위치를 각각 지정한다. DAT가 오프(off)되어 있을 때, 트랜잭션 진단 블록(TDB) 주소(TDBA)는 실 스토리지(real storage) 내 위치를 지정한다.
- [0047] 제1 오퍼랜드에 대한 저장 액세스 가능성(store accessibility)이 판정된다. 만일 액세스 가능하면, 상

기 오퍼랜드의 논리 주소가 트랜잭션 진단 블록 주소(TDBA)에 배치되고, TDBA는 유효하다.

- [0048] • CPU가 이미 비제약 트랜잭션 실행 모드에 있을 때, TDBA는 수정되지 않고, 제1 오퍼랜드의 액세스 가능성이 테스트되었는지는 예측 불가능하다.
- [0049] 상기 B₁ 필드가 제로일 때, 제1 오퍼랜드에 대한 액세스 예외들은 검출되지 않고, 최외부 TBEGIN 명령에 대하여 TDBA는 유효하지 않다.
- [0050] 상기 I₂ 필드의 비트들은, 한 예에서, 다음과 같이 정의된다:
- [0051] **범용 레지스터 세이브 마스크(GRSM)(210)(도 2b):** I₂ 필드의 비트들 0~7은 범용 레지스터 세이브 마스크(GRSM, general register save mask)를 보유한다. GRSM의 각 비트는 범용 레지스터들의 짝수-홀수 쌍을 나타내고, 여기에서 비트 0은 레지스터 0과 레지스터 1을 나타내고, 비트 1은 레지스터 2와 레지스터 3을 나타내는 등의 방식이다. 최외부 TBEGIN 명령의 GRSM 내 비트가 제로일 때, 대응하는 레지스터 쌍은 세이브되지 않는다. 최외부 TBEGIN 명령의 GRSM 내 비트가 일(one)일 때, 대응하는 레지스터 쌍은 모델 종속적 위치(프로그램에 의해 직접 액세스할 수 없음)에 세이브된다.
- [0052] 만일 트랜잭션이 중단되면, 세이브된 레지스터 쌍들은 최외부 TBEGIN 명령이 실행되었을 때 자신들의 콘텐츠로 복원된다. 다른 모든(세이브되지 않은) 범용 레지스터들의 콘텐츠는 트랜잭션이 중단될 때 복원되지 않는다.
- [0053] 범용 레지스터 세이브 마스크는 최외부 TBEGIN을 제외하고 모든 TBEGIN들 상에서는 무시된다.
- [0054] **AR 수정 허용(A)(212):** I₂ 필드의 비트 12인 A 제어는 트랜잭션이 액세스 레지스터를 수정하도록 허용되는지 여부를 제어한다. 유효 AR 수정 허용 제어는 현재의 내포 레벨 및 모든 외부 레벨들에 대한 TBEGIN 명령 내 A 제어의 논리적 AND(논리곱)이다.
- [0055] 만일 유효 A 제어가 제로이면, 액세스 레지스터를 수정하기 위한 시도가 이루어질 경우 트랜잭션은 중단 코드 11(제한된 명령)로 중단될 것이다. 만일 유효 A 제어가 일이면, 액세스 레지스터가 (다른 중단 조건이 없이) 수정될 경우, 트랜잭션은 중단되지 않을 것이다.
- [0056] **부동 소수점 연산 허용(F)(214):** I₂ 필드의 비트 13인 F 제어는 트랜잭션이 명시된 부동 소수점 명령들을 실행하도록 허용되는지 여부를 제어한다. 유효 부동 소수점 연산 허용 제어는 현재의 내포 레벨 및 모든 외부 레벨들에 대한 TBEGIN 명령 내 F 제어의 논리적 AND(논리곱)이다.
- [0057] 만일 유효 F 제어가 제로이면, (a) 부동 소수점 명령을 실행하기 위한 시도가 이루어질 경우 트랜잭션은 중단 코드 11(제한된 명령)로 중단될 것이고, (b) 부동 소수점 제어 레지스터(FPCR)의 바이트 2에 있는 데이터 예외 코드(DXC)는 어떤 데이터 예외 프로그램 예외 조건에 의해서도 세트되지 않을 것이다. 만일 유효 F 제어가 일이면, (a) (다른 중단 코드 없이) 부동 소수점 명령을 실행하기 위한 시도가 이루어질 경우 트랜잭션은 중단되지 않을 것이고, (b) FPCR에 있는 DXC는 데이터 예외 프로그램 예외 조건에 의해서 세트될 수 있다.
- [0058] **프로그램 인터럽션 필터링 제어(PIFC)(216):** I₂ 필드의 비트들 14~15는 프로그램 인터럽션 필터링 제어(PIFC)이다. PIFC는 CPU가 트랜잭션 실행 모드에 있는 동안 발생하는 특정 클래스의 프로그램 예외 조건들(예를 들어, 주소지정 예외, 데이터 예외, 연산 예외, 보호 예외 등)이 인터럽션을 초래하는지를 제어한다.
- [0059] 유효 PIFC는 현재의 내포 레벨 및 모든 외부 레벨들에 대한 TBEGIN 명령 내 최고 값의 PIFC이다. 유효 PIFC가 제로일 때, 모든 프로그램 예외 조건들은 인터럽션을 초래한다. 유효 PIFC가 일(one)일 때, 트랜잭션 실행 클래스 1과 트랜잭션 실행 클래스 2를 갖는 프로그램 예외 조건들은 인터럽션을 초래한다. (각 프로그램 예외 조건은 그 예외의 심각도(severity)에 따라 적어도 하나의 트랜잭션 실행 클래스가 할당된다. 심각도는 트랜잭션의 반복된 실행 동안 복구 가능성과 운영체제가 그 인터럽션을 확인할 필요가 있는지에 기초한다.) 유효 PIFC가 이(two)일 때, 트랜잭션 실행 클래스 1을 갖는 프로그램 예외 조건들은 인터럽션을 초래한다. 3의 PIFC는 유보된다.
- [0060] I₂ 필드의 비트들 8~11(명령의 비트들 40~43)은 유보되며 제로들을 보유해야 하는데, 그렇지 않으면 프로그램은 앞으로 호환되도록 운영될 수 없다.
- [0061] 도 3a와 3b를 참조하여 제약 트랜잭션 시작(TBEGINC) 명령의 포맷의 한 실시 예를 기술한다. 한 예에서, TBEGINC 명령(300)은 제약 트랜잭션 시작 연산(transaction begin constrained operation)을 명시하는 오피코

드를 포함하는 오퍼코드 필드(302); 베이스 필드(B_1)(304); 변위 필드(D_1)(306); 및 즉시 필드(I_2)(308)를 포함한다. B_1 (304)에 의해 명시되는 범용 레지스터의 콘텐츠가 D_1 (306)에 더해져 제1 오퍼랜드 주소를 획득한다. 하지만, 제약 트랜잭션 시작 명령에서, 제1 오퍼랜드 주소가 스토리지에 액세스하는 데 사용되지는 않는다. 그 대신에, 상기 명령의 B_1 필드는 제로들을 포함하는데, 그렇지 않으면 명세 예외(specification exception)가 인지된다.

[0062] 한 실시 예에서, I_2 필드는 여러 제어들을 포함하며, 이들의 예가 도 3b에 도시된다.

[0063] 상기 I_2 필드의 비트들은, 한 예에서, 다음과 같이 정의된다:

[0064] **범용 레지스터 세이브 마스크(GRSM)(310):** I_2 필드의 비트들 0~7은 범용 레지스터 세이브 마스크(GRSM, general register save mask)를 보유한다. GRSM의 각 비트는 범용 레지스터들의 짝수-홀수 쌍을 나타내고, 여기에서 비트 0은 레지스터 0과 레지스터 1을 나타내고, 비트 1은 레지스터 2와 레지스터 3을 나타내는 등의 방식이다. GRSM 내 비트가 제로일 때, 대응하는 레지스터 쌍은 세이브되지 않는다. GRSM 내 비트가 일일 때, 대응하는 레지스터 쌍은 모델 종속적 위치(프로그램에 의해 직접 액세스할 수 없음)에 세이브된다.

[0065] 만일 트랜잭션이 중단되면, 세이브된 레지스터 쌍들은 최외부 TRANSACTION BEGIN 명령이 실행되었을 때 자신들의 콘텐츠로 복원된다. 다른 모든(세이브되지 않은) 범용 레지스터들의 콘텐츠는 제약 트랜잭션이 중단될 때 복원되지 않는다.

[0066] TBEGIN이 비제약 트랜잭션 모드에서 실행을 계속하기 위해 사용될 때, 범용 레지스터 세이브 마스크는 무시된다.

[0067] **AR 수정 허용(A)(312):** I_2 필드의 비트 12인 A 제어는 트랜잭션이 액세스 레지스터를 수정하도록 허용되는지 여부를 제어한다. 유효 AR-수정-허용 제어는 현재의 내포 레벨 및 모든 외부 TBEGIN 또는 TBEGINC 명령들에 대한 TBEGINC 명령 내 A 제어의 논리적 AND(논리곱)이다.

[0068] 만일 유효 A 제어가 제로이면, 액세스 레지스터를 수정하기 위한 시도가 이루어질 경우 트랜잭션은 중단 코드 11(제한된 명령)로 중단될 것이다. 만일 유효 A 제어가 일이면, 액세스 레지스터가 (다른 중단 조건이 없이) 수정될 경우, 트랜잭션은 중단되지 않을 것이다.

[0069] I_2 필드의 비트들 8~11과 비트들 13~15(명령의 비트들 40~43과 비트들 45~47)는 유보되며 제로들을 보유해야 한다.

[0070] 트랜잭션 시작 명령의 종료(end)는 TRANSACTION END(TEND) 명령에 의해 명시되고, 그 포맷이 도 4에 도시된다. 한 예로서, TEND 명령(400)은 트랜잭션 종료 연산을 명시하는 오퍼코드를 포함하는 오퍼코드 필드(402)를 포함한다.

[0071] 트랜잭션 실행 퍼실리티와 관련하여 다수의 용어들이 사용되고 있기에, 단지 편의상 아래에 일련의 용어들을 알파벳 순으로 제공한다. 한 실시 예에서, 이 용어들은 다음의 정의를 갖는다:

[0072] **중단하다(Abort):** 제로의 트랜잭션 내포 깊이를 초래하는 TRANSACTION END 명령이 있기 전에 종료될 때 트랜잭션은 중단된다. 트랜잭션이 중단될 때, 한 실시 예에서, 다음과 같은 일이 발생한다:

[0073] • 트랜잭션의 임의 및 모든 레벨들에 의해 이루어진 트랜잭션 저장 액세스(transactional store accesses)는 폐기된다(즉, 커밋되지 않는다).

[0074] • 트랜잭션의 임의 및 모든 레벨들에 의해 이루어진 비트랜잭션 저장 액세스(non-transactional store accesses)는 커밋된다.

[0075] • 최외부 TRANSACTION BEGIN 명령의 범용 레지스터 세이브 마스크(GRSM)에 의해 지정되는 레지스터들은 트랜잭션 실행 전의 자신들의 콘텐츠로(즉, 최외부 TRANSACTION BEGIN 명령의 실행시 자신들의 콘텐츠로) 복원된다. 최외부 TRANSACTION BEGIN 명령의 범용 레지스터 세이브 마스크에 의해 지정되지 않는 범용 레지스터들은 복원되지 않는다.

[0076] • 액세스 레지스터들, 부동 소수점 레지스터들, 및 부동 소수점 제어 레지스터는 복원되지 않는다. 트

랜잭션이 중단될 때 트랜잭션 실행 동안에 이 레지스터들에 이루어진 모든 변경 사항들은 유지된다.

- [0077] 트랜잭션은 제한된 명령의 실행 시도(attempted execution of a restricted instruction), 제한된 자원의 수정 시도(attempted modification of a restricted resource), 트랜잭션 충돌(transactional conflict), 여러 CPU 자원들의 초과(exceeding various CPU resources), 해석적-실행 인터셉션 조건, 인터럽션, TRANSACTION ABORT 명령, 및 기타 이유들을 포함한 여러 가지 이유들로 인해 중단될 수 있다. 트랜잭션-중단 코드는 트랜잭션이 중단되는 특정한 이유를 제공한다.
- [0078] 도 5를 참조하여 TRANSACTION ABORT(TABORT) 명령의 포맷의 한 예를 기술한다. 한 예로서, TABORT 명령(500)은 트랜잭션 중단 연산을 명시하는 오퍼코드를 포함하는 오퍼코드 필드(502); 베이스 필드(B_2)(504); 변위 필드(D_2)(506)를 포함한다. B_2 필드가 비제로일 때, B_2 (504)에 의해 명시되는 범용 레지스터의 콘텐츠가 D_2 (506)에 더해져 제2 오퍼랜드 주소를 획득하며, 그렇지 않으면 제2 오퍼랜드 주소는 단지 D_2 필드로부터만 형성되고 B_2 필드는 무시된다. 제2 오퍼랜드 주소는 데이터를 주소지정하는 데 사용되지 않으며, 대신에 그 주소는 트랜잭션 중단 코드를 형성하며, 이것은 중단 처리 동안에 트랜잭션 진단 블록에 배치된다. 제2 오퍼랜드 주소를 위한 주소 계산은 다음의 주소 산술 규칙들을 따른다: 24-비트 주소지정 모드에서 비트들 0~29는 제로들로 세트되고, 31-비트 주소지정 모드에서 비트들 0~32는 제로들로 세트된다.
- [0079] **커밋하다(Commit):** 최외부 TRANSACTION END 명령의 완료시에, CPU는 상기 트랜잭션(즉, 최외부 트랜잭션 및 모든 내포 레벨들)에 의해 이루어진 저장 액세스들을 커밋하여 그들을 다른 CPU들과 I/O 서브시스템이 볼 수 있게 만든다. 다른 CPU들에 의해 그리고 I/O 서브시스템에 의해 관찰될 때, 트랜잭션의 모든 내포 레벨들에 의해 이루어지는 모든 페치 및 저장 액세스들은 커밋이 일어날 때 단일한 동시 연산(single concurrent operation)으로서 이루어지는 것으로 보인다.
- [0080] 범용 레지스터들, 액세스 레지스터들, 부동 소수점 레지스터들, 및 부동 소수점 제어 레지스터의 콘텐츠는 커밋 프로세스에 의해 수정되지 않는다. 트랜잭션의 저장들이 커밋될 때 트랜잭션 실행 동안에 이 레지스터들에 이루어진 모든 변경 사항들은 유지된다.
- [0081] **충돌하다(Conflict):** 한 CPU에 의해 이루어지는 트랜잭션 액세스는 (a) 다른 CPU에 의해 이루어지는 트랜잭션 액세스 또는 비트랜잭션 액세스와 또는 (b) I/O 서브시스템에 의해 이루어지는 비트랜잭션 액세스와 충돌한다 (만일 두 액세스들이 동일한 캐시 라인 내의 임의 위치에 대한 것이고, 액세스들의 하나 또는 그 이상이 저장일 경우).
- [0082] 개념적인 순서에서는 충돌이 검출되지 않을 수 있다 하더라도, CPU가 명령들을 추론적으로 실행함에 의해 충돌이 검출될 수 있다.
- [0083] **제약 트랜잭션(Constrained Transaction):** 제약 트랜잭션은 제약 트랜잭션 실행 모드에서 실행되는 트랜잭션이며 다음의 제한사항들이 적용된다:
- [0084] • 일반적인 명령들의 서브세트(subset)를 이용할 수 있다.
 - [0085] • 한정된 수의 명령들이 실행될 수 있다.
 - [0086] • 한정된 수의 스토리지-오퍼랜드 위치들을 액세스할 수 있다.
 - [0087] • 트랜잭션은 단일 내포 레벨로 한정된다.
- [0088] 반복되는 인터럽션들 또는 다른 CPU들 또는 I/O 서브시스템과의 충돌들이 없는 경우에, 제약 트랜잭션은 최종적으로 완료되므로, 중단-처리부(abort-handler) 루틴은 필요하지 않다. 제약 트랜잭션들은 아래에 상세하게 기술한다.
- [0089] CPU가 이미 비제약 트랜잭션 실행 모드에 있는 동안 제약 TRANSACTION BEGIN(TBEGINC) 명령이 실행될 때, 실행은 내포된 비제약 트랜잭션으로서 계속된다.
- [0090] **제약 트랜잭션 실행 모드(Constrained Transactional Execution Mode):** 트랜잭션 내포 깊이가 제로이고 트랜잭션이 TBEGINC 명령에 의해 개시될 때, CPU는 제약 트랜잭션 실행 모드로 진입한다. CPU가 제약 트랜잭션 실행 모드에 있는 동안, 트랜잭션 내포 깊이는 일(one)이다.

- [0091] **내포된 트랜잭션(Nested Transaction):** CPU가 비제약 트랜잭션 실행 모드에 있는 동안 TRANSACTION BEGIN 명령이 발행될 때, 트랜잭션은 내포된다(nested).
- [0092] 트랜잭션 실행 퍼실리티는 평탄화된 내포(flattened nesting)라 불리는 모델을 사용한다. 평탄화된 내포 모드에서, 최외부 트랜잭션이 자신의 저장들을 커밋할 때까지는 내부 트랜잭션에 의해 이루어지는 저장들은 다른 CPU들 및 I/O 서브시스템이 관찰할 수 없다. 유사하게, 만일 트랜잭션이 중단되면, 모든 내포된 트랜잭션들은 중단되고, 모든 내포된 트랜잭션들의 모든 트랜잭션 저장들은 폐기된다.
- [0093] 내포된 트랜잭션들의 한 예가 도 6에 도시된다. 도시된 바와 같이, 제1 TBEGIN(600)이 최외부 트랜잭션(601)을 시작하고, TBEGIN(602)이 제1 내포된 트랜잭션을 시작하고, TBEGIN(604)이 제2 내포된 트랜잭션을 시작한다. 이 예에서, TBEGIN(604)과 TEND(606)는 최내부(innermost) 트랜잭션(608)을 정의한다. TEND(610)가 실행될 때, 트랜잭션 저장들은 최외부 트랜잭션과 모든 내부 트랜잭션들에 대하여 커밋된다(612).
- [0094] **비제약 트랜잭션(Nonconstrained Transaction):** 비제약 트랜잭션은 비제약 트랜잭션 실행 모드에서 실행되는 트랜잭션이다. 비제약 트랜잭션은 제약 트랜잭션 같은 방식으로 제한되지는 않을지라도, 다양한 원인들로 인해 중단될 수도 있다.
- [0095] **비제약 트랜잭션 실행 모드(Nonconstrained Transactional Execution Mode):** 트랜잭션이 TBEGIN 명령에 의해 개시될 때, CPU는 비제약 트랜잭션 실행 모드로 진입한다. CPU가 비제약 트랜잭션 실행 모드에 있는 동안, 트랜잭션 내포 깊이는 1에서 최대 트랜잭션 내포 깊이까지 변할 수 있다.
- [0096] **비트랜잭션 액세스(Non-Transactional Access):** 비트랜잭션 액세스는 CPU가 트랜잭션 실행 모드에 있지 않을 때 CPU에 의해 이루어지는 스토리지 오퍼랜드 액세스(즉, 트랜잭션 밖의 고전적인 스토리지 액세스)이다. 추가로, I/O 서브시스템에 의해 이루어지는 액세스도 비트랜잭션 액세스이다. 추가로, NONTRANSACTIONAL STORE 명령은 CPU가 비제약 트랜잭션 실행 모드에 있는 동안 비트랜잭션 저장 액세스를 일으키는 데 사용될 수 있다.
- [0097] 도 7을 참조하여 NONTRANSACTIONAL STORE 명령의 포맷의 한 실시 예를 기술한다. 한 예로서, NONTRANSACTIONAL STORE 명령(700)은 비트랜잭션 저장 연산을 지정하는 오퍼코드를 명시하는 복수의 오퍼코드 필드(702a, 702b); 레지스터를 명시하는 레지스터 필드(R_1)(704)—이것의 콘텐츠는 제1 오퍼랜드라 불림—; 인덱스 필드(X_2)(706); 베이스 필드(B_2)(708); 제1 변위 필드(DL_2)(710); 및 제2 변위 필드(DH_2)(712)를 포함한다. X_2 필드와 B_2 필드에 의해 지정되는 범용 레지스터들의 콘텐츠가 DH_2 필드와 DL_2 필드의 콘텐츠가 접합(concatenation)된 콘텐츠에 더해져 제2 오퍼랜드 주소를 형성한다. X_2 필드 또는 B_2 필드 중 하나 또는 모두가 제로일 때, 대응하는 레지스터는 상기 덧셈에 참여하지 않는다.
- [0098] 64비트 제1 오퍼랜드는 제2 오퍼랜드 위치에 변경되지 않은 채 비트랜잭션적으로 배치된다.
- [0099] DH_2 필드와 DL_2 필드의 접합(concatenation)에 의해 형성되는 변위는 20비트 부호화된 2진 정수로 취급된다.
- [0100] 제2 오퍼랜드는 더블워드(double word) 경계 상에 정렬(align)되어야 하며, 그렇지 않으면 명세 예외가 인지되고 그 연산은 억제된다.
- [0101] **외부/최외부 트랜잭션(Outer/Outermost Transaction):** 낮은 수의 트랜잭션 내포 깊이를 갖는 트랜잭션이 외부 트랜잭션(outer transaction)이다. 일(one)의 트랜잭션 내포 깊이 값을 갖는 트랜잭션은 최외부 트랜잭션(outermost transaction)이다.
- [0102] 최외부 TRANSACTION BEGIN 명령은 트랜잭션 내포 깊이가 최초로 제로일 때 실행되는 것이다. 최외부 TRANSACTION END 명령은 트랜잭션 내포 깊이가 일에서 제로로 변하게 하는 것이다. 제약 트랜잭션은 이 실시 예에서 최외부 트랜잭션이다.
- [0103] **프로그램 인터럽션 필터링(Program Interruption Filtering):** 트랜잭션이 어떤 프로그램 예외 조건들로 인해 중단될 때, 프로그램은 선택에 따라 인터럽션의 발생을 막을 수 있다. 이 기술이 프로그램-인터럽션 필터링(program-interruption filtering)이라 불린다. 프로그램 인터럽션 필터링은 인터럽션의 트랜잭션 클래스, TRANSACTION BEGIN 명령으로부터의 유효 프로그램 인터럽션 필터링 제어, 및 제어 레지스터 0 내 트랜잭션 실행 프로그램 인터럽션 필터링 오버라이드(override)의 적용을 받는다.
- [0104] **트랜잭션(Transaction):** 트랜잭션은 CPU가 트랜잭션 실행 모드에 있는 동안 스토리지-오퍼랜드 액세스들이 이루어지는 것과, 선택된 범용 레지스터들이 변경되는 것을 포함한다. 비제약 트랜잭션에서, 스토리지-오퍼랜드 액

세스는 트랜잭션 액세스와 비트랜잭션 액세스를 모두 포함한다. 제약 트랜잭션에서, 스토리지-오퍼랜드 액세스는 트랜잭션 액세스로 한정된다. 다른 CPU들과 I/O 서브시스템에 의해 관찰될 때, 트랜잭션 실행 모드에 있는 동안 CPU에 의해 이루어지는 모든 스토리지-오퍼랜드 액세스는 단일한 동시 연산으로서 이루어지는 것으로 보인다. 만일 트랜잭션이 중단되면, 트랜잭션 저장 액세스들은 폐기되고, 최외부 TRANSACTION BEGIN 명령의 범용 레지스터 세이프 마스크에 의해 지정된 레지스터들은 트랜잭션 실행 전의 자신들의 콘텐츠로 복원된다.

- [0105] **트랜잭션 액세스(Transaction Accesses):** 트랜잭션 액세스는 NONTRANSACTIONAL STORE 명령에 의해 이루어지는 액세스를 제외하고는 CPU가 트랜잭션 실행 모드에 있는 동안 이루어지는 스토리지 오퍼랜드 액세스이다.
- [0106] **트랜잭션 실행 모드(Transaction Execution Mode):** 트랜잭션 실행 모드(transactional execution mode, 또한 transaction execution mode라고도 알려짐)라는 말은 비제약 트랜잭션 실행 모드 및 제약 트랜잭션 실행 모드 둘 모두의 공통 연산을 기술한다. 그러므로, 상기 연산이 기술될 때, 비제약과 제약이라는 말은 트랜잭션 실행 모드를 수식하는 데 사용된다.
- [0107] 트랜잭션 내포 깊이가 제로일 때, CPU는 트랜잭션 실행 모드에 있지 않다(또한 비트랜잭션 실행 모드라 불림).
- [0108] CPU에 의해 관찰될 때, 트랜잭션 실행 모드에서 이루어지는 폐치들과 저장들은 트랜잭션 실행 모드에 있지 않은 동안 이루어지는 것들과 다르지 않다.
- [0109] z/Architecture의 한 실시 예에서, 트랜잭션 실행 퍼실리티는 제어 레지스터 0의 비트들 8~9, 제어 레지스터 2의 비트들 61~63, 트랜잭션 내포 깊이, 트랜잭션 진단 블록 주소, 및 트랜잭션 중단 프로그램 상태 워드(PSW)의 제어를 받는다.
- [0110] 초기 CPU 리셋에 이어서, 제어 레지스터 0의 비트 위치들 8~9, 제어 레지스터 2의 비트 위치들 62~63, 및 트랜잭션 내포 깊이의 콘텐츠는 제로로 세트된다. 트랜잭션 실행 제어인 제어 레지스터 0의 비트 8이 제로일 때, CPU는 트랜잭션 실행 모드로 배치될 수 없다.
- [0111] 여러 제어들(controls)에 관한 더 상세한 사항은 아래에 기술한다.
- [0112] 표시된 바와 같이, 트랜잭션 실행 퍼실리티는 제어 레지스터 0 내의 2개 비트와 제어 레지스터 2 내의 3개 비트에 의해 제어된다. 예를 들면 다음과 같다:
- [0113] **제어 레지스터 0의 비트들(Control Register 0 Bits):** 한 실시 예에서, 비트 할당은 다음과 같다:
- [0114] 트랜잭션 실행 제어(Transaction Execution Control, TXC): 제어 레지스터 0의 비트 8은 트랜잭션 실행 제어이다. 이 비트는 제어 프로그램(예를 들어, 운영체제)이 트랜잭션 실행 퍼실리티가 프로그램에 의해 사용 가능한지 아닌지를 표시할 수 있는 메커니즘을 제공한다. 트랜잭션 실행 모드로 성공적으로 진입하기 위해 서는 비트 8이 일(one)이어야 한다.
- [0115] 제어 레지스터 0의 비트 8이 제로일 때, EXTRACT TRANSACTION NESTING DEPTH 명령과 TRANSACTION BEGIN 명령 및 TRANSACTION END 명령의 실행 시도 결과는 특수 연산의 실행이 된다.
- [0116] 도 8을 참조하여 EXTRACT TRANSACTION NESTING DEPTH(트랜잭션 내포 깊이 추출) 명령의 포맷의 한 실시 예를 기술한다. 한 예로서, EXTRACT TRANSACTION NESTING DEPTH 명령(800)은 트랜잭션 내포 깊이 추출 연산을 표시하는 오피코드를 명시하는 오피코드 필드(802); 및 범용 레지스터를 지정하는 레지스터 필드 R₁(804)을 포함한다.
- [0117] 현재의 트랜잭션 내포 깊이는 범용 레지스터 R₁의 비트들 48~63에 배치된다. 상기 레지스터의 비트들 0~31은 변하지 않은 채로 남아 있고, 상기 레지스터의 비트들 32~47은 제로로 세트된다.
- [0118] 추가 실시 예에서, 최대 트랜잭션 내포 깊이 또한 범용 레지스터 R₁에 배치되는데, 예를 들면 비트들 16~31에 배치된다.
- [0119] 트랜잭션 실행 프로그램 인터럽션 필터링 오버라이드(PIFO): 제어 레지스터 0의 비트 9가 트랜잭션 실행 프로그램 인터럽션 필터링 오버라이드이다. 이 비트는 제어 프로그램이, CPU가 트랜잭션 실행 모드에 있는 동안 발생하는 모든 프로그램 예외 조건은 TRANSACTION BEGIN 명령(들)에 의해 명시 또는 암시되는 유효 프로그램 인터럽션 필터링 제어와 상관 없이 인터럽션을 초래한다는 것을, 보장할 수 있는 메커니즘을 제공한다.
- [0120] **제어 레지스터 2의 비트들(Control Register 2 Bits):** 한 실시 예에서, 상기 할당은 다음과 같다:

- [0121] 트랜잭션 진단 범위(Transaction Diagnostic Scope, TDS): 제어 레지스터 2의 비트 61은 동 레지스터의 비트들 62~63 내의 트랜잭션 진단 제어(TDC)의 적용성(applicability)을 제어한다.
- [0122] TDS
- [0123] 값 의미
- [0124] 0 TDC는 CPU가 문제 상태(problem state) 또는 슈퍼바이저 상태(supervisor state)에 있는지에 상관없이 적용된다.
- [0125] 1 TDC는 CPU가 문제 상태에 있을 때만 적용된다. CPU가 슈퍼바이저 상태에 있을 때, 처리는 TDC가 제로를 보유한 것처럼 된다.
- [0126] 트랜잭션 진단 제어(Transaction Diagnostic Control, TDC): 제어 레지스터 2의 비트들 62~63은 트랜잭션들이 진단 목적들에 따라서 랜덤으로 중단되도록 하는데 사용될 수 있는 2-비트 무부호 정수이다. 한 예에서, TDC의 인코딩은 다음과 같다:
- [0127] TDC
- [0128] 값 의미
- [0129] 0 정상 연산; 트랜잭션들은 TDC의 결과로 중단되지 않는다.
- [0130] 1 모든 트랜잭션을 랜덤 명령(random instruction)에 따라 중단시키지만, 최외부 TRANSACTION END 명령의 실행 전에 이루어진다.
- [0131] 2 랜덤 명령에 따라 랜덤 트랜잭션들을 중단한다.
- [0132] 3 유보됨(Reserved)
- [0133] 트랜잭션이 비제로 TDC로 인해 중단될 때, 그 다음에 다음 둘 중 하나가 발생할 수 있다:
- [0134] • 중단 코드가 코드들 7~11, 13~16, 또는 255 중 어느 한 가지에 세트되며, 상기 코드의 값은 CPU에 의해 랜덤으로 선택된다; 조건 코드는 상기 중단 코드에 대응하여 세트된다. 중단 코드들은 아래에 더 기술된다.
- [0135] • 비제약 트랜잭션에서, 조건 코드는 일(one)로 세트된다. 이 경우에, 중단 코드는 적용되지 않는다.
- [0136] TDC 값 1이 구현되는지 여부는 모델 종속적이다. 만일 구현되지 않으면, 1의 값은 2가 명시된 것처럼 작동한다.
- [0137] 제약 트랜잭션에서, 1의 TDC 값은 2의 TDC 값이 명시된 것처럼 취급된다.
- [0138] 만일 3의 TDC 값이 명시되면, 결과는 예측 불가능하다.
- [0139] **트랜잭션 진단 블록 주소(Transaction Diagnostic Block Address, TDBA)**
- [0140] 유효한 트랜잭션 진단 블록 주소(TDBA)는 최외부 TRANSACTION BEGIN(TBEGIN) 명령의 B₁ 필드가 비제로일 때 상기 최외부 TBEGIN 명령의 제1 오퍼랜드 주소로부터 세트된다. CPU가 1차 공간 모드 또는 액세스 레지스터 모드에 있을 때, TDBA는 1차 주소 공간 내 위치를 지정한다. CPU가 2차 공간 모드 또는 홈 공간 모드에 있을 때, TDBA는 2차 또는 홈 주소 공간 내 위치를 각각 지정한다. DAT(Dynamic Address Translation)가 오프(off)되어 있을 때, TDBA는 실 스토리지(real storage) 내 위치를 지정한다.
- [0141] TDBA는, 트랜잭션이 이어서(subsequently) 중단되면, TBEGIN-명시 TDB라 불리는 트랜잭션 진단 블록을 찾기 위해 CPU에 의해 사용된다. TDBA의 최우측 3개 비트는 제로이며, 이것은 TBEGIN-명시 TDB가 더블워드 경계 상에 있다는 것을 의미한다.
- [0142] 최외부 TRANSACTION BEGIN(TBEGIN) 명령의 B₁ 필드가 제로일 때, 트랜잭션 진단 블록 주소는 유효하지 않으며, 트랜잭션이 이어서 중단되면 TBEGIN-명시 TDB는 저장되지 않는다.
- [0143] **트랜잭션 중단 PSW(Transaction Abort PSW, TAPSW)**
- [0144] 내포 깊이가 최초로 제로일 때 TRANSACTION BEGIN(TBEGIN) 명령의 실행 동안에, 트랜잭션 중단 PSW는 현재의 PSW의 콘텐츠로 세트되고, 트랜잭션 중단 PSW의 명령 주소는 다음 순차 명령(즉, 상기 최외부 TBEGIN 다음에 오

는 명령)을 지정한다. 내포 깊이가 최초로 제로일 때 제약 TRANSACTION BEGIN(TBEGINC) 명령의 실행 동안에, 트랜잭션 중단 PSW의 명령 주소가 (TBEGINC 다음에 오는 다음 순차 명령 대신에) TBEGINC 명령을 지정하는 것을 제외하고는, 트랜잭션 중단 PSW는 현재의 PSW의 콘텐츠로 세트된다.

[0145] 트랜잭션이 중단될 때, 트랜잭션 중단 PSW 내 조건 코드는 상기 중단 조건의 심각도를 표시하는 코드로 대체된다. 이어서, 만일 트랜잭션이 인터럽션으로 귀결되지 않는 원인들로 인해 중단되었다면, PSW는 트랜잭션 중단 PSW로부터 로드되고; 만일 트랜잭션이 인터럽션으로 귀결되는 원인들로 인해 중단되었다면, 트랜잭션 중단 PSW는 인터럽션의 구(old) PSW로서 저장된다.

[0146] 트랜잭션 중단 PSW는 임의의 내부 TRANSACTION BEGIN 명령의 실행 동안에 변경되지 않는다.

[0147] **트랜잭션 내포 깊이(Transaction Nesting Depth, TND)**

[0148] 트랜잭션 내포 깊이는 예를 들어 16-비트 무부호 값인데, 이 값은 TRANSACTION BEGIN 명령이 조건 코드 0으로 완료될 때마다 증분되고(incremented) TRANSACTION END 명령이 완료될 때마다 감분된다(decremented). 트랜잭션 내포 깊이는 트랜잭션이 중단될 때 또는 CPU 리셋에 의해 제로로 리셋된다.

[0149] 한 실시 예에서, 15의 최대 TND가 구현된다.

[0150] 한 구현 예에서, CPU가 제약 트랜잭션 실행 모드에 있을 때, 트랜잭션 내포 깊이는 일(one)이다. 추가로, 최대 TND는 4-비트 값으로서 표시될 수 있지만, TND는 트랜잭션 진단 블록에서 자신의 검사(inspection)를 용이하게 하기 위해 16-비트 값으로 정의된다.

[0151] **트랜잭션 진단 블록(TDB)**

[0152] 트랜잭션이 중단될 때, 여러 상태 정보가 트랜잭션 진단 블록(TDB) 내에 다음과 같이 세이브될 수 있다:

[0153] 1. **TBEGIN-명시 TDB:** 비제약 트랜잭션에서, 최외부 TBEGIN 명령의 B_1 필드가 비제로일 때, 상기 명령의 제1 오퍼랜드 주소는 TBEGIN-명시 TDB를 지정한다. 이것은 애플리케이션 프로그램 명시 위치이며, 이 위치는 상기 애플리케이션의 중단 처리부(abort handler)에 의해 조사(examine)될 수 있다.

[0154] 2. **프로그램-인터럽션(PI) TDB:** 만일 비제약 트랜잭션이 필터링되지 않은(non-filtered) 프로그램 예외 조건으로 인해 중단되거나, 또는 만일 제약 트랜잭션이 임의의 프로그램 예외 조건(즉, 프로그램 인터럽션 인지로 귀결되는 모든 조건)으로 인해 중단되면, PI-TDB는 프리픽스 영역(prefix area) 내 위치들로 저장된다. 이것은 운영체제가 검사하는 데 그리고 그것이 제공할 수 있는 모든 진단 보고(any diagnostic reporting)에서 로그 아웃하는 데 이용 가능하다.

[0155] 3. **인터셉션(Interception) TDB:** 만일 트랜잭션이 인터셉션으로 귀결되는 임의의 프로그램 예외 조건으로 인해 중단되면(즉, 상기 조건은 해석적 실행이 종료되게 하고 제어가 호스트 프로그램으로 반환되게 한다), TDB는 게스트 운영체제에 대하여 상태 묘사 블록(state description block)에서 명시되는 위치로 저장된다.

[0156] TBEGIN-명시 TDB는 한 실시 예에서 TDB 주소가 유효할 때(즉, 최외부 TBEGIN 명령의 B_1 필드가 비제로일 때)에만 저장된다.

[0157] 필터링되지 않은 프로그램 예외 조건들로 인한 중단들에서는, PI-TDB 또는 인터셉션 TDB 중 하나만 저장될 것이다. 그러므로, 중단에서는 0개, 1개, 또는 2개 TDB가 저장될 수 있다.

[0158] 상기 TDB들의 각각의 한 예와 관련된 추가 상세 사항이 아래에 기술된다:

[0159] **TBEGIN-명시 TDB:** 유효한 트랜잭션 진단 블록 주소에 의해 명시되는 256-바이트 위치. 트랜잭션 진단 블록 주소가 유효할 때, TBEGIN-명시 TDB는 트랜잭션 중단시에 저장된다. TBEGIN-명시 TDB는 최외부 TRANSACTION BEGIN 명령의 실행시에 효력이 있는 모든 스토리지 보호 메커니즘의 적용을 받는다. TBEGIN-명시 TDB의 임의 부분의 PER(Program Event Recording) 스토리지 변경 이벤트는 트랜잭션 중단 처리 동안이 아니라 최외부 TBEGIN의 실행 동안에 검출된다.

[0160] PER의 한 가지 목적은 프로그램을 디버깅하는 것을 돕는 것이다. 이것은 프로그램이 다음의 이벤트 유형들을 보고받을 수 있게 하며, 예는 다음과 같다:

[0161] • 성공적인 분기 명령의 실행. 분기 타겟 위치가 지정된 스토리지 영역(storage area) 내에 있을 때

만 이벤트가 발생하게 하는 옵션이 제공된다.

[0162] • 지정된 스토리지 영역으로부터 명령의 폐지.

[0163] • 지정된 스토리지 영역의 콘텐츠의 변경. 상기 스토리지 영역이 지정된 주소 공간들 내에 있을 때만 이벤트가 발생하게 하는 옵션이 제공된다.

[0164] • STORE USING REAL ADDRESS 명령의 실행.

[0165] • TRANSACTION END 명령의 실행.

[0166] 프로그램은, STORE USING REAL ADDRESS에 대한 이벤트가 스토리지 변경 이벤트와 함께만 명시될 수 있는 것을 제외하고는, 하나 또는 그 이상의 상기 이벤트 유형들이 인지되도록 선택적으로 명시할 수 있다. PER 이벤트에 관한 정보가 프로그램 인터럽션에 의하여 프로그램에 제공되며, 상기 인터럽션의 원인은 인터럽션 코드에서 식별된다.

[0167] 트랜잭션 진단 블록 주소가 유효하지 않을 때, TBEGIN-명시 TDB는 저장되지 않는다.

[0168] **프로그램-인터럽션 TDB:** 실 위치들 6,144~6,399(1800~18FF hex). 프로그램 인터럽션 TDB는 트랜잭션이 프로그램 인터럽션으로 인해 중단될 때 저장된다. 트랜잭션이 다른 원인들로 인해 중단될 때, 프로그램 인터럽션 TDB의 콘텐츠는 예측 불가능하다.

[0169] 프로그램 인터럽션 TDB는 어느 보호 메커니즘의 적용도 받지 않는다. PER 스토리지 변경 이벤트들은 프로그램 인터럽션 TDB가 프로그램 인터럽션 동안에 저장될 때 프로그램 인터럽션 TDB에 대하여 검출되지 않는다.

[0170] **인터셉션(Interception) TDB:** 상태 묘사(state description)의 위치들 488~495에 의해 명시되는 256-바이트 호스트 실 위치. 인터셉션 TDB는 중단된 트랜잭션이 게스트 프로그램 인터럽션 인터셉션을 일으킬 때(즉, 인터셉션 코드 8) 저장된다. 트랜잭션이 다른 원인들로 인해 중단될 때, 인터셉션 TDB의 콘텐츠는 예측 불가능하다. 인터셉션 TDB는 어느 보호 메커니즘의 적용도 받지 않는다.

[0171] 도 9에 도시된 바와 같이, 트랜잭션 진단 블록(900)의 필드들은 한 실시 예에서 다음과 같다:

[0172] 포맷(902): 바이트 0은 다음과 같이 유효성(validity)과 포맷 인디케이션(format indication)을 보유한다:

[0173]	값	의미
[0174]	0	TDB의 나머지 필드들은 예측 불가능하다.
[0175]	1	포맷-1 TDB, 이것의 나머지 필드들은 아래에 기술된다.
[0176]	2~255	유보됨

[0177] 포맷 필드가 제로인 TDB는 널(null) TDB라 불린다.

[0178] 플래그들(904): 바이트 1은 다음과 같은 여러 인디케이션들을 보유한다:

[0179] 충돌 토큰 유효성(Conflict Token Validity, CTV): 트랜잭션이 폐지 충돌 또는 저장 충돌(즉, 각각 중단 코드 9 또는 중단 코드 10)로 인해 중단될 때, 바이트 1의 비트 0은 충돌 토큰 유효성 인디케이션(conflict token validity indication)이다. CTV 인디케이션이 일(one)일 때, TDB의 바이트들 16~23 내 충돌 토큰(910)은 충돌이 검출된 논리 주소를 보유한다. CTV 인디케이션이 제로일 때, TDB의 바이트들 16~23은 예측 불가능하다.

[0180] 트랜잭션이 폐지 충돌 또는 저장 충돌 이외의 이유로 인해 중단될 때, 바이트 1의 비트 0은 제로로 저장된다.

[0181] 제약-트랜잭션 인디케이션(CTI): CPU가 제약 트랜잭션 실행 모드에 있을 때, 바이트 1의 비트 1은 일(one)로 세트된다. CPU가 비제약 트랜잭션 실행 모드에 있을 때, 바이트 1의 비트 1은 제로로 세트된다.

[0182] 유보됨: 바이트 1의 비트들 2~7은 유보되고, 제로들로 저장된다.

[0183] 트랜잭션 내포 깊이(TND)(906): 바이트들 6~7은 트랜잭션이 중단되었을 때 트랜잭션 내포 깊이를 보유

한다.

- [0184] 트랜잭션 중단 코드(TAC)(908): 바이트들 8~15는 64-비트 무부호 트랜잭션 중단 코드를 보유한다. 각 코드 포인트는 트랜잭션이 중단되는 이유를 표시한다.
- [0185] 트랜잭션이 프로그램 인터럽션 이외의 조건들로 인해 중단될 때 트랜잭션 중단 코드가 프로그램 인터럽션 TDB에 저장되는지는 모델 종속적이다.
- [0186] 충돌 토큰(910): 페치 충돌 또는 저장 충돌(즉, 각각 중단 코드 9 또는 중단 코드 10)로 인해 중단되는 트랜잭션들에서, 바이트들 16~23은 충돌이 검출된 스토리지 위치의 논리 주소를 보유한다. 충돌 토큰은 바이트 1의 비트 0인 CTV 비트가 일(one)일 때 의미가 있다.
- [0187] CTV 비트가 제로일 때, 바이트들 16~23은 예측 불가능하다.
- [0188] CPU에 의한 추론적 실행 때문에, 충돌 토큰은 트랜잭션의 개념적인 실행 순서에 의해 반드시 액세스될 필요는 없는 스토리지 위치를 지정할 수 있다.
- [0189] 중단된 트랜잭션 명령 주소(ATIA)(912): 바이트들 24~31은 중단이 검출되었을 때 실행중인 명령을 식별하는 명령 주소를 보유한다. 트랜잭션이 중단 코드들 2, 5, 6, 11, 13, 또는 256 또는 그 이상의 코드로 인해 중단될 때, 또는 트랜잭션이 중단 코드들 4 또는 13으로 인해 중단되고 프로그램 예외 조건이 무효화될(nullifying) 때, ATIA는 실행중이었던 명령을 직접 가리킨다. 트랜잭션이 중단 코드들 4 또는 12로 인해 중단되고 프로그램 예외 조건이 무효화되지(nullifying) 않을 때, ATIA는 실행중이었던 명령을 지나서 가리킨다.
- [0190] 트랜잭션이 중단 코드들 7~10, 14~16, 또는 255로 인해 중단될 때, ATIA는 그 중단을 일으키는 정확한 명령을 반드시 표시할 필요는 없으나, 트랜잭션 내 더 이전 또는 이후 명령을 가리킬 수 있다.
- [0191] 만일 트랜잭션이 실행형(execute-type) 명령의 타겟인 명령으로 인해 중단되면, ATIA는 상기 실행형 명령을 식별하며, 위에 기술된 중단 코드에 따라 상기 명령을 가리키거나 상기 명령을 지나쳐 가리킨다. ATIA는 상기 실행형 명령의 타겟을 표시하지 않는다.
- [0192] ATIA는 트랜잭션이 중단될 때의 주소지정 모드(addressing mode)의 적용을 받는다. 24-비트 주소지정 모드에서, 필드의 비트들 0~40은 제로들을 보유한다. 31-비트 주소지정 모드에서, 필드의 비트들 0~32는 제로들을 보유한다.
- [0193] 트랜잭션이 프로그램 인터럽션 이외의 조건들로 인해 중단될 때 중단된 트랜잭션 명령 주소가 프로그램 인터럽션 TDB에 저장되는지는 모델 종속적이다.
- [0194] 트랜잭션이 중단 코드들 4 또는 12로 인해 중단되고 프로그램 예외 조건이 무효화되지(nullifying) 않을 때, ATIA는 그 중단을 일으키는 명령을 가리키지 않는다. ATIA에서 인터럽션 길이 코드(ILC)에 의해 표시되는 하프워드들의 수를 뺄으로써, 중단을 일으키는 명령은 억제(suppressing) 또는 종결(terminating)되는 조건 또는 비-PER 이벤트들에 대해서는 완료(completing)되는 조건에서 식별될 수 있다. 트랜잭션이 PER 이벤트로 인해 중단되고 다른 프로그램 예외 조건이 존재하지 않을 때, ATIA는 예측 불가능하다.
- [0195] 트랜잭션 진단 블록 주소가 유효할 때, ILC는 TBEGIN-명시 TDB의 바이트들 36~39에서 프로그램 인터럽션 식별(PIID) 조사를 받을 수 있다. 필터링이 적용되지 않을 때, ILC는 실 스토리지 내 위치 140~143에서 PIID 조사를 받을 수 있다.
- [0196] 예외 액세스 식별(EAID)(914): 특정한 필터링된 프로그램 예외 조건들로 인해 중단되는 트랜잭션들에서, TBEGIN-명시 TDB의 바이트 32는 예외 액세스 식별(exception access identification)을 보유한다. z/Architecture의 한 예에서, EAID의 포맷과, 그것이 저장되는 경우들은 예외 조건이 인터럽션으로 귀결될 때 실 위치 160에 기술되는 바와 동일하며, 참조에 의해 위에서 포함된 Principles of Operation(운영 원리)에 기술된 바와 같다.
- [0197] 프로그램 인터럽션으로 귀결되는 모든 예외 조건들을 포함하여 다른 이유들로 인해 중단되는 트랜잭션들에서, 바이트 32는 예측 불가능하다. 바이트 32는 프로그램 인터럽션 TDB에서 예측 불가능하다.
- [0198] 이 필드는 트랜잭션 진단 블록 주소에 의해 지정되는 TDB에만 저장되며; 그렇지 않으면, 필드는 유효된다. EAID는 액세스 목록 제어 또는 DAT 보호(access list controlled or DAT protection), ASCE-형, 페이지 변환, 영역 제1(region first) 변환, 영역 제2(region second) 변환, 영역 제3(region third) 변환, 및 세그먼트

변환 프로그램 예외 조건들을 위해서만 저장된다.

- [0199] 데이터 예외 코드(DXC)(916): 필터링된 데이터 예외 프로그램 예외 조건들로 인해 중단되는 트랜잭션들에서, TBEGIN-명시 TDB의 바이트 33은 데이터 예외 코드(data exception code)를 보유한다. z/Architecture의 한 예에서, DXC의 포맷과, 그것이 저장되는 경우들은 예외 조건이 인터럽션으로 귀결될 때 실 위치 147에 기술되는 바와 동일하며, 참조에 의해 위에서 포함된 Principles of Operation(운영 원리)에 기술된 바와 같다. 한 예에서, 위치 147은 DXC를 포함한다.
- [0200] 프로그램 인터럽션으로 귀결되는 모든 예외 조건들을 포함하여 다른 이유들로 인해 중단되는 트랜잭션들에서, 바이트 33은 예측 불가능하다. 바이트 33은 프로그램 인터럽션 TDB에서 예측 불가능하다.
- [0201] 이 필드는 트랜잭션 진단 블록 주소에 의해 지정되는 TDB에만 저장되며; 그렇지 않으면, 필드는 유보된다. DXC는 데이터 프로그램 예외 조건들을 위해서만 저장된다.
- [0202] 프로그램 인터럽션 식별(PIID)(918): 필터링된 프로그램 예외 조건들로 인해 중단되는 트랜잭션들에서, TBEGIN-명시 TDB의 바이트들 36~39는 프로그램 인터럽션 식별(program interruption identification)을 보유한다. z/Architecture의 한 예에서, PIID의 포맷은 상기 조건이 인터럽션으로 귀결될 때 실 위치들 140~143에 기술되는 바와 같으며(참조에 의해 위에서 포함된 Principles of Operation(운영 원리)에 기술된 바와 같음), 단 예외가 있는데 PIID의 비트들 13~14 내 명령 길이 코드(ILC)는 예외 조건이 검출된 명령에 관한 것이다.
- [0203] 프로그램 인터럽션으로 귀결되는 예외 조건들을 포함하여 다른 이유들로 인해 중단되는 트랜잭션들에서, 바이트들 36~39는 예측 불가능하다. 바이트들 36~39는 프로그램 인터럽션 TDB에서 예측 불가능하다.
- [0204] 이 필드는 트랜잭션 진단 블록 주소에 의해 지정되는 TDB에만 저장되며; 그렇지 않으면, 필드는 유보된다. 프로그램 인터럽션 식별은 프로그램 예외 조건들을 위해서만 저장된다.
- [0205] 변환 예외 식별(TEID)(920): 다음의 필터링된 프로그램 예외 조건들 중 어느 하나로 인해 중단되는 트랜잭션들에서, TBEGIN-명시 TDB의 바이트들 40~47은 변환 예외 식별(translation exception identification)을 보유한다.
- [0206] • 액세스 목록 제어 또는 DAT 보호(Access list controlled or DAT protection)
 - [0207] • ASCE-형(ASCE-type)
 - [0208] • 페이지 변환
 - [0209] • 영역-제1 변환(Region-first translation)
 - [0210] • 영역-제2 변환(Region-second translation)
 - [0211] • 영역-제3 변환(Region-third translation)
 - [0212] • 세그먼트 변환 예외
- [0213] z/Architecture의 한 예에서, TEID의 포맷은 상기 조건이 인터럽션으로 귀결될 때 실 위치들 168~175에 기술되는 바와 동일하며, 참조에 의해 위에서 포함된 Principles of Operation(운영 원리)에 기술된 바와 같다.
- [0214] 프로그램 인터럽션으로 귀결되는 예외 조건들을 포함하여 다른 이유들로 인해 중단되는 트랜잭션들에서, 바이트들 40~47은 예측 불가능하다. 바이트들 40~47은 프로그램 인터럽션 TDB에서 예측 불가능하다.
- [0215] 이 필드는 트랜잭션 진단 블록 주소에 의해 지정되는 TDB에만 저장되며; 그렇지 않으면, 필드는 유보된다.
- [0216] 중단 이벤트 주소(Breaking Event Address)(922): 필터링된 프로그램 예외 조건들로 인해 중단되는 트랜잭션들에서, TBEGIN-명시 TDB의 바이트들 48~55는 중단 이벤트 주소(breaking event address)를 보유한다. z/Architecture의 한 예에서, 중단 이벤트 주소의 포맷은 상기 조건이 인터럽션으로 귀결될 때 실 위치들 272~279에 기술되는 바와 동일하며, 참조에 의해 위에서 포함된 Principles of Operation(운영 원리)에 기술된

바와 같다.

- [0217] 프로그램 인터럽션으로 귀결되는 예외 조건들을 포함하여 다른 이유들로 인해 중단되는 트랜잭션들에서, 바이트들 48~55는 예측 불가능하다. 바이트들 48~55는 프로그램 인터럽션 TDB에서 예측 불가능하다.
- [0218] 이 필드는 트랜잭션 진단 블록 주소에 의해 지정되는 TDB에만 저장되며; 그렇지 않으면, 필드는 유효된다.
- [0219] 중단 이벤트들(breaking events)과 관련된 더 상세사항은 아래에 기술된다.
- [0220] z/Architecture의 한 실시 예에서, PER-3 퍼실리티가 설치되어 있을 때, 그것은 프로그램에 CPU의 순차적인 실행에서 중단(break)을 일으키는 마지막 명령의 주소를 제공한다. 중단 이벤트 주소의 기록은 와일드 분기 검출(wild branch detection)을 위한 디버깅 보조수단(debugging assist)으로서 사용될 수 있다. 이 퍼실리티는 예를 들어 CPU에 중단 이벤트 주소 레지스터(breaking event address register)라 불리는 64-비트 레지스터를 제공한다. TRANSACTION ABORT 이외의 명령이 순차적인 명령 실행에서 중단(break)을 일으킬(즉, PSW 내 상기 명령 주소가 상기 명령의 길이에 의해 증분되기보다는 대체될) 때마다, 그 명령의 주소가 중단 이벤트 주소 레지스터에 배치된다. 프로그램 인터럽션이 발생할 때마다, PER이 표시되든 되지 않든 간에, 중단 이벤트 주소 레지스터의 현재 콘텐츠가 실 스토리지 위치들 272~279에 배치된다.
- [0221] 만일 중단 이벤트를 일으키는 명령이 실행형 명령(EXECUTE 또는 EXECUTE RELATIVE LONG)의 타겟이면, 상기 실행형 명령을 폐지하는 데 사용되는 명령 주소가 중단 이벤트 주소 레지스터에 배치된다.
- [0222] z/Architecture의 한 실시 예에서, 중단 이벤트는 다음의 명령들 중 하나가 분기(branching)를 일으킬 때마다 발생하는 것으로 간주된다: BRANCH AND LINK (BAL, BALR); BRANCH AND SAVE (BAS, BASR); BRANCH AND SAVE AND SET MODE (BASSM); BRANCH AND SET MODE (BSM); BRANCH AND STACK (BAKR); BRANCH ON CONDITION (BC, BCR); BRANCH ON COUNT (BCT, BCTR, BCTG, BCTGR); BRANCH ON INDEX HIGH (BXH, BXHG); BRANCH ON INDEX LOW OR EQUAL (BXLE, BXLEG); BRANCH RELATIVE ON CONDITION (BRC); BRANCH RELATIVE ON CONDITION LONG (BRCL); BRANCH RELATIVE ON COUNT (BRCT, BRCTG); BRANCH RELATIVE ON INDEX HIGH (BRXH, BRXHG); BRANCH RELATIVE ON INDEX LOW OR EQUAL (BRXLE, BRXLG); COMPARE AND BRANCH (CRB, CGRB); COMPARE AND BRANCH RELATIVE (CRJ, CGRJ); COMPARE IMMEDIATE AND BRANCH (CIB, CGIB); COMPARE IMMEDIATE AND BRANCH RELATIVE (CIJ, CGIJ); COMPARE LOGICAL AND BRANCH (CLRB, CLGRB); COMPARE LOGICAL AND BRANCH RELATIVE (CLRJ, CLGRJ); COMPARE LOGICAL IMMEDIATE AND BRANCH (CLIB, CLGIB); 및 COMPARE LOGICAL IMMEDIATE AND BRANCH RELATIVE (CLIJ, CLGIJ).
- [0223] 중단 이벤트는 또한 다음의 명령들 중 하나가 완료될 때마다 발생하는 것으로 간주된다: BRANCH AND SET AUTHORITY (BSA); BRANCH IN SUBSPACE GROUP (BSG); BRANCH RELATIVE AND SAVE (BRAS); BRANCH RELATIVE AND SAVE LONG (BRASL); LOAD PSW (LPSW); LOAD PSW EXTENDED (LPSWE); PROGRAM CALL (PC); PROGRAM RETURN (PR); PROGRAM TRANSFER (PT); PROGRAM TRANSFER WITH INSTANCE (PTI); RESUME PROGRAM (RP); 및 TRAP (TRAP2, TRAP4).
- [0224] 중단 이벤트는 트랜잭션이 중단되는 것의 결과로서(암시적으로 또는 TRANSACTION ABORT 명령의 결과로서) 발생하지는 않는 것으로 간주된다.
- [0225] 모델 종속적 진단 정보(924): 바이트들 112~127은 모델 종속적 진단 정보(model dependent diagnostic information)를 보유한다.
- [0226] 12(필터링된 프로그램 인터럽션)를 제외한 모든 중단 코드들에서, 모델 종속적 진단 정보는 저장되는 각 TDB에 세이브된다.
- [0227] 한 실시 예에서, 모델 종속적 진단 정보는 다음을 포함한다:
- [0228] • 바이트들 112~119는 트랜잭션 실행 분기 인디케이션(transactional execution branch indications, TXBI)이라 불리는 64비트들로 이루어진 벡터를 보유한다. 상기 벡터의 첫 63개 비트들의 각각은 CPU가 트랜잭션 실행 모드에 있던 동안 분기 명령(branching instruction)을 실행한 결과들을 다음과 같이 표시한다:
- [0229] 값 의미

- [0230] 0 명령이 분기가 일어나지 않고 완료되었음.
- [0231] 1 명령이 분기가 일어나고 완료되었음.
- [0232] 비트 0은 제1의 위와 같은 분기 명령의 결과를 나타내고, 비트 1은 제2의 위와 같은 분기 명령의 결과를 나타내는 등의 방식이다.
- [0233] 만일 CPU가 트랜잭션 실행 모드에 있던 동안 63개 미만의 분기 명령들이 실행되었다면, 분기 명령들과 대응하지 않는 최우측 비트들은 제로들로 세트된다(비트 63 포함). 63개 이상의 분기 명령들이 실행되었을 때, TXBI의 비트 63은 일(one)로 세트된다.
- [0234] TXBI 내 비트들은 위에 열거된 바와 같이 중단 이벤트를 일으킬 수 있는 명령들에 의해 세트되며, 다음은 예외이다:
- [0235] - 모든 제한된 명령은 비트가 TXBI에 세트되게 하지 않는다.
- [0236] - 예를 들어 z/Architecture의 명령들에서, BRANCH ON CONDITION, BRANCH RELATIVE ON CONDITION, 또는 BRANCH RELATIVE ON CONDITION LONG 명령의 M_1 필드가 제로일 때, 또는 다음의 명령들의 R_2 필드가 제로일 때, 그 명령의 실행이 비트가 TXBI에 세트되게 할 것인지는 모델 종속적이다.
- [0237] • BRANCH AND LINK (BALR); BRANCH AND SAVE (BASR); BRANCH AND SAVE AND SET MODE (BASSM); BRANCH AND SET MODE (BSM); BRANCH ON CONDITION (BCR); 및 BRANCH ON COUNT (BCTR, BCTGR)
- [0238] • 호스트 액세스 예외에 의해 일어난 중단 조건들에서, 바이트 127의 비트 위치 0은 일로 세트된다. 다른 모든 중단 조건들에서, 바이트 127의 비트 위치 0은 제로로 세트된다.
- [0239] • 로드/저장 유닛(LSU)에 의해 검출된 중단 조건들에서, 바이트 127의 최우측 5개 비트들은 그 원인의 표시(indication of the cause)를 보유한다. LSU에 의해 검출되지 않은 중단 조건들에서, 바이트 127은 유보된다.
- [0240] 범용 레지스터들(930): 바이트들 128~255는 트랜잭션이 중단된 시점에 범용 레지스터들 0~15의 콘텐츠를 보유한다. 상기 레지스터들은 범용 레지스터 0부터 시작하여 범용 레지스터 0은 바이트들 128~135에, 범용 레지스터 1은 바이트들 136~143에 오름차순으로 저장되는 등의 방식이다.
- [0241] 유보됨: 다른 모든 필드들은 유보된다. 다르게 표시되지 않는 한, 유보된 필드들의 콘텐츠는 예측 불가능하다.
- [0242] 다른 CPU들과 I/O 서브시스템에 의해 관찰될 때, 트랜잭션 중단 동안에 TDB(들)를 저장하는 것은 비트랜잭션 저장들 후에 발생하는 다중 액세스 참조(multiple access reference)이다.
- [0243] 트랜잭션은 그것이 실행되는 즉시 구성(immediate configuration)의 범위 밖의 원인들로 인해 중단될 수 있다. 예를 들어, (LPAR 또는 z/VM 같은) 하이퍼바이저에 의해 인지되는 과도 이벤트(transient events)는 트랜잭션에 중단을 일으킬 수 있다.
- [0244] 트랜잭션 진단 블록에 제공되는 정보는 진단 목적을 위한 것이며 실질적으로 정확하다. 하지만, 즉시 구성(immediate configuration)의 범위 밖의 이벤트에 의해 중단이 일어날 수도 있기 때문에, 중단 코드 또는 프로그램 인터럽션 식별 같은 정보는 상기 구성 내의 조건들을 정확히 반영하지 못할 수도 있으며, 그러므로 프로그램 동작(program action)을 결정하는 데 사용하면 안된다.
- [0245] TDB에 세이브되는 진단 정보에 더해서, 트랜잭션이 데이터 예외 프로그램 예외 조건으로 인해 중단되고 제어 레지스터 0의 비트 45인 AFP 레지스터 제어와 유효 부동 소수점 연산 허용 제어(F)가 모두 일(one)일 때, 필터링이 프로그램 예외 조건에 적용되는지와 상관없이, 데이터 예외 코드(DXC)가 부동 소수점 제어 레지스터(FPCR)의 바이트 2로 배치된다. 트랜잭션이 중단되고 AFP 레지스터 제어 또는 유효 부동 소수점 연산 허용 제어 중 둘 모두 또는 둘 중 하나가 제로일 때, DXC는 FPCR로 배치되지 않는다.
- [0246] 한 실시 예에서, 여기에 표시된 바와 같이, 트랜잭션 실행 퍼실리티가 설치되어 있을 때, 다음의 범용 명령들이 제공된다:

- [0247] • EXTRACT TRANSACTION NESTING DEPTH
- [0248] • NONTRANSACTIONAL STORE
- [0249] • TRANSACTION ABORT
- [0250] • TRANSACTION BEGIN
- [0251] • TRANSACTION END
- [0252] CPU가 트랜잭션 실행 모드에 있을 때, 특정한 명령들의 실행 시도는 제한되며 트랜잭션의 종단을 일으킨다.
- [0253] 제약 트랜잭션 실행 모드에서 발행될 때, 제한된 명령들의 실행 시도도 트랜잭션 제약 프로그램 인터럽션(transaction constraint program interruption)으로 귀결될 수 있거나, 또는 마치 트랜잭션이 제약되지 않은 것처럼 실행의 진행으로 귀결될 수 있다.
- [0254] z/Architecture의 한 예에서, 제한된 명령들(restricted instructions)은 예로서 다음의 비특권 명령들(non-privileged instructions)을 포함한다: COMPARE AND SWAP AND STORE; MODIFY RUNTIME INSTRUMENTATION CONTROLS; PERFORM LOCKED OPERATION; M_1 필드 내 코드가 6 또는 7일 때, PREFETCH DATA (RELATIVE LONG); M_3 필드는 제로이고 R_1 필드 내 코드가 6 또는 7일 때, STORE CHARACTERS UNDER MASK HIGH; STORE FACILITY LIST EXTENDED; STORE RUNTIME INSTRUMENTATION CONTROLS; SUPERVISOR CALL; 및 TEST RUNTIME INSTRUMENTATION CONTROLS.
- [0255] 상기 목록에서, COMPARE AND SWAP AND STORE와 PERFORM LOCKED OPERATION은 TX 모드에서 기본 명령들(basic instructions)의 사용에 의해 더 효율적으로 구현될 수 있는 복합 명령들(complex instructions)이다. PREFETCH DATA와 PREFETCH DATA RELATIVE LONG의 경우는 6 및 7의 코드들이 캐시 라인(cache line)을 해제하므로 제한되며, 트랜잭션의 완료 전에 데이터의 커밋(commitment)을 필요로 할 가능성이 있다. SUPERVISOR CALL은 그것이 인터럽션(이것은 트랜잭션의 종단을 일으킴)을 일으키므로 제한된다.
- [0256] 아래에 나열된 조건들하에서, 다음의 명령들은 제한된다:
- [0257] • 명령의 R_2 필드가 비제로이고 분기 추적(branch tracing)이 가능할 때, BRANCH AND LINK (BALR), BRANCH AND SAVE (BASR), 및 BRANCH AND SAVE AND SET MODE.
- [0258] • R_2 필드가 비제로이고 모드 추적(mode tracing)이 가능할 때, BRANCH AND SAVE AND SET MODE 및 BRANCH AND SET MODE; 모드 추적이 가능할 때, SET ADDRESSING MODE.
- [0259] • 모니터 이벤트 조건(monitor event condition)이 인지될 때, MONITOR CALL.
- [0260] 상기 목록은 추적 엔트리들(trace entries)을 형성할 수 있는 명령들을 포함한다. 만일 이 명령들이 트랜잭션적으로 실행되도록 허용되고 추적 엔트리들이 형성되었다면, 그리고 그 트랜잭션이 이어져 중단되었다면, 제어 레지스터 12 내 추적 테이블 포인터(trace table pointer)가 개선될 것이지만, 추적 테이블에 대한 저장들은 폐기될 것이다. 이것은 추적 테이블에 일관성이 없는 갭(gap)을 남길 것이므로, 상기 명령들은 추적 엔트리들을 형성하는 경우들로 제한된다.
- [0261] CPU가 트랜잭션 실행 모드에 있을 때, 다음의 명령들이 제한되는지는 모델 종속적이다: CIPHER MESSAGE; CIPHER MESSAGE WITH CFB; CIPHER MESSAGE WITH CHAINING; CIPHER MESSAGE WITH COUNTER; CIPHER MESSAGE WITH OFB; COMPRESSION CALL; COMPUTE INTERMEDIATE MESSAGE DIGEST; COMPUTE LAST MESSAGE DIGEST; COMPUTE MESSAGE AUTHENTICATION CODE; CONVERT UNICODE-16 TO UNICODE-32; CONVERT UNICODE-16 TO UNICODE-8; CONVERT UNICODE-32 TO UNICODE-16; CONVERT UNICODE-32 TO UNICODE-8; CONVERT UNICODE-8 TO UNICODE-16; CONVERT UNICODE-8 TO UNICODE-32; PERFORM CRYPTOGRAPHIC COMPUTATION; RUNTIME INSTRUMENTATION OFF; 및 RUNTIME INSTRUMENTATION ON.
- [0262] 상기 명령들 중 각각은 하드웨어 코프로세서(hardware coprocessor)에 의해 현재 구현되거나, 또는 과거 머신들에 있었으며, 그러므로 제한된 것으로 간주된다.

- [0263] 유효 AR 수정 허용(effective allow AR modification, A) 제어가 제로일 때, 다음의 명령들은 제한된다: COPY ACCESS; LOAD ACCESS MULTIPLE; LOAD ADDRESS EXTENDED; 및 SET ACCESS.
- [0264] 상기 명령들 중 각각은 액세스 레지스터의 콘텐츠의 수정을 일으킨다. 만일 TRANSACTION BEGIN 명령 내 A 제어가 제로이면, 프로그램은 액세스 레지스터의 수정이 허용되지 않는다고 명시적으로 표시한 것이다.
- [0265] 유효 부동 소수점 연산 허용(F) 제어가 제로일 때, 부동 소수점 명령들은 제한된다.
- [0266] 어떤 환경들하에서, 다음의 명령들은 제한될 수 있다: EXTRACT CPU TIME; EXTRACT PSW; STORE CLOCK; STORE CLOCK EXTENDED; 및 STORE CLOCK FAST.
- [0267] 상기 명령들 중 각각은 해석적 실행 상태 묘사(interpretative execution state description)에서 인터셉션 제어의 적용을 받는다. 만일 하이퍼바이저가 이 명령들에 대하여 상기 인터셉션 제어를 세트했다면, 그들의 실행은 하이퍼바이저 구현으로 인해 오래 지속될 수 있는데; 그러므로, 그들은 만일 인터셉션이 발생하면 제한된 것으로 간주된다.
- [0268] 비제약 트랜잭션이 제한된 명령(restricted instruction)의 실행 시도 때문에 중단되고, 트랜잭션 진단 블록 내 트랜잭션 중단 코드는 11로 세트되고(제한된 명령), 그리고 조건 코드가 3으로 세트될 때, 예외는 다음과 같다: 비제약 트랜잭션이 명령의 실행 시도(그렇지 않은 경우 특권 연산 예외(privileged operation exception)로 귀결될 수 있는)로 인해 중단될 때, 중단 코드가 11(제한된 명령)로 세트될지 또는 4(특권 연산 프로그램 인터럽션의 인지로부터 기인하는 필터링되지 않은 프로그램 인터럽션)로 세트될지는 예측 불가능하다. M_1 필드 내 코드가 6 또는 7일 때 PREFETCH DATA (RELATIVE LONG)의 실행 시도로 인해 또는 M_3 필드가 제로이고 R_1 필드 내 코드가 6 또는 7일 때 STORE CHARACTERS UNDER MASK HIGH의 실행 시도로 인해 비제약 트랜잭션이 중단될 때, 중단 코드가 11(제한된 명령)로 세트될지 또는 16(캐시 기타(cache other))으로 세트될지는 예측 불가능하다. 비제약 트랜잭션이 MONITOR CALL의 실행 시도로 인해 중단되고, 모니터 이벤트 조건 및 명세 예외 조건이 모두 존재할 때, 중단 코드가 11로 세트될지 또는 4로 세트될지, 아니면 프로그램 인터럽션이 필터링된 경우 12로 세트될지는 예측 불가능하다.
- [0269] 추가적인 명령들이 제약 트랜잭션에서 제한될 수 있다. 이 명령들은 현재 비제약 트랜잭션에서 제한되는 것으로 정의되지는 않지만, 그들은 미래의 프로세서들 상의 비제약 트랜잭션에서 어떤 환경들하에서는 제한될 수 있다.
- [0270] 어떤 제한된 명령들은 미래의 프로세서들 상의 트랜잭션 실행 모드에서 허용될 수 있다. 따라서, 프로그램은 제한된 명령의 실행 시도로 인해 트랜잭션이 중단되는 것에 의존해서는 안 된다. TRANSACTION ABORT 명령은 트랜잭션이 신뢰할 수 있을 정도로 중단되도록 하는 데 사용될 수 있다.
- [0271] 비제약 트랜잭션에서, 프로그램은 제한된 명령으로 인해 중단되는 트랜잭션을 수용하기 위한 대체 가능한(alternative) 비트랜잭션 코드 경로를 제공해야 한다.
- [0272] 연산시에, 트랜잭션 내포 깊이가 제로일 때, 조건 코드 제로로 귀결되는 TRANSACTION BEGIN(TBEGIN) 명령의 실행은 CPU를 비제약 트랜잭션 실행 모드로 진입하게 한다. 트랜잭션 내포 깊이가 제로일 때, 조건 코드 제로로 귀결되는 제약 TRANSACTION BEGIN(TBEGINC) 명령의 실행은 CPU를 제약 트랜잭션 실행 모드로 진입하게 한다.
- [0273] 명시적으로 다르게 언급된 경우를 제외하고, 비제약 실행에 적용되는 모든 규칙들은 트랜잭션 실행에도 적용된다. 아래는 CPU가 트랜잭션 실행 모드에 있는 동안의 추가적인 처리의 특성들이다.
- [0274] CPU가 비제약 트랜잭션 실행 모드에 있을 때, 조건 코드 제로로 귀결되는 TRANSACTION BEGIN 명령의 실행은 CPU를 비제약 트랜잭션 실행 모드에 남아 있게 한다.
- [0275] CPU에 의해 관찰될 때, 트랜잭션 실행 모드에서 이루어지는 페치들과 저장들은 트랜잭션 실행 모드에 있지 않은 동안 이루어지는 것들과 다르지 않다. 다른 CPU들과 I/O 서브시스템에 의해 관찰될 때, CPU가 트랜잭션 실행 모드에 있는 동안 이루어지는 모든 스토리지 오퍼랜드 액세스는 단일한 블록 동시적 액세스(single block concurrent access)인 것으로 보인다. 즉, 하프워드, 워드, 더블워드, 또는 쿼드워드 내의 모든 바이트들에 대한 액세스는 다른 CPU들과 I/O(예를 들어, 채널) 프로그램들에 의해 관찰될 때 블록 동시적(block concurrent)인 것으로 보이도록 명시된다. 하프워드, 워드, 더블워드, 또는 쿼드워드는 이 섹션에서 블록이라 불린다. 페치형(fetch-type) 참조가 블록 내에서 동시적(concurrent)인 것으로 보이도록 명시될 때, 또 다른 CPU 또는 I/O 프로그램에 의한 그 블록에 대한 저장 액세스는 그 블록에 포함된 바이트들이 페치되는 시간 동안에는 허용되지 않는다. 저장형(store-type) 참조가 블록 내에서 동시적인 것으로 보이도록 명시될 때, 그 블록 내 바이트들이

저장되는 시간 동안에는 또 다른 CPU 또는 I/O 프로그램에 의한, 폐치든 또는 저장이든, 그 블록에 대한 액세스는 허용되지 않는다.

- [0276] 명령에 대한 스토리지 액세스와 DAT 테이블 폐치 및 ART(Access Register Table) 테이블 폐치는 비트랜잭션 규칙을 따른다.
- [0277] CPU는 트랜잭션 내포 깊이가 제로로 변화도록 만드는(이 경우에, 트랜잭션은 완료됨) TRANSACTION END 명령을 이용하여 트랜잭션 실행 모드에서 정상적으로 나간다.
- [0278] CPU가 TRANSACTION END 명령의 완료를 이용하여 트랜잭션 실행 모드에서 나갈 때, 트랜잭션 실행 모드에 있는 동안 이루어진 모든 저장들은 커밋된다. 즉, 상기 저장들은 다른 CPU들과 I/O 서브시스템에 의해 관찰될 때 단일한 블록-동시 연산으로서 이루어지는 것으로 보인다.
- [0279] 트랜잭션은 다양한 이유들로 인해 암시적으로 중단될 수도 있고, 또는 TRANSACTION ABORT 명령에 의해 명시적으로 중단될 수도 있다. 트랜잭션 중단의 가능한 원인들의 예, 그에 대응하는 중단 코드, 및 트랜잭션 중단 PSW에 배치되는 조건 코드를 아래에 기술한다.
- [0280] 외부 인터럽션(External Interruption): 트랜잭션 중단 코드는 2로 세트되고, 트랜잭션 중단 PSW 내 조건 코드는 2로 세트된다. 트랜잭션 중단 PSW는 외부 인터럽션 처리의 일부로서 외부 구(old) PSW로서 저장된다.
- [0281] 프로그램 인터럽션(필터링 안됨): 인터럽션으로 귀결되는 프로그램 예외 조건(즉, 필터링 안된 조건)은 트랜잭션에 코드 4로 중단을 일으킨다. 트랜잭션 중단 PSW 내 조건 코드는 상기 프로그램 인터럽션 코드에 특화되어(specific) 세트된다. 트랜잭션 중단 PSW는 프로그램 인터럽션 처리의 일부로서 프로그램 구(old) PSW로서 저장된다.
- [0282] 다른 경우 연산 예외로 인해 트랜잭션의 중단으로 귀결될 수 있는 명령이 대체 가능한(alternate) 결과들을 산출할 수 있는데: 비제약 트랜잭션에서, 트랜잭션은 대신에 중단 코드 11(제한된 명령)로 중단될 수 있고; 제약 트랜잭션에서, 트랜잭션 제약 프로그램 인터럽션이 연산 예외 대신 인지될 수 있다.
- [0283] PER(Program Event Recording) 이벤트가 다른 필터링되지 않은 프로그램 예외 조건과 함께 인지될 때, 조건 코드는 3으로 세트된다.
- [0284] 기계 검사 인터럽션(Machine Check Interruption): 트랜잭션 중단 코드는 5로 세트되고, 트랜잭션 중단 PSW 내 조건 코드는 2로 세트된다. 트랜잭션 중단 PSW는 기계 검사 인터럽션 처리의 일부로서 기계 검사 구(old) PSW로서 저장된다.
- [0285] I/O 인터럽션: 트랜잭션 중단 코드는 6으로 세트되고, 트랜잭션 중단 PSW 내 조건 코드는 2로 세트된다. 트랜잭션 중단 PSW는 I/O 인터럽션 처리의 일부로서 I/O 구(old) PSW로서 저장된다.
- [0286] 폐치 오버플로(Fetch Overflow): 폐치 오버플로 조건은 트랜잭션이 CPU에서 지원되는 것들보다 더 많은 위치들로부터 폐치하려고 시도할 때 검출된다. 트랜잭션 중단 코드는 7로 세트되고, 조건 코드는 2 또는 3으로 세트된다.
- [0287] 저장 오버플로(Store Overflow): 저장 오버플로 조건은 트랜잭션이 CPU에서 지원되는 것들보다 더 많은 위치들에 저장하려고 시도할 때 검출된다. 트랜잭션 중단 코드는 8로 세트되고, 조건 코드는 2 또는 3으로 세트된다.
- [0288] 폐치 또는 저장 오버플로 중단에 응답하여 조건 코드가 2 또는 3이 되게 하면 CPU가 잠재적으로 재시도 가능한 상황들을 표시하는 것이 가능할 수 있다(예를 들어, 조건 코드 2는 트랜잭션의 재실행이 생산적일(productive) 수 있다고 표시하는 반면; 조건 코드 3은 재실행을 권고하지 않는다).
- [0289] 폐치 충돌(Fetch Conflict): 폐치 충돌 조건은 또 다른 CPU 또는 I/O 서브시스템이 이 CPU에 의해 트랜잭션적으로 폐치된 위치에 저장을 시도할 때 검출된다. 트랜잭션 중단 코드는 9로 세트되고, 조건 코드는 2로 세트된다.
- [0290] 저장 충돌(Store Conflict): 저장 충돌 조건은 또 다른 CPU 또는 I/O 서브시스템이 이 CPU에 의한 트랜잭션 실행 동안에 저장된 위치에 액세스를 시도할 때 검출된다. 트랜잭션 중단 코드는 10으로 세트되고, 조건 코드는 2로 세트된다.
- [0291] 제한된 명령(Restricted Instruction): CPU가 트랜잭션 실행 모드에 있을 때, 제한된 명령들의 실행 시

도는 트랜잭션의 종단을 일으킨다. 트랜잭션 중단 코드는 11로 세트되고, 조건 코드는 3으로 세트된다.

- [0292] CPU가 제약 트랜잭션 실행 모드에 있을 때, 제한된 명령의 실행 시도가 트랜잭션 제약 프로그램 인터럽션으로 귀결될지 또는 제한된 명령으로 인해 중단으로 귀결될지는 예측 불가능하다. 트랜잭션은 여전히 중단되지만 중단 코드는 둘 중 한 원인을 표시할 수 있다.
- [0293] 프로그램 예외 조건(필터링 됨): 인터럽션으로 귀결되지 않는 프로그램 예외 조건(즉, 필터링된 조건)은 트랜잭션에 12의 트랜잭션 중단 코드로 종단을 일으킨다. 조건 코드는 3으로 세트된다.
- [0294] 내포 깊이 초과(Nesting Depth Exceeded): 내포 깊이 초과 조건(nesting depth exceeded condition)은 트랜잭션 내포 깊이가 그 구성에 대한 최대 허용값에 있을 때 검출되며, TRANSACTION BEGIN 명령이 실행된다. 트랜잭션은 13의 트랜잭션 중단 코드로 중단되고, 조건 코드는 3으로 세트된다.
- [0295] 캐시 페치 관련 조건(Cache Fetch Related Condition): 트랜잭션에 의해 페치된 스토리지 위치들과 관련된 조건은 CPU의 캐시 회로(cache circuitry)에 의해 검출된다. 트랜잭션은 14의 트랜잭션 중단 코드로 중단되고, 조건 코드는 2 또는 3으로 세트된다.
- [0296] 캐시 저장 관련 조건(Cache Store Related Condition): 트랜잭션에 의해 저장된 스토리지 위치들과 관련된 조건은 CPU의 캐시 회로(cache circuitry)에 의해 검출된다. 트랜잭션은 15의 트랜잭션 중단 코드로 중단되고, 조건 코드는 2 또는 3으로 세트된다.
- [0297] 캐시 기타 조건(Cache Other Condition): 캐시 기타 조건은 CPU의 캐시 회로에 의해 검출된다. 트랜잭션은 16의 트랜잭션 중단 코드로 중단되고, 조건 코드는 2 또는 3으로 세트된다.
- [0298] 트랜잭션 실행 동안에, 만일 CPU가 동일한 절대 주소에 매핑된 각각 다른 논리 주소들을 사용하여 명령들 또는 스토리지 오퍼랜드들을 액세스하면, 트랜잭션이 중단될지는 모델 종속적이다. 만일 트랜잭션이 동일한 절대 주소에 매핑된 각각 다른 논리 주소들을 사용한 액세스들로 인해 중단되면, 조건에 따라서 중단 코드 14, 15, 또는 16이 세트된다.
- [0299] 기타 조건(Miscellaneous Condition): 기타 조건은 트랜잭션에 종단을 일으키는, CPU에 의해 인지되는 기타 다른 조건이다. 트랜잭션 중단 코드는 255로 세트되고, 조건 코드는 2 또는 3으로 세트된다.
- [0300] 다수의 구성들(multiple configurations)이 동일한 머신(예를 들어, 논리적 파티션들 또는 가상 머신들)에서 실행 중일 때, 트랜잭션은 각각 다른 구성에서 발생한 외부 기계 검사 인터럽션 또는 I/O 인터럽션으로 인해 중단될 수 있다.
- [0301] 위에 예들이 제공되지만, 대응하는 중단 코드들과 조건 코드들을 갖는 트랜잭션 종단의 다른 원인들이 제공될 수 있다. 예를 들어, 재시작 인터럽션(Restart Interruption)이 한 원인일 수 있으며, 이 인터럽션에서 트랜잭션 중단 코드는 1로 세트되고, 트랜잭션 중단 PSW 내 조건 코드는 2로 세트된다. 트랜잭션 중단 PSW는 재시작 처리의 일부로서 재시작-구(restart-old) PSW로서 저장된다. 추가로 예를 들면, 슈퍼바이저 콜(Supervisor Call) 조건이 한 원인일 수 있으며, 이 조건에서 중단 코드는 3으로 세트되고, 트랜잭션 중단 PSW 내 조건 코드는 3으로 세트된다. 다른 또는 각각 다른 예들도 가능하다.
- [0302] 주(Notes):
- [0303] 1. 기타 조건(miscellaneous condition)은 다음 중 하나로부터 기인할 수 있다:
- [0304] • z/Architecture에서, 그 구성에서 또 다른 CPU에 의해 수행되는 다음과 같은 명령들: COMPARE AND REPLACE DAT TABLE ENTRY, COMPARE AND SWAP AND PURGE, INVALIDATE DAT TABLE ENTRY, INVALIDATE PAGE TABLE ENTRY, NQ 제어는 제로이고 SK 제어는 일인 PERFORM FRAME MANAGEMENT FUNCTION, NQ 제어는 제로인 SET STORAGE KEY EXTENDED; 조건 코드는 2로 세트된다.
- [0305] • 리셋(reset), 재시작(restart) 또는 정지(stop) 같은 오퍼레이터 기능(operator function) 또는 등가의 SIGNAL PROCESSOR 명령(order)이 CPU 상에서 수행된다.
- [0306] • 위에 열거되지 않은 기타 다른 조건; 조건 코드는 2 또는 3으로 세트된다.
- [0307] 2. 페치 충돌 및 저장 충돌이 검출되는 위치는 동일한 캐시 라인 내 어느 곳이든 될 수 있다.
- [0308] 3. 어떤 조건들하에서, CPU는 비슷한 중단 조건들을 구분하지 못할 수 있다. 예를 들어, 페치 또는 저

장 오버플로는 각각 페치 또는 저장 충돌과 구분이 안될 수 있다.

- [0309] 4. CPU에 의한 다중 명령 경로들의 추론적 실행은 충돌 조건 또는 오버플로 조건으로 인해—그러한 조건들이 개념적인 순서에서는 일어나지 않을지라도—트랜잭션의 중단으로 귀결될 수 있다. 제약 트랜잭션 실행 모드에 있는 동안, CPU는 추론적 실행을 일시적으로 금지시켜서 트랜잭션이 그러한 충돌 또는 오버플로를 추론적으로 검출하지 않고 완료를 시도할 수 있도록 할 수 있다.
- [0310] TRANSACTION ABORT 명령의 실행은 트랜잭션에 중단을 일으킨다. 트랜잭션 중단 코드는 제2 오퍼랜드 주소로부터 세트된다. 제2 오퍼랜드 주소의 비트 63이 각각 제로 또는 일인지에 따라서, 조건 코드는 각각 2 또는 3으로 세트된다.
- [0311] 도 10은 트랜잭션 진단 블록에 저장된 예시적인 중단 코드들과, 그에 대응하는 조건 코드(CC)를 요약한 것이다. 도 10에 있는 설명은 하나의 구체적인 구현 예를 예시한다. 다른 구현 예 및 값들의 인코딩도 가능하다.
- [0312] 한 실시 예에서, 그리고 전술한 바와 같이, 트랜잭션 퍼실리티는 제약 트랜잭션에 대하여 그리고 그와 연관된 처리를 제공한다. 제약 트랜잭션은 폴백(fall-back) 경로 없이 트랜잭션 모드에서 실행된다. 그것은 컴팩트한 기능들(compact functions)에 유용한 처리의 모드이다. 제약 트랜잭션에 관한 더 상세한 사항을 아래에 기술한다.
- [0313] (트랜잭션이 성공적으로 완료될 수 없게 하는 조건들에 의해 일어나는) 반복되는 인터럽션들 또는 다른 CPU들 또는 I/O 서브시스템과의 충돌들이 없는 경우에, 제약 트랜잭션은 최종적으로 완료되므로, 중단-처리부(abort-handler) 루틴은 필요하지 않으며 명시되지 않는다. 예를 들어, 주소지정될 수 없는(예를 들어, 0으로 나눔) 조건의 위반이 없는 경우에; 트랜잭션이 완료될 수 없게 하는 조건의 위반이 없는(예를 들어, 명령이 실행될 수 없게 하는 타이머 인터럽션; 핫(hot) I/O 등등) 경우에; 또는 제약 트랜잭션과 연관된 제한 또는 제약의 위반이 없는 경우에, 트랜잭션은 최종적으로 완료될 것이다.
- [0314] 제약 트랜잭션은 트랜잭션 내포 깊이가 처음에 제로일 때 제약 TRANSACTION BEGIN(TBEGINC) 명령에 의해 개시된다. 제약 트랜잭션은 한 예에서 다음의 제약들의 적용을 받는다.
- [0315] 1. 트랜잭션은 32개 명령들을 초과하여 실행하지 않으며, 여기에 제약 TRANSACTION BEGIN(TBEGINC) 명령과 TRANSACTION END 명령은 포함되지 않는다.
- [0316] 2. 트랜잭션 내 모든 명령들은 스토리지의 256개 인접한 바이트들 내에 있어야 하며, 여기에 제약 TRANSACTION BEGIN(TBEGINC) 명령과 TRANSACTION END 명령이 포함된다.
- [0317] 3. 상기 제한된 명령들에 더하여, 다음의 제한들도 제약 트랜잭션에 적용된다.
- [0318] a. 명령들은 예를 들어 더하기(add), 빼기(subtract), 곱하기(multiply), 나누기(divide), 시프트(shift), 로테이트(rotate) 등을 포함한 범용 명령들(General Instructions)이라 불리는 명령들로 한정된다.
- [0319] b. 분기 명령들은 다음(나열된 명령들은 z/Architecture의 한 예에서 온 것임)으로 한정된다:
- [0320] • M_1 이 비제로이고 RI_2 필드가 양의 값을 보유하는, BRANCH RELATIVE ON CONDITION.
- [0321] • M_1 이 비제로이고 RI_2 필드는 주소의 순환(address wraparound)을 일으키지 않는 양의 값을 보유하는, BRANCH RELATIVE ON CONDITION LONG.
- [0322] • COMPARE AND BRANCH RELATIVE, COMPARE IMMEDIATE AND BRANCH RELATIVE, COMPARE LOGICAL AND BRANCH RELATIVE, and COMPARE LOGICAL IMMEDIATE AND BRANCH RELATIVE, 여기에서 M_3 필드는 비제로이고 RI_4 필드는 양의 값을 보유함. (즉, 비제로 분기 마스크들을 갖는 순방향 분기들(forward branch)만 해당됨.)
- [0323] c. TRANSACTION END 및 명시된 오퍼랜드 직렬화를 일으키는 명령들을 제외하고, 직렬화 기능을 일으키는 명령들은 제한된다.
- [0324] d. 스토리지-및-스토리지 연산들(SS-) 명령들, 및 확장된 오퍼코드를 갖는 스토리지-및-스토리지 연산들(SSE-) 명령들은 제한된다.

- [0325] e. 다음의 모든 범용 명령들(이들은 z/Architecture의 이 예에서 온 것임)은 제한된다:
CHECKSUM; CIPHER MESSAGE; CIPHER MESSAGE WITH CFB; CIPHER MESSAGE WITH CHAINING; CIPHER MESSAGE WITH COUNTER; CIPHER MESSAGE WITH OFB; COMPARE AND FORM CODEWORD; COMPARE LOGICAL LONG; COMPARE LOGICAL LONG EXTENDED; COMPARE LOGICAL LONG UNICODE; COMPARE LOGICAL STRING; COMPARE UNTIL SUBSTRING EQUAL; COMPRESSION CALL; COMPUTE INTERMEDIATE MESSAGE DIGEST; COMPUTE LAST MESSAGE DIGEST; COMPUTE MESSAGE AUTHENTICATION CODE; CONVERT TO BINARY; CONVERT TO DECIMAL; CONVERT UNICODE-16 TO UNICODE-32; CONVERT UNICODE-16 TO UNICODE-8; CONVERT UNICODE-32 TO UNICODE-16; CONVERT UNICODE-32 TO UNICODE-8; CONVERT UNICODE-8 TO UNICODE-16; CONVERT UNICODE-8 TO UNICODE-32; DIVIDE; DIVIDE LOGICAL; DIVIDE SINGLE; EXECUTE; EXECUTE RELATIVE LONG; EXTRACT CACHE ATTRIBUTE; EXTRACT CPU TIME; EXTRACT PSW; EXTRACT TRANSACTION NESTING DEPTH; LOAD AND ADD; LOAD AND ADD LOGICAL; LOAD AND AND; LOAD AND EXCLUSIVE OR; LOAD AND OR; LOAD PAIR DISJOINT; LOAD PAIR FROM QUAD WORD; MONITOR CALL; MOVE LONG; MOVE LONG EXTENDED; MOVE LONG UNICODE; MOVE STRING; NON-TRANSACTIONAL STORE; PERFORM CRYPTOGRAPHIC COMPUTATION; PREFETCH DATA; PREFETCH DATA RELATIVE LONG; RUNTIME INSTRUMENTATION EMIT; RUNTIME INSTRUMENTATION NEXT; RUNTIME INSTRUMENTATION OFF; RUNTIME INSTRUMENTATION ON; SEARCH STRING; SEARCH; STRING UNICODE; SET ADDRESSING MODE; M_3 필드가 제로이고, R_1 필드 내 코드가 6 또는 7일 때, STORE CHARACTERS UNDER MASK HIGH; STORE CLOCK; STORE CLOCK EXTENDED; STORE CLOCK FAST; STORE FACILITY LIST EXTENDED; STORE PAIR TO QUADWORD; TEST ADDRESSING MODE; TRANSACTION ABORT; TRANSACTION BEGIN (TBEGIN과 TBEGINC 모두); TRANSLATE AND TEST EXTENDED; TRANSLATE AND TEST REVERSE EXTENDED; TRANSLATE EXTENDED; TRANSLATE ONE TO ONE; TRANSLATE ONE TO TWO TRANSLATE TWO TO ONE; 및 TRANSLATE TWO TO TWO.
- [0326] 4. 트랜잭션의 스토리지 오퍼랜드들은 4개의 옥토휄드를 초과하여 액세스하지 않는다. 주(Note): LOAD ON CONDITION과 STORE ON CONDITION은 조건 코드에 상관없이 스토리지를 참조하는 것으로 간주된다. 옥토휄드(octoword)는 예를 들어 32바이트 경계 상의 32개의 연속 바이트들의 그룹이다.
- [0327] 5. 이 CPU 상에서 실행중인 트랜잭션 또는 다른 CPU들 또는 I/O 서브시스템에 의한 저장들은 제약 TRANSACTION BEGIN(TBEGINC) 명령부터 시작하여 256 바이트의 스토리지를 보유하는 모든 4K-바이트 블록들 내의 스토리지 오퍼랜드들을 액세스하지 않는다.
- [0328] 6. 트랜잭션은 동일한 절대 주소에 매핑된 각각 다른 논리 주소들을 사용하여 명령들 또는 스토리지 오퍼랜드들을 액세스하지 않는다.
- [0329] 7. LOAD ACCESS MULTIPLE, LOAD MULTIPLE, LOAD MULTIPLE HIGH, STORE ACCESS MULTIPLE, STORE MULTIPLE, 및 STORE MULTIPLE HIGH에 대하여 오퍼랜드 참조들이 단일 옥토휄드 내에 있어야 하는 것을 제외하고, 트랜잭션에 의해 이루어지는 오퍼랜드 참조들은 단일 더블워드 내에 있어야 한다.
- [0330] 만일 제약 트랜잭션이 위에 나열된 제약들 1~7 중 어느 하나를 위반하면, (a) 트랜잭션 제약 프로그램 인터럽션이 인지되거나, 또는 (b) 추가적인 제약 위반들이 트랜잭션 제약 프로그램 인터럽션을 일으킬 수 있는 것을 제외하고는, 마치 트랜잭션이 제약되지 않은 것처럼 실행이 진행된다. 어느 액션이 취해지는지는 예측 불가능하며, 취해지는 액션은 어느 제약이 위반되는지에 기초하여 달라질 수 있다.
- [0331] 제약 위반들, 반복되는 인터럽션들, 또는 다른 CPU들 또는 I/O 서브시스템과의 충돌들이 없는 경우에, 제약 트랜잭션은 최종적으로 위에 기술된 바와 같이 완료될 것이다.
- [0332] 1. 만일 제약 트랜잭션이 다음의 기준(criteria)을 충족하면 제약 트랜잭션을 성공적으로 완료할 가능성은 향상된다.
- [0333] a. 발행된 명령들은 최대 32개보다 적다.
- [0334] b. 스토리지 오퍼랜드 참조들은 최대 4개 옥토휄드보다 적다.
- [0335] c. 스토리지 오퍼랜드 참조들은 동일한 캐시 라인 상에 있다.
- [0336] d. 동일한 위치들에 대한 스토리지 오퍼랜드 참조들은 모든 트랜잭션들에서 동일한 순서로 일어난다.
- [0337] 2. 제약 트랜잭션은 자신의 제1회 실행에서 성공적으로 완료할 것이 반드시 보장될 필요는 없다. 하지만, 만일 상기 나열된 제약들 중 어느 것도 위반하지 않는 제약 트랜잭션이 중단되면, CPU는 트랜잭션의 반복적

인 실행이 이후에 성공하도록 보장하기 위한 회로(circuitry)를 채용한다.

- [0338] 3. 제약 트랜잭션 내에서, TRANSACTION BEGIN은 제한된 명령이므로, 제약 트랜잭션은 내포될 수 없다.
- [0339] 4. 제약 트랜잭션에 의한 상기 제약들 1~7 중 어느 하나의 위반은 프로그램 루프(program loop)로 귀결될 수 있다.
- [0340] 5. 제약 트랜잭션의 한계들은 비교-및-교환 루프(compare-and-swap loop)의 것들과 비슷하다. 다른 CPU들 및 I/O 서브시스템으로부터 방해 가능성 때문에, COMPARE AND SWAP 명령이 조건 코드 0으로 항상 완료될 것이라는 아키텍처적인 보장은 없다. 제약 트랜잭션은 폐지-충돌 중단 또는 저장-충돌 중단 또는 핫(hot) 인터럽션의 형태로 비슷한 방해를 겪을 수 있다.
- [0341] CPU는 제약 위반들이 없을 경우에 제약 트랜잭션이 최종적으로 완료되도록 보장하기 위해 공정성 알고리즘들(fairness algorithms)을 채용한다.
- [0342] 6. 제약 트랜잭션을 완료하는 데 필요한 반복 횟수를 결정하기 위해, 프로그램은 범용 레지스터 세이프 마스크의 적용을 받지 않는 범용 레지스터 내 카운터를 채용할 수 있다. 아래에 예를 도시한다.
- [0343] LH1 15,0 Zero retry counter.
- [0344] Loop TBEGINC 0(0),X'FE00' Preserve GRs 0-13
- [0345] AHI 15,1 Increment counter
- [0346] ...
- [0347] ... Constrained transactional-execution code
- [0348] TEND End of transaction.
- [0349] * 이제 R15는 반복적인 트랜잭션 시도의 카운트를 보유한다.
- [0350] 레지스터 14와 15는 모두 이 예에서 복원되지 않는다는 것에 유의한다. 어떤 모델들에서, 만일 CPU가 TBEGINC 명령의 완료에 이어서 그러나 AHI 명령의 완료 전에 중단 조건을 검출하면 범용 레지스터 15 내 카운트가 낮을 수 있음에 또한 유의한다.
- [0351] CPU에 의해 관찰될 때, 트랜잭션 실행 모드에서 이루어지는 폐지들과 저장들은 트랜잭션 실행 모드에 있지 않은 동안 이루어지는 것들과 다르지 않다.
- [0352] 한 실시 예에서, 사용자(즉, 트랜잭션을 생성하는 자)는 트랜잭션이 제약될 것인지 아닌지를 선택한다. 도 11을 참조하여 위와 같은 결정을 내리는 데 사용되는 로직의 한 실시 예를 도시한다. 처음에, 사용자는 트랜잭션이 제약(constrained)될 것인지를 검토한다(단계 1100). 예를 들어, 사용자는 제약 트랜잭션들과 연관된 제한사항들이 수용 가능한지를 결정한다. 만일 그렇다면(수용 가능)(질의 1102), 제약 트랜잭션이 생성된다(단계 1104). 이것은 TBEGINC 이후의 하나 또는 그 이상의 명령들을 포함하여 TBEGINC 명령을 사용하여 제약 트랜잭션을 시작하는 것과 TEND로 트랜잭션을 종료하는 것을 포함한다. 그렇지 않고(수용 불가능), 만일 제약 트랜잭션이 생성되지 않는다면, 한 실시 예에서, 비제약 트랜잭션이 생성된다(단계 1106). 이것은 하나 또는 그 이상의 명령들을 포함하여 TBEGIN을 사용하여 비제약 트랜잭션을 시작하고 TEND로 종료하는 것을 포함한다.
- [0353] 추가 실시 예로서, 프로세서는 사용자에게 제약 트랜잭션이 사용될 것인지에 대한 추천을 한다. 이 추천은 프로세서에 의해 액세스 가능한, 저장된 제약 규칙(예를 들어, 제한사항들)에 기초한다. 예를 들어, 프로세서는 상기 규칙을 액세스하고 모든 제한사항들이 수용 가능한지를 결정한다. 만일 그렇다면(수용 가능), 프로세서는 제약 트랜잭션을 추천하고, 그렇지 않으면(수용 불가능), 비제약 트랜잭션을 추천한다.
- [0354] 도 12를 참조하여 트랜잭션과 연관된 중단 처리를 개괄적으로 기술한다. 처음에, 프로세서가 트랜잭션(예를 들어, TBEGIN 또는 TBEGINC)을 획득(예를 들어, 폐지, 수신, 등)(단계 1200)하고 프로세서에 의해 트랜잭션이 제약이될 것인지에 대한 결정이 이루어진다(질의 1202). 예를 들어, 트랜잭션을 개시하는 명령의 오피코드(또는 다른 실시 예에서는 상기 명령의 다른 필드)가 TBEGINC 트랜잭션을 명시하는지에 대한 결정이 이루어진다. 만일 그렇다면(제약), 트랜잭션은 제약 트랜잭션으로서 처리된다(단계 1204). 제약 트랜잭션에서, 트랜잭션-중단 PSW는 트랜잭션을 시작한 TBEGINC 명령을 가리키도록 세트된다. 그 다음에, 만일 중단 조건에 도달하면(질의 1206), 제어가 이 PSW로 넘어가고, 이 PSW는 TBEGINC 명령을 가리키며, 조건 코드는 그 조건이 복구 가능한지 아닌지 예측을 표시하도록 세트된다. 제약 트랜잭션에서, 중단 처리부는 (TBEGINC로 돌아가는 것 이외에) 호출

되지 않고, 그 대신에 처리는 TBEGINC 명령의 재실행으로 계속된다. 만일 중단 조건에 도달하지 않으면, 한 실시 예에서, 트랜잭션은 TRANSACTION END 명령에 의해 완료된다.

- [0355] 하지만, 만일 트랜잭션이 제약되지 않으면(질의 1202), 트랜잭션은 비제약 트랜잭션으로서 처리된다(단계 1208). 비제약 트랜잭션에서, 트랜잭션 중단 PSW는 TBEGIN 명령 다음에 오는 명령을 가리키도록 세트된다. 비제약 트랜잭션에 대한 예상되는 용법은, 만일 조건 코드가 비제로일 경우, TBEGIN 다음에 오는 명령이 제어를 어떤 중단 처리부 루틴으로 넘겨주는 분기 명령일 것이라는 점이다. 그러므로, 만일 중단 코드에 도달하면(질의 1210), 비제약 트랜잭션에서, 처리가 TBEGIN 명령 이후의 명령으로 진행된다(단계 1212). 그렇지 않으면, 처리는 TRANSACTION END 명령으로 완료된다.
- [0356] 추가 실시 예에서, 트랜잭션이 제약 트랜잭션으로서 처리되어야 하는지를 확인하기 위한 추가 검사들이 이루어진다. 예를 들어, 만일 오피코드가 그것은 제약 트랜잭션이지만 내포된 트랜잭션이라고 표시하면, 한 실시 예에서, 트랜잭션은 비제약 트랜잭션으로서 실행된다. 다른 검사들도 이루어질 수 있다.
- [0357] 도 13을 참조하여 제약 트랜잭션의 처리와 관련한 더 상세한 사항과, 구체적으로 TBEGINC 명령과 연관된 처리를 기술한다. TBEGINC 명령의 실행은 CPU가 제약 트랜잭션 실행 모드로 진입하게 하거나 또는 비제약 실행 모드에 남아 있게 한다. TBEGINC를 실행하는 CPU(즉, 프로세서)는 도 13의 로직을 수행한다.
- [0358] 도 13을 참조하면, TBEGINC 명령의 실행에 기초하여, 직렬화 기능이 수행된다(단계 1300). 직렬화 기능 또는 연산은, 다른 CPU들과 I/O 서브시스템에 의해 관찰될 때, 개념적으로 후속 스토리지 액세스들(및 관련된 참조 비트 및 변경 비트 세팅들)이 일어나기 전에, CPU에 의해 모든 개념적으로 이전 스토리지 액세스들(및, z/Architecture에서 예시로서 관련된 참조 비트 및 변경 비트 세팅들)을 완료하는 것을 포함한다. 직렬화는, ART 테이블 엔트리 및 DAT 테이블 엔트리 페칭과 연관된 것들을 제외하고, 스토리지에 대한 그리고 스토리지 키(key)들에 대한 모든 CPU 액세스의 순서에 영향을 준다.
- [0359] 트랜잭션 실행 모드에서 CPU에 의해 관찰될 때, 직렬화는(전술한 바와 같이) 정상적으로 작동한다. 다른 CPU들과 I/O 서브시스템에 의해 관찰될 때, CPU가 트랜잭션 실행 모드에 있는 동안 수행되는 직렬화 연산은, CPU가 트랜잭션 실행 모드에서 나갈 때 트랜잭션 내포 깊이를 제로로 감분하는 TRANSACTION END 명령(정상 종료)의 결과로서 또는 트랜잭션이 중단되는 결과로서 일어난다.
- [0360] 직렬화를 수행하는 것에 이어서, 예외가 인지되는지에 대한 판정이 이루어진다(질의 1302). 만일 그렇다면, 예외가 처리된다(단계 1304). 예를 들어, 만일 제어 레지스터 0의 비트 8인 트랜잭션 실행 제어가 0이면 특수 연산 예외(special operation exception)가 인지되고 그 연산은 억제된다. 추가로 예를 들면, 만일 명령의 비트들 16~19인 B₁ 필드가 비제로이면 명세 예외가 인지되고 그 연산은 억제된다; 만일 TBEGINC가 실행형 명령의 타겟이면 실행 예외가 인지되고 그 연산은 억제된다; 그리고 만일 트랜잭션 실행 퍼실리티가 그 구성에 설치되어 있지 않으면 연산 예외가 인지되고 그 연산은 억제된다. 만일 CPU가 이미 제약 트랜잭션 실행 모드에 있으면, 트랜잭션 제약 예외 프로그램 예외가 인지되고 그 연산은 억제된다. 추가로, 만일 트랜잭션 내포 깊이가, 1이 충분될 때, 모델 종속적인 최대 트랜잭션 내포 깊이를 초과하면, 그 트랜잭션은 중단 코드 13으로 중단된다. 다른 또는 각각 다른 예외들이 인지되고 처리될 수 있다.
- [0361] 하지만, 만일 예외가 없으면, 트랜잭션 내포 깊이가 제로인지에 대한 판정이 이루어진다(질의 1306). 만일 트랜잭션 내포 깊이가 제로이면, 트랜잭션 진단 블록 주소는 유효하지 않은 것으로 간주된다(단계 1308); 트랜잭션 중단 PSW는, 트랜잭션 중단 PSW의 명령 주소가 다음 순차 명령보다는 TBEGINC 명령을 지정하는 것을 제외하고, 현재 PSW의 콘텐츠로부터 세트된다(단계 1310); 그리고 범용 레지스터 세이브 마스크에 의해 지정될 때 범용 레지스터 쌍들의 콘텐츠는 프로그램에 의해 직접 액세스될 수 없는 모델 종속적인 위치에 세이브된다(단계 1312). 이어서, 내포 깊이가 1로 세트된다(단계 1314). 이어서, 부동 소수점 연산 허용(F) 및 프로그램 인터럽션 필터링 제어들(PIFC)의 유효 값이 제로로 세트된다(단계 1316). 이어서, 명령의 I₂ 필드의 비트 12 필드인 AR 수정 허용(A) 제어의 유효 값이 판정된다(단계 1318). 예를 들어, 유효 A 제어는 현재의 레벨 및 모든 외부 TBEGIN 명령들에 대한 TBEGINC 명령 내 A 제어의 논리적 AND(논리곱)이다.
- [0362] 질의 1306으로 돌아가서, 만일 트랜잭션 내포 깊이가 제로보다 크면, 내포 깊이는 1만큼 증분된다(단계 1320). 이어서, 부동 소수점 연산 허용(F)의 유효 값이 제로로 세트되고, 프로그램 인터럽션 필터링 제어(PIFC)의 유효 값은 변경되지 않는다(단계 1322). 그 다음에 처리는 단계 1318에서 계속된다. 한 실시 예에서, 트랜잭션의 성공적인 개시는 조건 코드 0으로 귀결된다. 이것으로 TBEGINC 명령을 실행하는 것과 연관된 로직의 한 실시 예를 마친다.

- [0363] 한 실시 예에서, 위에서 제공되는 예외 검사(exception checking)는 여러 순서로 일어날 수 있다. 예외 검사를 위한 한 가지 구체적인 순서는 다음과 같다:
- [0364] 일반적인 경우에 프로그램-인터럽션 조건들의 우선순위로써 동일한 우선순위를 갖는 예외들.
- [0365] 비제로 값을 보유하는 B₁ 필드로 인한 명세 예외.
- [0366] 트랜잭션 내포 깊이 초과로 인한 중단.
- [0367] 정상 완료로 인한 조건 코드 0.
- [0368] 추가로, 다음은 하나 또는 그 이상의 실시 예들에서 적용된다:
- [0369] 1. 범용 레지스터 세이브 마스크에 의해 세이브되도록 지정된 레지스터들은 트랜잭션이 TRANSACTION END를 이용하여 정상적으로 종료될 때가 아니라 트랜잭션이 중단될 경우에만 복원된다. 최외부 TRANSACTION BEGIN 명령의 GRSM에 의해 지정된 레지스터들만 중단시에 복원된다.
- [0370] I₂ 필드는 제약 트랜잭션에 의해 변경되는 입력 값들을 제공하는 모든 레지스터 쌍들을 지정해야 한다. 그러므로, 만일 트랜잭션이 중단되면, 상기 입력 레지스터 값들은 제약 트랜잭션이 재실행될 때 자신의 원래 콘텐츠로 복원될 것이다.
- [0371] 2. 대부분 모델들에서, TRANSACTION BEGIN시에 그리고 트랜잭션이 중단될 때 모두, 세이브 및 복원되는 데 필요한 레지스터들의 최소 수를 범용 레지스터 세이브 마스크에 명시함으로써 성능 향상이 실현될 수 있다.
- [0372] 3. 다음은 현재 트랜잭션 내포 깊이(TND)에 기초하여, 그리고 CPU가 비제약 또는 제약 트랜잭션 실행 모드에 있는지 간에 TND가 비제로일 때, TRANSACTION BEGIN 명령(TBEGIN과 TBEGINC 모두)의 결과들을 예시한다:
- [0373] 명령 TND=0
- [0374] TBEGIN 비제약 트랜잭션 실행 모드로 진입한다
- [0375] TBEGINC 제약 트랜잭션 실행 모드로 진입한다
- [0376] 명령 TND>0
- [0377] TBEGIN NTX 모드 CTX 모드
- [0378] 트랜잭션-실행 모드에서 계속한다 트랜잭션-제약 예외
- [0379] TBEGINC 트랜잭션-실행 모드에서 계속한다 트랜잭션-제약 예외
- [0380] 설명:
- [0381] CTX CPU는 제약 트랜잭션 실행 모드에 있다
- [0382] NTX CPU는 비제약 트랜잭션 실행 모드에 있다
- [0383] TND 명령의 시작시의 트랜잭션 내포 깊이.
- [0384] 여기에서 기술할 때, 한 특징에 따라서, 제약 트랜잭션은 그것이 완료될 수 없게 만드는 조건을 보유하지 않는다고 가정하여 완료가 보장된다. 완료를 보장하기 위해서, 트랜잭션을 실행하는 프로세서(예를 들어, CPU)는 어떤 액션들을 취할 수 있다. 예를 들어, 만일 제약 트랜잭션이 중단 조건을 가지면, CPU는 일시적으로 다음과 같이 할 수 있다:
- [0385] (a) 비순차적 실행(out-of-order execution)을 금지한다;
- [0386] (b) 충돌하는 스토리지 위치들을 다른 CPU들이 액세스하는 것을 금지한다;
- [0387] (c) 중단 처리시 임의 지연(random delays)을 유도한다; 및/또는
- [0388] (d) 성공적인 완료를 가능하게 하기 위해 다른 조치(measures)를 호출한다.
- [0389] 도 14를 참조하여 제약 트랜잭션의 완료를 용이하게 하기 위해 액션을 취하는 것과 연관된 처리에 관한 더 상세한 사항을 기술한다.

- [0390] 처음에, 프로세서는 TBEGINC 명령에 의해 시작된 제약 트랜잭션을 처리중이다(단계 1400). 만일 인터럽션을 초래하지 않는 중단 조건을 만나면(질의 1402), 프로세서는 선택에 따라(예를 들어, 중단의 수, 및/또는 중단 이유에 따라서), 여기에 기술된 바와 같이, 완료를 보장하기 위한 하나 또는 그 이상의 액션들을 취한다(단계 1404). 이어서, 프로세서는 TBEGINC 명령을 재실행하고(단계 1406), 처리는 단계 1400에서 계속된다.
- [0391] 질의 1402로 돌아가서, 만일 중단 조건이 없거나 인터럽션을 초래하는 중단 조건이 있으면, 처리는 트랜잭션의 완료(예를 들어, TEND) 또는 인터럽션이 있을 때까지 계속된다(질의 1408). 만일 인터럽션이 있으면, CPU는 그 인터럽션의 유형과 연관된 PSW를 로드한다. 완료 또는 인터럽션에 기초하여, 트랜잭션은 종료되고, 그렇지 않으면 처리를 계속한다.
- [0392] 한 실시 예에서, 트랜잭션이 중단될 때 액션을 수행할 것인지에 대한 결정 및/또는 수행될 액션의 선택은 중단 카운터의 값에 기초하며, 중단 카운터의 값은 트랜잭션의 중단 빈도에 대한 카운트를 제공한다.
- [0393] 상기 카운터는 트랜잭션이 중단될 때마다 카운트를 증분하고, 트랜잭션의 성공적인 완료시 또는 더 이상 트랜잭션의 재실행으로 이어지지 않는 인터럽션시 제로로 리셋된다. 만일 카운트가 임계값(예를 들어, 63 카운트)에 도달하면, 인터럽트가 제시되고, 트랜잭션은 실패이며 카운터는 리셋된다. 하지만, 카운트가 임계값에 도달하기 전에, 트랜잭션을 성공적으로 실행할 가능성을 높이기 위한 다수 액션들이 취해질 수 있다. 이 액션들은 트랜잭션을 실행하는 프로세서 내에서 수행될 액션들 및/또는 충돌하는 프로세서들(CPU들)에 대하여 수행될 액션들을 포함한다.
- [0394] 동일한 프로세서 내에서, 중단 카운트 및 동일한 또는 겹치는 중단 카운트를 갖는 액션들에 대한 선택 기준에 따라서 다음 중 하나 또는 그 이상의 액션들이 취해질 수 있다: 트랜잭션의 재실행(카운트 1~3); 예를 들어 스위치를 통한 분기 예측의 디스에이블화(disabling)(카운트 8~20); 한 예에서, 파이프라인의 백엔드(backend)가 비어 있을 때 명령 폐칭을 넘는 캐시 라인 경계만을 허용함으로써 달성되는 현재 명령의 캐시 라인 경계를 넘는 추론적 명령 폐칭의 디스에이블화(카운트 8~20); 예를 들어 스위치를 통한 슈퍼스칼라 디스패칭(superscalar dispatching)의 디스에이블화(카운트 8~20); 예를 들어 스위치를 통한 비순차적(out-of-order) 실행의 디스에이블화(카운트 8~20); 폐치-전용 요청들에 대하여, 모든 캐시 미스(cache misses)를 독점적으로 폐칭(카운트 8~20); 파이프라인 전체에 걸쳐서 한 번에 하나의 명령 실행(카운트 21~23); 슈퍼스칼라 디스패칭의 디스에이블화(카운트 24~28); 및 파이프라인 전체에 걸쳐서 한 번에 하나의 명령 실행, 및 모든 캐시 미스를 독점적으로 폐칭(카운트 32~63). 이 액션들 중 일부를 채용할시에, 프로세서 파이프라인의 추론 적극성(speculation aggressiveness)은 반복적인 트랜잭션 중단시에 연속하여 제한된다. 프로세서 파이프라인의 최대한의 추론 적극성은 트랜잭션의 성공적인 완료 또는 더 이상 트랜잭션 제시도로 이어지지 않는 인터럽션에 기초하여 복원된다.
- [0395] 한 실시 예에서, 특정한 카운트에 대한 모든 액션들은 예를 들어 동시에 수행된다. 또 다른 실시 예에서, 오버랩(overlap)이 있을 때, 한 액션이, 예를 들어, 목록에서 순서대로 선택하거나, 선택 기능에 기초하여 무작위로 선택하거나, 또는 다른 기술에 기초하여 다른 액션보다 우선하여 선택된다.
- [0396] 또한, 한 예에서, 다음 레벨의 중단 카운트들로 진행할 때, 선택된 액션들이 이전 액션들을 대체한다. 예를 들어, 이전에 디스에이블된 것은 인에이블되고, 새로운 액션들이 취해진다. 하지만, 또 다른 실시 예에서, 새로운 액션들은 이전 액션들에 추가된다. 그러므로, 여기에서 사용할 때, 또 하나의 액션은 이전 액션을 대체하거나, 이전 액션에 추가되거나, 또는 이들의 어떤 조합이 이루어진다. 또한, 또 하나의 액션은 이전 액션과 동일하거나 또는 다른 액션일 수 있다.
- [0397] 한 예로서, 상기 액션은 특수한 처리 모드를 인에이블하는 하드웨어 비트를 세팅(예를 들어, 분기 예측 디스에이블화, 등)하는 펌웨어에 의해 수행된다. 상기 하드웨어는 카운터가 리셋되는 동일한 조건에서 이 비트를 자동적으로 리셋한다.
- [0398] 트랜잭션이 하나 또는 그 이상의 상기 액션들을 수행한 후에도 계속 중단되고 카운트가 선택된 값 또는 레벨에 도달하면, 충돌하는 프로세서들에 대한 액션들이 취해질 수 있다. 예를 들어, 중단 4~15회시에, 무작위 지연(random delays)(즉, 트랜잭션의 재실행이 특정한 시간 동안 또는 몇 번의 기계 주기 동안 등 지연됨)이 수행될 수 있고; 중단 16~23회시에, 이 프로세서가 실행되고 있는 파티션(예를 들어, LPAR 존)의 다른 프로세서들에 대한 세마포어(semaphore)가 획득될 수 있고, 그렇게 함으로써 다른 프로세서들에서 연산들을 멈출 수 있다. 이와 유사하게, 카운트 24~63회시에, 전체 시스템에 대한 세마포어가 획득될 수 있고, 그 시스템 내의 모든 프로세서들은 세마포어가 지워질 때까지 연산들을 멈추어야 한다. 이 처리 모드에서, 세마포어가 획득되도록 선택된 레벨에 도달하는 것에 기초하여, 예를 들어 비교 및 교환(Compare and Swap) 메커니즘을 이용하여 세마포어를 획득

득하기 위해 펌웨어 루틴이 호출된다. 세마포어가 획득되면, 인터럽트가 적절한 프로세서들(예를 들어, 동일한 파티션 내의 프로세서들 또는 시스템의 모든 프로세서들 또는 기타 다른 하부세트)에게 전달된다. 그 다음에, 프로세서는 상기 펌웨어 루틴에서 나가서 성공적인 완료 또는 인터럽션이 있을 때까지 트랜잭션을 한 번 또는 그 이상 재실행한다. 트랜잭션이 성공적으로 완료되거나 더 이상 재시도되지 않게될 때 세마포어는 리셋된다.

- [0399] 상기 실시 예에서 액션들은 중단 카운트에 기초하여 취해지지만, 다른 실시 예에서 액션은 중단의 이유에 기초하여 및/또는 카운트에 기초하여 취해진다. 그러므로, 액션들은 중단 조건에 기초하여 취해진다고 말할 수 있으며, 이때 상기 조건은 카운트, 중단 이유 또는 카운트와 중단 이유의 조합이다. 예를 들면, 프로세서는 중단이 또 다른 CPU에 의한 것이었음을 검출하고, 그 다음에 세마포어를 획득할 수 있다. 이것은 또한 카운트하는 것과 조합될 수 있는데, 예를 들면, "만일 중단 > 16이고 중단이 또 다른 CPU와의 충돌로 인한 것이면, 세마포어를 획득한다"이다. 많은 변형예와 가능한 일들이 존재한다.
- [0400] 한 실시 예에서, 제약 트랜잭션에서 상기 기술들을 사용하여, 성공적인 완료가 가능하다고(예를 들어, 제약 제한사항 또는 기타 다른 위반사항을 위반하지 않음) 가정하고, 트랜잭션의 성공적인 완료를 보장한다.
- [0401] 요약하자면, 제약 트랜잭션의 처리는 다음과 같다:
- [0402] • 만일 이미 제약-TX 모드에 있으면, 트랜잭션-제약 예외가 인지된다.
- [0403] • 만일 현재 TND(트랜잭션 내포 깊이)>0이면, 마치 비제약 트랜잭션인 것처럼 실행이 진행된다
- [0404] ◦ 유효 F 제어는 제로로 세트된다
- [0405] ◦ 유효 PIFC는 변경되지 않는다
- [0406] ◦ 외부 비제약 TX가 제약 TX를 사용 또는 사용하지 않을 수 있는 서비스 기능(service function)을 호출하도록 허용한다.
- [0407] • 만일 현재 TND=0이면:
- [0408] ◦ 트랜잭션 진단 블록 주소는 유효하지 않다
- [0409] - 중단시에 명령-명시 TDB는 저장되지 않음
- [0410] ◦ 트랜잭션 중단 PSW는 TBEGINC의 주소로 세트된다
- [0411] - 다음 순차 명령이 아님
- [0412] ◦ 모델 종속적 위치에 세이브된 GRSM에 의해 지정된 범용 레지스터 쌍들은 프로그램에 의해 액세스될 수 없음
- [0413] ◦ 트랜잭션 토큰이 (D₂ 오퍼랜드로부터) 선택에 따라 형성된다. 트랜잭션 토큰은 트랜잭션의 식별자이다. 그것은 스토리지 오퍼랜드 주소 또는 또 다른 값과 같을 수 있다.
- [0414] • 유효 A = TBEGINC A & 임의 외부 A
- [0415] • TND가 증분됨
- [0416] ◦ 만일 TND가 0에서 1로 변하면, CPU는 제약 TX 모드로 진입한다
- [0417] ◦ 그렇지 않으면, CPU는 비제약 TX 모드로 남아 있다
- [0418] • 명령은 CC0으로 완료된다
- [0419] • 예외들:
- [0420] ◦ 만일 B₁ 필드가 비제로이면 명세 예외(PIC(프로그램 인터럽션 코드) 0006)

- [0421] ◦ 만일 트랜잭션 실행 제어(CR0.8)가 제로면 특수 연산 예외(PIC 0013 hex)
- [0422] ◦ 만일 제약 TX 모드에서 발행되었으면 트랜잭션 제약 예외(PIC 0018 hex)
- [0423] ◦ 만일 제약 트랜잭션 실행 퍼실리티가 설치되어 있지 않으면 연산 예외(PIC 0001)
- [0424] ◦ 만일 명령이 실행형 명령의 타겟이면 실행 예외(PIC 0003)
- [0425] ◦ 만일 내포 깊이가 초과되면 중단 코드 13
- [0426] • 제약 트랜잭션에서 중단 조건들:
- [0427] ◦ 중단 PSW가 TBEGINC 명령을 가리킨다
- [0428] - 그것의 다음에 오는 명령이 아님
- [0429] - 중단 조건이 전체 TX의 재구동을 일으킨다
- [0430] * 페일 경로 없음(No fail path)
- [0431] ◦ 재구동시 성공적인 완료를 보장하기 위해 CPU가 특수 조치를 취한다
- [0432] ◦ 지속적인(persistent) 충돌, 인터럽트, 또는 제약 위반이 없다고 가정하고, 트랜잭션은 최종적인 완료가 보장된다.
- [0433] • 제약 위반:
- [0434] ◦ PIC 0018 hex - 트랜잭션 제약의 위반을 표시한다
- [0435] ◦ 아니면, 트랜잭션은 마치 비제약인 것처럼 실행된다
- [0436] 위에 제공된 것은 로킹(locking) 같은 고전적인(개괄적인) 직렬화 없이 메모리 내 다중의 비인접 객체들을 업데이트하는 효율적인 수단이며, 이것은 유의미한 멀티프로세서 성능 향상의 가능성을 제공한다. 즉, 다중의 비인접 객체들은 로크(locks)와 세마포어(semaphores) 같은 고전적인 기술들에 의해 제공되는 더 개괄적인 스토리지-액세스 배열의 실행 없이 업데이트된다. 제약 트랜잭션들은 단순하고 작은 공간(small-footprint)의 업데이트를 위해 제공된다.
- [0437] 트랜잭션 실행은 로크 생략(lock elision)을 포함한(그러나 이에 한정되지 않음) 여러 상황에서 사용될 수 있다. 로크 생략 사용의 한 예가 도 15a~15b와 아래에 제공되는 코드 조각(code fragment)을 참조하여 기술된다.
- [0438] 도 15a는 복수의 큐 엘리먼트들(1502a~1502d)의 이중 연결 목록(1500)을 도시한다. 새로운 큐 엘리먼트(1502e)가 큐 엘리먼트들의 이중 연결 목록(1500)으로 삽입될 예정이다. 각 큐 엘리먼트(1502a~1502e)는 전방향 포인터(1504a~1504e)와 역방향 포인터(1506a~1506e)를 포함한다. 도 15b에서 보는 바와 같이, 큐 엘리먼트(1502e)를 큐 엘리먼트(1502b)와 큐 엘리먼트(1502c) 사이에 추가하기 위해, (1) 역방향 포인터(1506e)가 큐 엘리먼트(1502b)를 가리키도록 세트되고, (2) 전방향 포인터(1504e)가 큐 엘리먼트(1502c)를 가리키도록 세트되고, (3) 역방향 포인터(1506c)가 큐 엘리먼트(1502e)를 가리키도록 세트되고, 그리고 (4) 전방향 포인터(1504b)가 큐 엘리먼트(1502e)를 가리키도록 세트된다.
- [0439] 아래는 도 15a~15b에 대응하는 코드 조각 예시이다:
- [0440] * R1 - 삽입될 새로운 큐 엘리먼트의 주소.
- [0441] * R2 - 삽입 지점의 주소; 새로운 엘리먼트는 R2가 가리키는 엘리먼트 앞에 삽입된다.
- [0442] NEW USING QEL, R1
- [0443] CURR USING QEL, R2
- [0444] LHI R15,10 재시도 카운트 로드.

- [0445] LOOP TBEGIN TDB,X'C000' 트랜잭션 시작(GR들 0~3 세이브)
- [0446] JNZ ABORTED 비제로 CC는 중단됨을 의미함.
- [0447] LG R3,CURR.BWD 이전 엘리먼트를 가리킴.
- [0448] PREV USING QEL,R3 주소지정 가능하게 만듦.
- [0449] STG R1,PREV.FWD 이전 전방향 포인터 업데이트.
- [0450] STG R1,CURR.BWD 현재 역방향 포인터 업데이트.
- [0451] STG R2,NEW.FWD 새로운 전방향 포인터 업데이트.
- [0452] STG R3,NEW.BWD 새로운 역방향 포인터 업데이트.
- [0453] TEND 트랜잭션 종료.
- [0454] ...
- [0455] ABORTED JO NO_RETRY CC3: 재시도 불가능 중단.
- [0456] JCT R15,LOOP 트랜잭션 몇 번 재시도.
- [0457] J NO_RETRY No joy after 10x; do it the hard way.
- [0458] 한 예에서, 만일 트랜잭션이 로크 생략에 사용되지만, 폴백 경로가 로크를 사용하면, 트랜잭션은 적어도 로크 워드를 폐치하여 그것이 이용 가능한지 알아본다. 프로세서는 만일 또 다른 CPU가 로크를 비트랜잭션적으로 액세스하면 트랜잭션이 중단되도록 보장한다.
- [0459] 여기에서 사용할 때, 스토리지, 중앙 스토리지, 메인 스토리지, 메인 메모리 및 메모리는 용법에 의해 암시적으로 또는 명시적으로 다르게 언급되지 않는 한 교환하여 사용할 수 있다. 또한, 한 실시 예에서는 트랜잭션을 효과적으로 지연시키는(delaying) 것은 선택된 트랜잭션이 완료될 때까지 메인 메모리에 대한 트랜잭션 저장을 커밋하는 것을 효과적으로 지연시키는 것을 포함하고; 다른 실시 예에서는 트랜잭션을 효과적으로 지연시키는 것은 메모리에 대한 트랜잭션 업데이트를 허용하지만 구(old) 값들을 보관하고 중단시에 메모리를 구(old) 값들로 복원시키는 것을 포함한다.
- [0460] 이 기술분야에서 통상의 지식을 가진 자는 인식할 수 있는 바와 같이, 하나 또는 그 이상의 특징들은 시스템, 방법 또는 컴퓨터 프로그램 제품으로 구현될 수 있다. 따라서, 하나 또는 그 이상의 특징들은 전적으로 하드웨어 실시 예, 전적으로 소프트웨어 실시 예(펌웨어, 상주 소프트웨어, 마이크로-코드 등 포함) 또는 소프트웨어와 하드웨어 특징들을 조합한 실시 예(여기에서는 모두 "회로", "모듈", "시스템"으로 불릴 수 있음)의 형태를 취할 수 있다. 또한, 하나 또는 그 이상의 실시 예들은 컴퓨터 판독 가능 프로그램 코드가 그 위에 구현된 하나 또는 그 이상의 컴퓨터 판독 가능 매체(들)에 구현된 컴퓨터 프로그램 제품의 형태를 취할 수 있다.
- [0461] 하나 또는 그 이상의 컴퓨터 판독 가능 매체(들)의 임의의 조합이 사용될 수 있다. 컴퓨터 판독 가능 매체는 컴퓨터 판독 가능 스토리지 매체일 수 있다. 컴퓨터 판독 가능 스토리지 매체는, 예를 들어, 전자, 자기, 광학, 전자 자기, 적외선 또는 반도체 시스템, 장치, 또는 디바이스이거나 전술한 것들의 모든 적절한 조합으로 될 수 있으나 그에 한정되지는 않는다. 컴퓨터 판독 가능 스토리지 매체의 더 구체적인 예들(비포괄적인 목록)에는 하나 또는 그 이상의 와이어들을 갖는 전기 배선(electrical connection), 휴대용 컴퓨터 디스켓, 하드 디스크, 랜덤 액세스 메모리(RAM), 판독-전용 메모리(ROM), 소거 및 프로그램가능 판독-전용 메모리(EPROM 또는 플래시 메모리), 광섬유, 휴대용 콤팩트 디스크 판독-전용 메모리(CD-ROM), 광 스토리지 디바이스, 자기 스토리지 디바이스, 또는 전술한 것들의 모든 적절한 조합이 포함된다. 이 문서의 컨텍스트에서, 컴퓨터 판독 가능 스토리지 매체는 명령 실행을 위한 시스템, 장치, 또는 디바이스에 의해 또는 그와 연결하여 사용할 프로그램을 포함 또는 저장할 수 있는 모든 유형의(tangible) 매체일 수 있다.
- [0462] 이제 도 16를 참조하면, 한 예에서, 컴퓨터 프로그램 제품(1600)은 예를 들어 하나 또는 그 이상의 비-일시적인(non-transitory) 컴퓨터 판독 가능 스토리지 매체(1602)를 포함하며 이 매체상에 컴퓨터 판독 가능 프로그램 코드 수단 또는 로직(1604)을 저장하여 하나 또는 그 이상의 실시 예들을 제공 및 가능하게 만든다.
- [0463] 컴퓨터 판독 가능 매체상에 구현된 프로그램 코드는 무선, 유선, 광섬유 케이블, RF 등 또는 전술한 것들의 적절한 조합으로 된 것을 포함한다(그러나 이에 한정되지는 않는) 적절한 매체를 사용하여 전송될 수 있다.

- [0464] 하나 또는 그 이상의 실시 예들에 대한 동작들을 실행하기 위한 컴퓨터 프로그램 코드는 Java, Smalltalk, C++ 또는 그와 유사 언어 등의 객체 지향 프로그래밍 언어와 "C" 프로그래밍 언어, 어셈블러 언어 또는 그와 유사한 언어 등의 종래의 절차적 프로그래밍 언어들을 포함하여, 하나 또는 그 이상의 프로그래밍 언어들을 조합하여 작성될 수 있다. 상기 프로그램 코드는 전적으로 사용자의 컴퓨터상에서, 부분적으로 사용자의 컴퓨터상에서, 독립형(stand-alone) 소프트웨어 패키지로써, 부분적으로 사용자의 컴퓨터상에서 그리고 부분적으로 원격 컴퓨터상에서 또는 전적으로 원격 컴퓨터나 서버상에서 실행될 수 있다. 위에서 마지막의 경우에, 원격 컴퓨터는 근거리 통신망(LAN) 또는 광역 통신망(WAN)을 포함한 모든 종류의 네트워크를 통해서 사용자의 컴퓨터에 접속될 수 있고, 또는 이 접속은 (예를 들어, 인터넷 서비스 제공자를 이용한 인터넷을 통해서) 외부 컴퓨터에 이루어질 수도 있다.
- [0465] 여기에서는 방법들, 장치들(시스템들) 및 컴퓨터 프로그램 제품들의 순서 예시도들 및/또는 블록도들을 참조하여 하나 또는 그 이상의 실시 예들을 기술한다. 순서 예시도들 및/또는 블록도들의 각 블록과 순서 예시도들 및/또는 블록도들 내 블록들의 조합들은 컴퓨터 프로그램 명령들에 의해 구현될 수 있다는 것을 이해할 수 있을 것이다. 이 컴퓨터 프로그램 명령들은 범용 컴퓨터, 특수목적용 컴퓨터, 또는 기타 프로그램가능 데이터 처리 장치의 프로세서에 제공되어 머신(machine)을 생성하고, 그렇게 하여 그 명령들이 상기 컴퓨터 또는 기타 프로그램가능 데이터 처리 장치의 프로세서를 통해서 실행되어, 상기 순서도 및/또는 블록도의 블록 또는 블록들에 명시된 기능들/동작들을 구현하기 위한 수단을 생성할 수 있다.
- [0466] 상기 컴퓨터 프로그램 명령들은 또한 컴퓨터 판독 가능 매체에 저장될 수 있으며, 컴퓨터, 기타 프로그램가능 데이터 처리 장치 또는 다른 디바이스들에 지시하여 상기 컴퓨터 판독 가능 매체에 저장된 명령들이 상기 순서도 및/또는 블록도의 블록 또는 블록들에 명시된 기능/동작을 구현하는 명령들을 포함하는 제조품(an article of manufacture)을 생성하도록 특정한 방식으로 기능하게 할 수 있다.
- [0467] 상기 컴퓨터 프로그램 명령들은 또한 컴퓨터, 기타 프로그램가능 데이터 처리 장치, 또는 다른 디바이스들에 로드되어, 컴퓨터, 기타 프로그램가능 장치 또는 다른 디바이스들에서 일련의 동작 단계들이 수행되게 하여 컴퓨터 구현 프로세스를 생성하며, 그렇게 하여 상기 컴퓨터 또는 기타 프로그램가능 장치상에서 실행되는 명령들이 순서도 및/또는 블록도의 블록 또는 블록들에 명시된 기능들/동작들을 구현하기 위한 프로세스들을 제공할 수 있다.
- [0468] 도면들 내 순서도 및 블록도들은 여러 실시 예들에 따른 시스템들, 방법들 및 컴퓨터 프로그램 제품들의 가능한 구현들의 아키텍처, 기능(functionality), 및 연산(operation)을 예시한다. 이와 관련하여, 상기 순서도 또는 블록도들 내 각 블록은 상기 명시된 논리적 기능(들)을 구현하기 위한 하나 또는 그 이상의 실행 가능한 명령들을 포함한 모듈, 세그먼트 또는 코드의 일부분을 나타낼 수 있다. 일부 다른 구현들에서, 상기 블록에 언급되는 기능들은 도면들에 언급된 순서와 다르게 일어날 수도 있다는 것에 또한 유의해야 한다. 예를 들면, 연속적으로 도시된 두 개의 블록들은 실제로는 사실상 동시에 실행될 수도 있고, 또는 이 두 블록들은 때때로 관련된 기능에 따라서는 역순으로 실행될 수도 있다. 블록도들 및/또는 순서 예시도의 각 블록, 및 블록도들 및/또는 순서 예시도 내 블록들의 조합들은 특수목적용 하드웨어 및 컴퓨터 명령들의 명시된 기능들 또는 동작들, 또는 이들의 조합들을 수행하는 특수목적용 하드웨어-기반 시스템들에 의해 구현될 수 있다는 것에 또한 유의한다.
- [0469] 진술한 것에 추가하여, 하나 또는 그 이상의 특징들은 컴퓨터 환경의 관리를 서비스하는 서비스 제공자에 의해 제공, 공급, 배치, 관리, 서비스 등이 될 수 있다. 예를 들면, 서비스 제공자는 하나 또는 그 이상의 고객들을 위해 하나 또는 그 이상의 특징들을 수행하는 컴퓨터 코드 및/또는 컴퓨터 인프라스트럭처의 제작, 유지, 지원 등을 할 수 있다. 그 대가로, 서비스 제공자는 가입제(subscription) 및/또는 수수료 약정에 따라 고객으로부터 대금을 수령할 수 있으며, 이는 예이다. 또한, 서비스 제공자는 하나 또는 그 이상의 제3자들에게 광고 콘텐츠를 판매하고 대금을 수령할 수 있다.
- [0470] 한 특징으로, 하나 또는 그 이상의 실시 예들을 수행하기 위한 애플리케이션이 배치될 수 있다. 한 예로서, 애플리케이션의 배치는 하나 또는 그 이상의 실시 예들을 수행하는 데 실시 가능한 컴퓨터 인프라스트럭처를 제공하는 것을 포함한다.
- [0471] 추가 특징으로서, 컴퓨터 판독 가능 코드를 컴퓨팅 시스템으로 통합하는 것을 포함하는 컴퓨팅 인프라스트럭처가 배치될 수 있으며, 그 컴퓨팅 시스템에서 상기 코드는 상기 컴퓨팅 시스템과 결합하여 하나 또는 그 이상의 실시 예들을 수행하는 것이 가능하다.
- [0472] 추가 특징으로서, 컴퓨터 판독 가능 코드를 컴퓨터 시스템으로 통합시키는 것을 포함하는 컴퓨팅 인프라스트럭

처 통합을 위한 프로세스가 제공될 수 있다. 상기 컴퓨터 시스템은 컴퓨터 판독 가능 매체를 포함하고, 상기 컴퓨터 시스템에서 상기 컴퓨터 매체는 하나 또는 그 이상의 실시 예들을 포함한다. 상기 코드는 상기 컴퓨터 시스템과 결합하여 하나 또는 그 이상의 실시 예들을 수행하는 것이 가능하다.

[0473] 위에서 여러 실시 예들이 기술되었지만, 이들은 단지 예시일 뿐이다. 예를 들면, 다른 아키텍처들로 된 컴퓨팅 환경들이 하나 또는 그 이상의 실시 예들을 포함하고 사용하는 데 사용될 수 있다. 또한, 각각 다른(different) 명령들, 명령 포맷들, 명령 필드들 및/또는 명령 값들이 사용될 수 있다. 또한, 각각 다른(different), 다른(other), 및/또는 추가 제한사항들/제약사항들이 제공/사용될 수 있다. 많은 변형 예들이 가능하다.

[0474] 또한, 다른 종류의 컴퓨팅 환경들도 이득을 얻을 수 있고 사용될 수 있다. 예로서, 프로그램 코드를 저장 및/또는 실행하기에 적합한 데이터 처리 시스템이 사용될 수 있으며, 이 시스템은 시스템 버스를 통해서 메모리 엘리먼트들에 직접적으로 또는 간접적으로 결합된 적어도 두 개의 프로세서를 포함한다. 상기 메모리 엘리먼트들은, 예를 들어 프로그램 코드의 실제 실행 동안 사용되는 로컬 메모리, 대용량 스토리지(bulk storage), 및 코드가 실행 동안에 대용량 스토리지로부터 검색되어야 하는 횟수를 줄이기 위해 적어도 일부 프로그램 코드의 임시 저장(temporary storage)을 제공하는 캐시 메모리를 포함한다.

[0475] 입력/출력 또는 I/O 디바이스들(키보드, 디스플레이, 포인팅 디바이스, DASD, 테이프, CD, DVD, 썸 드라이브 및 기타 메모리 매체 등을 포함하나 이에 한정되지는 않음)은 직접 또는 중개(intervening) I/O 컨트롤러들을 통해서 상기 시스템에 결합될 수 있다. 네트워크 어댑터 또한 상기 시스템에 결합되어 상기 데이터 처리 시스템이 중개하는 사실 또는 공공 네트워크를 통해서 기타 데이터 처리 시스템 또는 원격 포인터 또는 스토리지 디바이스에 결합되는 것을 가능하게 한다. 모뎀, 케이블 모뎀, 및 이더넷 카드는 이용 가능한 네트워크 어댑터의 단지 일부 예이다.

[0476] 도 17을 참조하면, 하나 또는 그 이상의 실시 예들을 구현하기 위한 호스트 컴퓨터 시스템(5000)의 대표적인 컴포넌트들이 도시된다. 대표적인 호스트 컴퓨터(5000)는 컴퓨터 메모리(즉, 중앙 스토리지)(5002)와 통신하는 하나 또는 그 이상의 CPU들(5001)을 포함하고, 또한 스토리지 매체 디바이스들(5011)로 그리고 다른 컴퓨터들 또는 SAN들 등과 통신하기 위한 네트워크들(5010)로 가는 I/O 인터페이스들을 포함한다. CPU(5001)는 아키텍처화된 명령 세트((architected instruction set)와 아키텍처화된 기능(architected functionality)을 갖는 아키텍처에 부합한다. CPU(5001)는 액세스 레지스터 변환(ART)(5012)을 가질 수 있으며, 이것은 프로그램 주소들(가상 주소들)을 메모리의 실제 주소들로 변환하는 동적 주소 변환(DAT)(5003)에 의해 사용될 주소 공간을 선택하기 위한 ART 색인 버퍼(ALB)(5013)를 포함한다. DAT는 통상적으로 컴퓨터 메모리(5002)의 블록에 나중에 액세스할 때 주소 변환의 지연이 필요 없도록 변환들을 캐시하기 위한 변환 색인 버퍼(TLB, translation lookaside buffer)(5007)를 포함한다. 통상적으로, 캐시(5009)는 컴퓨터 메모리(5002)와 프로세서(5001) 사이에서 사용된다. 캐시(5009)는 하나 이상의 CPU가 이용 가능한 큰 캐시(large cache)와 그 큰 캐시와 각 CPU 사이에 있는 더 작고 더 빠른 (더 하위 레벨) 캐시들을 갖는 계층형(hierarchical)일 수 있다. 어떤 구현들에서는, 더 하위 레벨(lower level) 캐시들은 명령 페치와 데이터 액세스를 위한 별개의(separate) 하위 레벨 캐시들을 제공하기 위해 분할된다. 한 실시 예에서, TX 퍼실리티를 위해서, 트랜잭션 진단 블록(TDB)(5100) 및 하나 또는 그 이상의 버퍼들(5101)이 하나 또는 그 이상의 캐시(5009)와 메모리(5002)에 저장될 수 있다. 한 예에서, TX 모드에서, 데이터는 처음에 TX 버퍼에 저장되고, TX 모드가 종료될 때(예를 들어, 최외부 TEND) 상기 버퍼 내 데이터는 메모리에 저장(커밋)되거나, 또는 만일 중단(abort)이 있으면 상기 버퍼 내 데이터는 폐기된다.

[0477] 한 실시 예에서, 한 명령이 명령 페치 유닛(5004)에 의해 캐시(5009)를 통해서 메모리(5002)로부터 페치된다. 명령은 명령 디코드 유닛(instruction decode unit)(5006)에서 디코드되고 (어떤 실시 예들에서는 다른 명령들과 함께) 명령 실행 유닛 또는 유닛들(5008)로 디스패치된다(dispatched). 통상적으로 몇 가지의 실행 유닛들(5008)이 채용되며, 예를 들면 산술 실행 유닛(arithmetic execution unit), 부동 소수점 실행 유닛(floating point execution unit) 및 분기 명령 실행 유닛(branch instruction execution unit)이 있다. 또한, TX 퍼실리티의 한 실시 예에서, 여러 TX 제어들(5110)이 채용될 수 있다. 명령은 실행 유닛에 의해 실행되고, 명령이 명시한 레지스터들 또는 메모리로부터 필요한 만큼 오퍼랜드들에 액세스한다. 만일 오퍼랜드가 메모리(5002)로부터 액세스(로드 또는 저장)되면, 로드/저장 유닛(load/store unit)(5005)이 통상적으로 실행되는 명령의 제어에 따라 액세스를 처리한다. 명령들은 하드웨어 회로들에서 또는 내부 마이크로코드(펌웨어)에서 또는 이 둘의 조합에 의해서 실행될 수 있다.

[0478] TX 퍼실리티의 일 특징에 따라서, 프로세서(5001)는 또한 PSW(예를 들어, TX 및/또는 중단 PSW)(5102), 내포 깊이(nesting depth)(5104), TDBA(5106), 및 하나 또는 그 이상의 제어 레지스터들(5108)을 포함한다.

- [0479] 기술한 바와 같이, 컴퓨터 시스템은 로컬 (또는 메인) 스토리지에 정보를 포함하고, 또한 주소지정 (addressing), 보호(protection), 그리고 참조 및 변경 기록(reference and change recording)을 포함한다. 주소지정의 몇 가지 예로는 주소의 형식(format of addresses), 주소 공간의 개념(concept of address spaces), 주소의 여러 유형(various types of addresses), 및 한 유형의 주소가 또 다른 유형의 주소로 변환되는 방식(manner)이 있다. 메인 스토리지의 일부는 영구적으로 할당된 스토리지 위치들을 포함한다. 메인 스토리지는 시스템에 데이터의 직접 주소지정 가능한 고속 액세스 스토리지(fast-access storage)를 제공한다. 데이터와 프로그램들은 모두 (입력 디바이스들로부터) 메인 스토리지로 로드된 후에 처리될 수 있다.
- [0480] 메인 스토리지는 때때로 캐시라고 불리는 하나 또는 그 이상의 더 작고 더 고속의 액세스 버퍼 스토리지들을 포함한다. 캐시는 통상적으로 CPU 또는 I/O 프로세서와 물리적으로 연관된다. 구별되는(distinct) 스토리지 매체의 물리적 구축과 사용의 영향들은, 수행을 제외하고는, 일반적으로 프로그램에 의해 관찰되지 않는다.
- [0481] 명령들 용과 데이터 오퍼랜드들 용으로 별개 캐시들이 유지될 수 있다. 캐시 내의 정보는 캐시 블록(cache block) 또는 캐시 라인(또는 줄여서 라인)이라 불리는 인테그럴 범위(integral boundary)상의 인접 바이트들에 보존된다. 어떤 모델은 캐시 라인의 사이즈를 바이트로 회신하는 EXTRACT CACHE ATTRIBUTE 명령을 제공할 수 있다. 어떤 모델은 또한 스토리지를 데이터 또는 명령 캐시로의 프리페치(prefetch) 또는 캐시로부터 데이터의 해제를 실현하는 PREFETCH DATA 명령과 PREFETCH DATA RELATIVE LONG 명령을 제공할 수 있다.
- [0482] 스토리지는 비트들의 긴 수평의 열(a long horizontal string of bits)로 보인다. 대부분의 연산들에 있어서, 스토리지에 대한 액세스는 좌측-에서-우측(left-to-right) 순으로 진행된다. 비트들의 문자열(string)은 8비트의 유닛들로 세분된다. 8-비트 단위를 바이트(byte)라 부르고, 이것은 모든 정보 포맷들의 기본적인 빌딩 블록(building block)이다. 스토리지에서 각 바이트 위치는 음이 아닌 고유한 정수로 식별되고, 이것은 그 바이트 위치의 주소, 또는, 간단히 말해서 바이트 주소(byte address)이다. 인접 바이트 위치들은 좌측의 0부터 시작해서 좌측-에서-우측 순으로 진행되는 연속되는 주소들이다. 주소들은 무부호 2진 정수들이며 24, 31, 또는 64비트이다.
- [0483] 정보는 스토리지와 CPU 또는 채널 서브시스템 사이에서, 1 바이트 또는 바이트들의 그룹으로, 한 번에 전송된다. 다르게 명시되지 않으면, 예를 들어, z/Architecture에서 스토리지 내 바이트들의 그룹은 그 그룹의 제일 좌측 바이트에 의해 주소지정된다. 그룹 내 바이트의 수는 수행될 연산에 의해 암시되거나 분명하게 명시된다. CPU 연산에서 사용될 때, 바이트들의 그룹은 필드(field)라 불린다. 각 바이트들의 그룹 내에서, 예를 들어, z/Architecture에서, 비트들은 좌측-에서-우측 순으로 번호가 붙는다. z/Architecture에서, 제일 좌측 비트들은 때때로 "상위(high-order)" 비트들로 불리고 제일 우측 비트들은 "하위(low-order)" 비트들로 불린다. 그러나 비트 번호는 스토리지 주소가 아니다. 바이트만 주소지정될 수 있다. 스토리지 내 한 바이트의 개별 비트들에서 연산하기 위해서는, 전체 바이트가 액세스된다. 한 바이트 내 비트들은 (예를 들어, z/Architecture에서) 0에서 7까지, 좌측에서 우측으로 번호가 붙는다. 한 주소 내 비트들은 24-비트 주소에서는 8-31 또는 40-63으로, 또는 31-비트 주소에서는 1-31 또는 33-63으로 번호가 붙을 수 있고; 64-비트 주소에서는 0-63으로 번호가 붙는다. 한 예에서, 비트들 8-31과 1~31은 32 비트 넓이인 위치(예를 들어, 레지스터)에 있는 주소들에 적용되고, 비트들 40-63과 33~63은 64비트 넓이 위치에 있는 주소들에 적용된다. 다른 고정-길이 포맷의 다수 바이트들 내에서, 그 포맷을 이루는 비트들은 0부터 시작해서 연속적으로 번호가 붙는다. 예러 검출의 목적을 위해서, 그리고 바람직하게는 교정을 위해서, 하나 또는 그 이상의 검사용 비트들이 각 바이트와 또는 바이트들의 그룹과 함께 전송된다. 이러한 검사용 비트들은 머신에 의해 자동적으로 생성되며 프로그램에 의해 직접적으로 제어될 수 없다. 스토리지 용량은 바이트 수로 표시된다. 스토리지-오퍼랜드 필드의 길이가 명령의 연산 코드에 의해 암시될 때, 그 필드는 고정 길이(fixed length)를 가졌다고 말하며, 그 길이는 1, 2, 4, 8, 또는 16 바이트일 수 있다. 어떤 명령들에는 더 큰 필드들이 암시될 수 있다. 스토리지-오퍼랜드 필드의 길이가 암시되지 않고 분명하게 언급될 때, 그 필드는 가변 길이(variable length)를 가졌다고 말한다. 가변-길이 오퍼랜드는 길이가 1 바이트의 증분들 만큼씩 (또는 어떤 명령들에서는, 2 바이트의 배수로 또는 다른 배수들로) 변할 수 있다. 정보가 스토리지에 배치될 때, 비록 스토리지에 대한 물리적 경로의 폭이 저장되는 필드의 길이보다 더 클 수 있을지라도, 단지 그 지정된 필드에 포함된 그 바이트 위치들의 내용들만 대체된다.
- [0484] 정보의 일정 유닛들(units)은 스토리지에서 인테그럴 경계(integral boundary) 상에 있어야 한다. 경계(boundary)는 그 스토리지 주소가 그 유닛의 길이의 바이트 배수일 때 정보의 유닛에 대해서 인테그럴(integral)하다고 불린다. 인테그럴 경계 상의 2, 4, 8, 16 및 32 바이트의 필드들에는 특별한 명칭들이 주어진다. 하프워드(halfword)는 2-바이트 경계 상의 2개의 연속 바이트들의 그룹이고 명령들의 기본 빌딩 블록이다. 워드(word)는 4-바이트 경계 상의 4개의 연속 바이트들의 그룹이다. 더블워드(doubleword)는 8-바이트 경계 상

의 8개의 연속 바이트들의 그룹이다. 쿼드워드(quadword)는 16-바이트 경계 상의 16개의 연속 바이트들의 그룹이다. 옥토워드(octoword)는 32-바이트 경계 상의 32개의 연속 바이트들의 그룹이다. 스토리지 주소들이 하프워드, 워드, 더블워드, 쿼드워드, 및 옥토워드를 지정할 때, 그 주소의 2진 표시는 1개, 2개, 3개, 4개, 또는 5개의 제일 우측 제로(zero)비트들을 각각 포함한다. 명령들은 2-바이트 인테그럴 경계들 상에 있어야 한다. 대부분의 명령들의 스토리지 오퍼랜드들은 경계-정렬(boundary-alignment) 요건들을 갖지 않는다.

[0485] 명령들과 데이터 오퍼랜드들에 대한 별개의 캐시들을 구현하는 디바이스들상에서, 만일 프로그램이 어떤 캐시 라인에 저장되고 그 캐시 라인으로부터 명령들이 후속적으로 폐치되면, 그 저장이 후속적으로 폐치되는 명령들을 변경하는지 여부와 상관 없이, 상당한 지연을 겪게 될 것이다.

[0486] 한 예에서, 실시 예들은 소프트웨어로 실시될 수 있다(이 소프트웨어는 때때로 라이선스된 내부 코드, 펌웨어, 마이크로-코드, 밀리-코드, 피코-코드 등으로 불리며, 이들 중 어떤 것이든 하나 또는 그 이상의 실시 예들에 부합할 것이다). 도 17을 참조하면, 하나 또는 그 이상의 특징들을 구현하는 소프트웨어 프로그램 코드는 CD-ROM 드라이브, 테이프 드라이브 또는 하드 드라이브와 같은 장기 스토리지(long-term storage) 매체 디바이스들(5011)로부터 호스트 시스템(5000)의 프로세서(5001)에 의해 액세스된다. 소프트웨어 프로그램 코드는 디스켓, 하드 드라이브, 또는 CD-ROM과 같은 데이터 처리 시스템에 사용할 용도로 알려진 여러 가지 매체들 중 어느 하나에 구현될 수 있다. 코드는 그러한 매체상에 배포되거나, 또는 한 컴퓨터 시스템의 컴퓨터 메모리(5002) 또는 스토리지의 사용자들로부터 네트워크(5010)를 통해서 다른 컴퓨터 시스템들에, 그러한 다른 시스템들의 사용자에 의해 사용될 용도로 배포될 수 있다.

[0487] 소프트웨어 프로그램 코드는 여러 가지 컴퓨터 컴포넌트들의 기능과 상호작용(interaction) 및 하나 또는 그 이상의 애플리케이션 프로그램들을 제어하는 운영체제를 포함한다. 프로그램 코드는 보통으로 스토리지 매체 디바이스(5011)로부터 상대적으로 더 고속의 컴퓨터 스토리지(5002)—이것은 프로세서(5001)에 의한 처리에 이용 가능함—로 페이지된다. 메모리 내 소프트웨어 프로그램 코드를 물리적 매체상에 구현하는 기술과 방법, 및/또는 네트워크들을 통해서 소프트웨어 코드를 배포하는 기술과 방법은 잘 알려져 있으며 여기에서는 더 논의하지 않을 것이다. 프로그램 코드는, 유형의 매체(전자 메모리 모듈들(RAM), 플래시 메모리, 콤팩트 디스크(CDs), DVDs, 자기 테이프 등을 포함하나, 이러한 것들로 한정되지 않음)상에 생성되고 저장될 때, 흔히 "컴퓨터 프로그램 제품"으로 불린다. 컴퓨터 프로그램 제품 매체는 통상적으로 처리 회로에 의해 판독 가능하며, 컴퓨터 시스템에서 처리 회로에 의해 실행하기 위해 판독 가능한 것이 바람직하다.

[0488] 도 18은 하나 또는 그 이상의 실시 예들이 실시될 수 있는 대표적인 워크스테이션 또는 서버 하드웨어 시스템을 예시한다. 도 18의 시스템(5020)은 선택적인 주변 디바이스들을 포함하여, 개인용 컴퓨터, 워크스테이션 또는 서버 같은 대표적인 베이스 컴퓨터 시스템(5021)을 포함한다. 베이스 컴퓨터 시스템(5021)은 하나 또는 그 이상의 프로세서들(5026)과 버스를 포함하며, 버스는 알려진 기술들에 따라 프로세서(들)(5026)와 시스템(5021)의 다른 컴포넌트들 사이를 연결하여 통신을 가능하게 하기 위해 채용되는 것이다. 버스는 프로세서(5026)를 메모리(5025)와 장기 스토리지(5027)에 연결하며 장기 스토리지는, 예를 들어, 하드 드라이브(예를 들어, 자기 매체, CD, DVD 및 플래시 메모리를 포함함) 또는 테이프 드라이브를 포함할 수 있다. 시스템(5021)은 또한 사용자 인터페이스 어댑터를 포함할 수 있으며, 이 사용자 인터페이스 어댑터는 마이크로프로세서(5026)를 버스를 통해서 키보드(5024), 마우스(5023), 프린터/스캐너(5030) 및/또는 기타 인터페이스 디바이스들과 같은 하나 또는 그 이상의 인터페이스 디바이스들에 연결하며, 상기 기타 인터페이스 디바이스들은 터치 감응식 스크린(touch sensitive screen), 디지털 입력 패드(digitized entry pad) 등과 같은 사용자 인터페이스 디바이스일 수 있다. 버스는 또한 LCD 스크린 또는 모니터와 같은 디스플레이 디바이스(5022)를 디스플레이 어댑터를 통해서 마이크로프로세서(5026)에 연결한다.

[0489] 시스템(5021)은 네트워크(5029)와 통신(5028)이 가능한 네트워크 어댑터를 경유하여 다른 컴퓨터들 또는 컴퓨터들의 네트워크들과 통신할 수 있다. 네트워크 어댑터들의 예로는 통신 채널(communications channels), 토큰 링(token ring), 이더넷(Ethernet) 또는 모뎀(modems)이 있다. 이와는 달리, 시스템(5021)은 CDPD(cellular digital packet data) 카드 같은 무선 인터페이스를 사용하여 통신할 수 있다. 시스템(5021)은 근거리 통신망(LAN) 또는 광역 통신망(WAN)에서 다른 컴퓨터들과 연관될 수 있고, 또는 시스템(5021)은 또 다른 컴퓨터와 클라이언트/서버 배열방식(arrangement)에서 클라이언트가 될 수 있다. 이들 모든 구성들과 적절한 통신 하드웨어 및 소프트웨어는 이 기술분야에서 알려져 있다.

[0490] 도 19는 하나 또는 그 이상의 실시 예들이 실시될 수 있는 데이터 처리 네트워크(5040)를 예시한다. 데이터 처리 네트워크(5040)는 무선 네트워크와 유선 네트워크 같은 복수의 개별 네트워크들을 포함할 수 있으며, 이들의

각각은 복수의 개별 워크스테이션들(5041, 5042, 5043, 5044)을 포함할 수 있다. 또한, 이 기술분야에서 통상의 지식을 가진 자들은 인식할 수 있는 바와 같이, 하나 또는 그 이상의 LAN들이 포함될 수 있으며, 여기에서 LAN은 호스트 프로세서에 결합된 복수의 지능형(intelligent) 워크스테이션들을 포함할 수 있다.

[0491] 계속해서 도 19를 참조하면, 네트워크들은 또한 게이트웨이 컴퓨터 (클라이언트 서버 5046) 또는 애플리케이션 서버(데이터 저장소를 액세스할 수 있고 또한 워크스테이션 5045로부터 직접 액세스될 수 있는 원격 서버 5048)와 같은 메인프레임 컴퓨터들 또는 서버들을 포함할 수 있다. 게이트웨이 컴퓨터(5046)는 각 개별 네트워크로의 진입점(a point of entry) 역할을 한다. 게이트웨이는 하나의 네트워킹 프로토콜을 또 하나의 네트워킹 프로토콜에 연결할 때 필요하다. 게이트웨이(5046)는 바람직하게는 통신 링크를 통해 또 하나의 네트워크(예를 들면 인터넷 5047)에 결합될 수 있다. 게이트웨이(5046)는 또한 통신 링크를 사용하여 하나 또는 그 이상의 워크스테이션들(5041, 5042, 5043, 5044)에 직접 결합될 수 있다. 게이트웨이 컴퓨터는 인터넷 내셔널 비즈니스 머신즈 코퍼레이션에서 입수 가능한 IBM eServer System z 서버를 활용하여 구현될 수 있다.

[0492] 도 18과 도 19를 동시에 참조하면, 하나 또는 그 이상의 특징들을 구현할 수 있는 소프트웨어 프로그래밍 코드(5031)가 시스템(5020)의 프로세서(5026)에 의해 CD-ROM 드라이브 또는 하드 드라이브와 같은 장기 스토리지 매체(5027)로부터 액세스될 수 있다. 소프트웨어 프로그래밍 코드는 디스켓, 하드 드라이브, 또는 CD-ROM과 같은 데이터 처리 시스템과 함께 사용할 용도로 알려진 여러 가지 매체들 중 어느 하나에 구현될 수 있다. 코드는 그러한 매체상에 배포되거나, 또는 한 컴퓨터 시스템의 메모리 또는 스토리지의 사용자들(5050, 5051)로부터 네트워크를 통해서 다른 컴퓨터 시스템들에, 그러한 다른 시스템들의 사용자에게 의해 사용될 용도로 배포될 수 있다.

[0493] 이와는 달리, 프로그래밍 코드는 메모리(5025)에 구현되고, 프로세서 버스를 사용하여 프로세서(5026)에 의해 액세스될 수 있다. 이러한 프로그래밍 코드는 여러 가지 컴퓨터 컴포넌트들의 기능과 상호작용 및 하나 또는 그 이상의 애플리케이션 프로그램들(5032)을 제어하는 운영체제를 포함한다. 프로그램 코드는 보통으로 스토리지 매체(5027)로부터 고속의 메모리(5025) - 이것은 프로세서(5026)에 의한 처리에 이용 가능함 - 로 페이지된다. 메모리 내 소프트웨어 프로그래밍 코드를 물리적 매체상에 구현하는 기술과 방법, 및/또는 네트워크들을 통해서 소프트웨어 코드를 배포하는 기술과 방법은 잘 알려져 있으며 여기에서는 더 논의하지 않을 것이다. 프로그램 코드는, 유형의 매체(전자 메모리 모듈들(RAM), 플래시 메모리, 콤팩트 디스크(CDs), DVDs, 자기 테이프 등을 포함하나, 이러한 것들로 한정되지 않음)상에 생성되고 저장될 때, 흔히 "컴퓨터 프로그램 제품"으로 불린다. 컴퓨터 프로그램 제품 매체는 통상적으로 처리 회로에 의해 판독 가능하며, 컴퓨터 시스템에서 처리 회로에 의해 실행하기 위해 판독 가능한 것이 바람직하다.

[0494] 프로세서가 가장 쉽게 이용 가능한 캐시(보통으로 프로세서의 다른 캐시들보다 더 빠르고 더 작음)는 가장 낮은(L1 또는 레벨 1) 캐시이고 메인 저장소(메인 메모리)는 가장 높은 레벨의 캐시(만일 3개의 레벨이 있다면 L3)이다. 가장 낮은 레벨의 캐시는 흔히 실행될 기계어 명령들을 보유하는 명령 캐시(I-캐시)와 데이터 오퍼랜드들을 보유하는 데이터 캐시(D-캐시)로 나뉜다.

[0495] 도 20을 참조하면, 예시적인 프로세서 실시 예가 프로세서(5026)에 대해 도시된다. 프로세서 성능을 향상시키기 위해서 메모리 블록들을 버퍼하기 위해 통상적으로 하나 또는 그 이상의 캐시(5053) 레벨들이 채용된다. 캐시(5053)는 사용될 가능성이 있는 메모리 데이터의 캐시 라인들을 보유하는 고속 버퍼이다. 통상적인 캐시 라인들은 64, 128 또는 256 바이트의 메모리 데이터이다. 별개의 캐시들은 흔히 데이터를 캐시하기 위해서보다는 명령들을 캐시하기 위해 채용된다. 이 기술분야에서 잘 알려진 "스누프(snoop)" 알고리즘들에 의해 캐시 일관성(cache coherence)(메모리 내 라인들의 사본들과 캐시들의 동기화(synchronization))이 종종 제공된다. 프로세서 시스템의 메인 메모리 스토리지(5025)는 종종 캐시로 불린다. 4개 레벨의 캐시(5053)를 가진 프로세서 시스템에서, 메인 스토리지(5025)는 때로 레벨 5(L5) 캐시로 불리는데, 왜냐하면 그것은 통상적으로 더 빠르며 컴퓨터 시스템이 이용 가능한 비휘발성 스토리지(DASD, 테이프 등)의 일부분만을 보유하기 때문이다. 메인 스토리지(5025)는 운영체제에 의해 메인 스토리지(5025)의 안팎으로(in and out of) 페이지되는 데이터의 페이지들을 "캐시"한다.

[0496] 프로그램 카운터(명령 카운터)(5061)는 실행될 현재 명령의 주소를 추적한다. z/Architecture 프로세서 내 프로그램 카운터는 64비트이고 이전의 주소지정 한계(addressing limits)를 지원하기 위해 31비트 또는 24비트로 잘려질 수 있다. 프로그램 카운터는 통상적으로 컴퓨터의 PSW(프로그램 상태 워드)에 구현되어, 그것이 컨텍스트 전환(context switching) 동안 지속되도록 한다. 그리하여, 프로그램 카운터 값을 갖는 진행중인 프로그램은, 예를 들어, 운영체제에 의해 인터럽트될 수 있다(프로그램 환경에서 운영체제 환경으로 컨텍스트 전환). 프로그램이 활성이 아닐 때, 프로그램의 PSW는 프로그램 카운터 값을 유지하고, 운영체제가 실행 중일 때 운영체제의

(PSW 내) 프로그램 카운터가 사용된다. 통상적으로, 프로그램 카운터는 현재 명령의 바이트 수와 동일한 양으로 증분된다. 감소된 명령 세트 컴퓨팅(Reduced Instruction Set Computing, RISC) 명령들은 통상적으로 고정 길이이고, 한편 콤플렉스 명령 세트 컴퓨팅(Complex Instruction Set Computing, CISC) 명령들은 통상적으로 가변 길이이다. IBM z/Architecture의 명령들은 2, 4 또는 6 바이트의 길이를 갖는 CISC 명령들이다. 프로그램 카운터(5061)는, 예를 들어, 분기 명령의 분기 채택 연산(branch taken operation) 또는 컨텍스트 전환 연산에 의해 변경된다. 컨텍스트 전환 연산에서, 현재의 프로그램 카운터 값은 실행되고 있는 프로그램에 관한 상태 정보(예를 들어, 조건 코드들과 같은 것)와 함께 프로그램 상태 워드에 세이브되고(saved), 실행될 새로운 프로그램 모듈의 명령을 가리키는 새로운 프로그램 카운터 값이 로드된다. 프로그램 카운터(5061) 내에 분기 명령의 결과를 로딩함으로써 프로그램이 결정을 내리거나 그 프로그램 내에서 루프를 돌도록 허용하기 위해, 분기 채택 연산(branch taken operation)이 수행된다.

[0497] 통상적으로 프로세서(5026)를 대신하여 명령들을 폐치하기 위해 명령 폐치 유닛(5055)이 채용된다. 폐치 유닛은 "다음 순차의 명령들"이나, 분기 채택 명령들의 타겟 명령들, 또는 컨텍스트 전환에 뒤이은 프로그램의 첫 번째 명령들을 폐치한다. 현대 명령(Modern Instruction) 폐치 유닛은 프리페치된(prefetched) 명령들이 사용될 수 있는 가능성에 기초하여 추론적으로 명령들을 프리페치하는 프리페치 기술들을 종종 채용한다. 예를 들어, 폐치 유닛은 16 바이트의 명령—이는 그 다음 순차 명령 및 그 이후 순차 명령들의 추가 바이트들을 포함함—을 폐치할 수 있다.

[0498] 그런 다음, 폐치된 명령들이 프로세서(5026)에 의해 실행된다. 한 실시 예에서, 폐치된 명령(들)은 폐치 유닛의 디스패치 유닛(5056)으로 보내진다. 디스패치 유닛이 그 명령(들)을 디코드하고, 디코드된 명령(들)에 관한 정보를 적절한 유닛들(5057, 5058, 5060)로 전달한다. 실행 유닛(5057)이 통상적으로 명령 폐치 유닛(5055)으로부터 디코드된 산술 명령들(arithmetic instructions)에 관한 정보를 수신할 것이고, 그 명령의 오퍼코드(opcode)에 따라 오퍼랜드들에 대한 산술 연산들(arithmetic operations)을 수행할 것이다. 오퍼랜드들이 바람직하게는, 메모리(5025), 아키텍처화된 레지스터들(5059)로부터 또는 실행되고 있는 명령의 즉시 필드(immediate field)로부터 실행 유닛(5057)에 제공된다. 저장될 때, 실행의 결과들이 메모리(5025)나, 레지스터들(5059)에 또는 다른 머신 하드웨어(예를 들어, 제어 레지스터들, PSW 레지스터들 및 그와 유사한 것)에 저장된다.

[0499] 가상 주소들은 동적 주소 변환(5062)을 이용하여, 선택에 따라서는 액세스 레지스터 변환(5063)을 이용하여 실제 주소들로 변환된다.

[0500] 통상적으로 프로세서(5026)는 명령의 기능을 실행하기 위한 하나 또는 그 이상의 유닛들(5057, 5058, 5060)을 갖는다. 도 21a를 참조하면, 실행 유닛(5057)은 인터페이스 로직(5071)을 거쳐서 아키텍처화된 범용 레지스터들(5059), 디코드/디스패치 유닛(5056), 로드 저장 유닛(5060), 및 기타(5065) 프로세서 유닛들과 통신할 수 있다(5071). 실행 유닛(5057)은, 산술 논리 유닛(arithmetic logic unit, ALU)(5066)이 연산할 정보를 보유하기 위해 몇몇의 레지스터 회로들(5067, 5068, 5069)을 채용할 수 있다. ALU는 논리곱(AND), 논리합(OR) 및 배타논리합(XOR), 로테이트(rotate) 및 시프트(shift)와 같은 논리 함수뿐만 아니라 더하기, 빼기, 곱하기 및 나누기와 같은 산술 연산들도 수행한다. 바람직하게는, ALU는 설계에 종속적인 특수 연산들을 지원한다. 다른 회로들은, 예를 들어, 조건 코드들 및 복구 지원 로직을 포함하는 다른 아키텍처화된 퍼실리티들(5072)을 제공할 수 있다. 통상적으로, ALU 동작의 결과는 출력 레지스터 회로(5070)에 보유(hold)되고, 이 출력 레지스터 회로(5070)는 여러 가지 다른 처리 기능들에 그 결과를 전달할 수 있다. 프로세서 유닛들의 배열방식(arrangements)은 다양하며, 본 설명은 본 발명의 한 실시 예에 관한 대표적인 이해를 제공하려는 의도일 뿐이다.

[0501] 예를 들어, ADD 명령은 산술 및 논리 기능을 갖는 실행 유닛(5057)에서 실행될 것이고, 한편 예를 들어 부동 소수점 명령은 특수한 부동 소수점 능력을 갖는 부동 소수점 실행에서 실행될 것이다. 바람직하게는, 실행 유닛은 오퍼랜드들에 관한 오퍼코드 정의 기능(opcode defined function)을 수행함으로써 명령에 의해 식별된 오퍼랜드들에 관해 연산한다. 예를 들어, ADD 명령은 그 명령의 레지스터 필드들에 의해 식별되는 두 개의 레지스터들(5059)에서 발견되는 오퍼랜드들에 관해 실행 유닛(5057)에 의해 실행될 수 있다.

[0502] 실행 유닛(5057)은 두 개의 오퍼랜드들에 관해 산술 덧셈(arithmetic addition)을 수행하고 그 결과를 제3 오퍼랜드에 저장하며, 여기서, 제3 오퍼랜드는 제3 레지스터 또는 두 개의 소스 레지스터들 중 하나일 수 있다. 바람직하게는, 실행 유닛은 산술 논리 유닛(ALU)(5066)을 이용하며 이 ALU(5066)는 더하기, 빼기, 곱하기, 나누기 중 어느 것이든지 포함하는 여러 가지 대수 함수들(algebraic functions) 뿐만 아니라 시프트(Shift), 로테이트(Rotate), 논리곱(And), 논리합(Or) 및 배타논리합(XOR)과 같은 여러 가지 논리 함수들을 수행할 수 있다. 일부 ALU들(5066)은 스칼라 연산들을 위해 설계되며 일부는 부동 소수점을 위해 설계된다. 데이터는 아키텍처에

따라 빅 엔디언(Big Endian)(여기서 최하위 바이트(least significant byte)는 가장 높은 바이트 주소에 있음) 또는 리틀 엔디언(Little Endian)(여기서 최하위 바이트는 가장 낮은 바이트 주소에 있음)일 수 있다. IBM z/Architecture는 빅 엔디언이다. 부호화된 필드들(signed fields)은 아키텍처에 따라, 부호(sign) 및 크기(magnitude), 1의 보수 또는 2의 보수일 수 있다. 2의 보수에서 음의 값 또는 양의 값은 단지 ALU 내에서 덧셈만을 필요로 하므로, ALU가 뺄셈 능력을 설계할 필요가 없다는 점에서 2의 보수가 유리하다. 숫자들은 일반적으로 속기(shorthand)로 기술되는데, 12비트 필드는 예를 들어, 4,096바이트 블록의 주소를 정의하고 일반적으로 4 Kbyte(Kilo-byte) 블록으로 기술된다.

[0503] 도 21b를 참조하면, 분기 명령을 실행하기 위한 분기 명령 정보는 통상적으로 분기 유닛(5058)으로 보내지는데, 이 분기 유닛(5058)은 다른 조건부 연산들(conditional operations)이 완료되기 전에 그 분기의 결과를 예측하도록 분기 이력 테이블(5082)과 같은 분기 예측 알고리즘을 흔히 채용한다. 현재 분기 명령의 타겟은, 그 조건부 연산들이 완료되기 전에 폐지되고 추론적으로 실행될 것이다. 조건부 연산들이 완료될 때, 추론적으로 실행된 분기 명령들은 조건부 연산 및 추론된 결과의 조건들에 기초하여 완료되거나 폐기된다. 통상적인 분기 명령은, 만일 그 조건 코드들이 분기 명령의 분기 요건을 충족한다면, 조건 코드들을 테스트하고 타겟 주소로 분기할 수 있고, 타겟 주소는, 예를 들어, 레지스터 필드들 또는 그 명령의 즉시 필드에서 발견되는 수들을 포함하는 몇 개의 수들에 기초하여 계산될 수 있다. 분기 유닛(5058)은 복수의 입력 레지스터 회로들(5075, 5075, 5077) 및 출력 레지스터 회로(5080)를 갖는 ALU(5074)를 채용할 수 있다. 분기 유닛(5058)은, 예를 들어, 범용 레지스터들(5059), 디코드 디스패치 유닛(5056) 또는 기타 회로들(5073)과 통신할 수 있다(5081).

[0504] 명령들의 그룹의 실행은 여러 가지 이유들로 인터럽트될 수 있는데, 이러한 이유들에는, 예를 들어, 운영체제에 의해 개시되는 컨텍스트 전환, 컨텍스트 전환을 초래하는 프로그램 예외 또는 에러, 컨텍스트 전환 또는 (멀티-스레드 환경에서) 복수의 프로그램들의 멀티-스레딩 활동을 초래하는 I/O 인터럽션 신호가 포함된다. 바람직하게는 컨텍스트 전환 액션은 현재 실행중인 프로그램에 관한 상태 정보(state information)를 세이브하고, 그런 다음 호출되는 또 다른 프로그램에 관한 상태 정보를 로드한다. 상태 정보는, 예를 들어, 하드웨어 레지스터들 또는 메모리에 저장될 수 있다. 바람직하게는, 상태 정보는 실행될 다음 명령을 가리키는 프로그램 카운터 값, 조건 코드들, 메모리 변환 정보 및 아키텍처화된 레지스터 콘텐츠를 포함한다. 컨텍스트 전환 활동은, 하드웨어 회로들, 애플리케이션 프로그램들, 운영체제 프로그램들 또는 펌웨어 코드(마이크로코드, 펌웨어 코드 또는 라이선스된 내부 코드(LIC)) 단독으로 또는 이것들의 조합으로 실행될 수 있다.

[0505] 프로세서는 명령 정의 방법들(instruction defined methods)에 따라 오퍼랜드들에 액세스한다. 명령은 명령의 일부분의 값을 사용하는 즉시 오퍼랜드(immediate operand)를 제공할 수 있고, 범용 레지스터들 또는 특수 목적용 레지스터들(예를 들어, 부동 소수점 레지스터들)을 분명하게 가리키는 하나 또는 그 이상의 레지스터 필드들을 제공할 수 있다. 명령은 오퍼코드 필드에 의해 오퍼랜드들로서 식별되는 암시 레지스터들(implied registers)을 이용할 수 있다. 명령은 오퍼랜드들에 대한 메모리 위치들을 이용할 수 있다. 오퍼랜드의 메모리 위치는 레지스터, 즉시 필드(immediate field), 또는 레지스터들과 즉시 필드의 조합에 의해 제공될 수 있고, 이는 z/Architecture 장 변위(long displacement) 퍼실리티가 전형적인 예이며, 여기서 명령은 기준 레지스터, 인덱스 레지스터 및 즉시 필드(변위 필드)—이것들은 예를 들어 메모리에서 오퍼랜드의 주소를 제공하기 위해 함께 더해짐—를 정의한다. 만일 다르게 표시되지 않는다면, 여기서의 위치는 통상적으로 메인 메모리(메인 스토리지) 내 위치를 암시한다.

[0506] 도 21c를 참조하면, 프로세서는 로드/저장 유닛(5060)을 사용하여 스토리지에 액세스한다. 로드/저장 유닛(5060)은 메모리(5053)에서 타겟 오퍼랜드의 주소를 획득하고 레지스터(5059) 또는 또 다른 메모리(5053) 위치에 오퍼랜드를 로딩함으로써 로드 연산을 수행할 수 있고, 또는 메모리(5053)에서 타겟 오퍼랜드의 주소를 획득하고 레지스터(5059) 또는 또 다른 메모리(5053) 위치로부터 획득된 데이터를 메모리(5053) 내 타겟 오퍼랜드 위치에 저장함으로써 저장 연산을 수행할 수 있다. 로드/저장 유닛(5060)은 추론적(speculative)일 수 있고, 명령 순서에 비해 순서가 다른(out-of-order) 순서로 메모리에 액세스할 수 있지만, 로드/저장 유닛(5060)은 명령들이 순서대로 실행된 것으로 프로그램들에 대한 외관(appearance)을 유지할 것이다. 로드/저장 유닛(5060)은 범용 레지스터들(5059), 디코드/디스패치 유닛(5056), 캐시/메모리 인터페이스(5053) 또는 기타 엘리먼트들(5083)과 통신(5084)할 수 있고, 스토리지 주소들을 계산하기 위해 그리고 순서대로 연산들을 유지하기 위한 파이프라인 시퀀싱을 제공하기 위해 여러 가지 레지스터 회로들(5086, 5087, 5088, 5089), ALU들(5085) 및 제어 논리(5090)를 포함한다. 일부 연산들은 순서가 바뀔 수 있으나, 이 기술분야에서 잘 알려진 바와 같이, 로드/저장 유닛은, 순서가 바뀐 연산들이 그 프로그램에 순서대로 수행된 것처럼 나타나도록 하는 기능을 제공한다.

[0507] 바람직하게는, 애플리케이션 프로그램이 "보는(sees)" 주소들은 흔히 가상 주소들로 불린다. 가상 주소들은 때

로는 "논리적 주소들(logical addresses)" 및 "유효 주소들(effective addresses)"로 불린다. 이들 가상 주소들은 여러 가지 동적 주소 변환(DAT) 기술들 중 하나에 의해 물리적 메모리 위치로 다시 보내진다는 점에서 가상이고, 상기 여러 가지 동적 주소 변환(DAT) 기술들에는, 단순히 오프셋 값으로 가상 주소를 프리픽싱(prefixing)하는 것, 하나 또는 그 이상의 변환 테이블들을 통해 가상 주소를 변환하는 것이 포함될 수 있으나, 이러한 것들로 한정되는 것은 아니며, 바람직하게는, 변환 테이블들은 적어도 세그먼트 테이블 및 페이지 테이블만을 또는 이것들의 조합을 포함하며, 바람직하게는, 세그먼트 테이블은 페이지 테이블을 가리키는 엔트리를 갖는다. z/Architecture에서는, 변환의 계층(hierarchy of translation)이 제공되는데, 이 변환의 계층에는 영역 제1 테이블, 영역 제2 테이블, 영역 제3 테이블, 세그먼트 테이블 및 선택적인 페이지 테이블이 포함된다. 주소 변환의 수행은 흔히 변환 색인 버퍼(TLB)를 이용하여 향상되는데, 이 변환 색인 버퍼는 연관된 물리적 메모리 위치에 가상 주소를 매핑하는 엔트리들을 포함한다. DAT가 변환 테이블들을 사용하여 가상 주소를 변환할 때, 엔트리들이 생성된다. 그런 다음, 후속적으로 가상 주소를 사용할 때 느린 연속적인 변환 테이블 액세스들보다 오히려 빠른 TLB의 엔트리를 이용할 수 있다. TLB 콘텐츠는 LRU(Least Recently used)를 포함하는 여러 가지 대체 알고리즘들에 의해 관리될 수 있다.

[0508] 프로세서가 멀티-프로세서 시스템의 프로세서인 경우, 각각의 프로세서는 I/O, 캐시들, TLB들 및 메모리와 같은 공유 리소스들(shared resources)을 일관성(coherency)을 위해 인터로크(interlock)를 유지하는 역할을 한다. 통상적으로, "스누프(snoop)" 기술들이 캐시 일관성을 유지하는 데 이용될 것이다. 스누프 환경에서, 각각의 캐시 라인은 공유를 용이하게 하기 위해, 공유 상태(shared state), 독점 상태(exclusive state), 변경된 상태(changed state), 무효 상태(invalid state) 중 어느 하나에 있는 것으로 표시될 수 있다.

[0509] I/O 유닛들(5054, 도 20)은 프로세서에 주변기기에 연결하기 위한 수단을 제공하는데, 예를 들어, 그 주변기에는 테이프, 디스크, 프린터, 디스플레이, 및 네트워크가 포함된다. I/O 유닛들은 흔히 소프트웨어 드라이버들에 의해 컴퓨터 프로그램에 제공된다. IBM®의 System z 같은 메인프레임들에서, 채널 어댑터들 및 오픈 시스템 어댑터들은 운영체제와 주변 디바이스들 사이의 통신을 가능하게 하는, 메인프레임의 I/O 유닛들이다.

[0510] 또한, 다른 종류의 컴퓨팅 환경들도 하나 또는 그 이상의 특징들로부터 이득을 얻을 수 있다. 한 예로, 환경(environment)은 에뮬레이터(예, 소프트웨어 또는 다른 에뮬레이션 메커니즘들)를 포함할 수 있으며, 이 에뮬레이터에서 특정 아키텍처(예를 들어, 명령 실행, 주소 변환과 같은 아키텍처화된 함수들, 및 아키텍처화된 레지스터들을 포함함) 또는 그것의 서브세트(subset)가 (예를 들어, 프로세서 및 메모리를 갖는 네이티브 컴퓨터 시스템 상에서) 에뮬레이트된다. 이러한 환경에서, 비록 그 에뮬레이터를 실행하는 컴퓨터가 에뮬레이트되고 있는 능력들과는 다른 아키텍처를 가질 수 있지만, 에뮬레이터의 하나 또는 그 이상의 에뮬레이션 기능들은 하나 또는 그 이상의 실시 예들을 구현할 수 있다. 한 예로서, 에뮬레이션 모드에서, 에뮬레이트되고 있는 특정 명령 또는 연산은 디코드되고, 적절한 에뮬레이션 기능이 개별 명령 또는 연산을 구현하도록 만들어진다.

[0511] 에뮬레이션 환경에서, 호스트 컴퓨터는, 예를 들어, 명령들 및 데이터를 저장하는 메모리, 메모리로부터 명령들을 폐치하고 또한 선택적으로 그 폐치된 명령을 위한 로컬 버퍼링을 제공하는 명령 폐치 유닛, 폐치된 명령들을 수신하고 폐치된 명령들의 유형을 결정하는 명령 디코드 유닛, 및 명령들을 실행하는 명령 실행 유닛을 포함한다. 실행은 메모리로부터 레지스터 내에 데이터를 로딩하는 것; 레지스터로부터 메모리로 다시 데이터를 저장하는 것; 또는 디코드 유닛에 의해 결정된 바와 같이, 산술 또는 논리 연산의 몇몇 유형을 수행하는 것을 포함할 수 있다. 한 예에서, 각각의 유닛은 소프트웨어에서 구현된다. 예를 들어, 그 유닛들에 의해 수행되고 있는 연산들은 에뮬레이터 소프트웨어 내에서 하나 또는 그 이상의 서브루틴들로서 구현된다.

[0512] 더 구체적으로는, 메인프레임에서, 아키텍처화된 기계어 명령들(machine instructions)이 프로그래머들, 대개는 오늘날의 "C" 프로그래머들에 의해, 흔히 컴파일러 애플리케이션(compiler application)을 통해 사용되고 있다. 스토리지 매체에 저장되는 이들 명령들은 원래(natively) z/Architecture IBM® 서버에서 또는 이와는 다르게 다른 아키텍처들을 실행하는 머신들에서 실행될 수 있다. 그것들은 기존의 그리고 장래의 IBM® 메인프레임 서버들에서 그리고 IBM®의 다른 머신들(예, Power Systems 서버들 및 System x 서버들) 상에서 에뮬레이트될 수 있다. 그것들은 IBM®, Intel®, AMD 및 기타 회사에 의해 제조된 하드웨어를 사용하는 광범위한 머신들 상의 리눅스를 실행하는 머신들에서 실행될 수 있다. 또한, z/Architecture 하의 그 하드웨어 상에서의 실행 이외에, Hercules, UMX, 또는 FSI(Fundamental Software, Inc)—여기서 일반적으로 실행은 에뮬레이션 모드에 있음—에 의해 에뮬레이션을 사용하는 머신들 뿐만이 아니라 리눅스도 사용될 수 있다. 에뮬레이션 모드에서, 에뮬레이션 소프트웨어는 네이티브 프로세서에 의해 실행되어 에뮬레이트된 프로세서의 아키텍처를 에뮬레이트한다.

[0513] 네이티브 프로세서(native processor)는 통상적으로 에뮬레이트된 프로세서의 에뮬레이션을 수행하기 위해 펌웨어(firmware) 또는 네이티브 운영체제를 포함하는 에뮬레이션 소프트웨어를 실행한다. 에뮬레이션 소프트웨어는 그 에뮬레이트된 프로세서 아키텍처의 명령들을 페치 및 실행하는 역할을 한다. 에뮬레이션 소프트웨어는 명령 경계들(instruction boundaries)을 추적하기 위해 에뮬레이트된 프로그램 카운터를 유지한다. 에뮬레이션 소프트웨어는 한 번에 하나 또는 그 이상의 에뮬레이트된 기계어 명령들을 페치하여, 하나 또는 그 이상의 그 에뮬레이트된 기계어 명령들을 네이티브 프로세서에 의해 실행하기 위한 네이티브 기계어 명령들의 대응 그룹으로 변환시킬 수 있다. 이들 변환된 명령들은 캐시되어 더 빠른 변환이 수행될 수 있도록 할 수 있다. 그럼에도 불구하고, 에뮬레이션 소프트웨어는, 운영체제들 및 에뮬레이트된 프로세서를 위해 작성된 애플리케이션들이 정확하게 연산되도록 보장하기 위해, 그 에뮬레이트된 프로세서 아키텍처의 아키텍처 규칙들을 유지해야 한다. 더 나아가, 에뮬레이션 소프트웨어는 그 에뮬레이트된 프로세서 아키텍처에 의해 식별된 자원들을 제공해야 하며—이 자원들에는 제어 레지스터들, 범용 레지스터들, 부동 소수점 레지스터들, 예를 들어 세그먼트 테이블들 및 페이지 테이블들을 포함하는 동적 주소 변환 함수, 인터럽트 메커니즘들, 컨텍스트 전환 메커니즘들, TOD(Time of Day) 클럭들 및 I/O 서브시스템들에 대한 아키텍처화된 인터페이스들이 포함됨—그리하여 운영체제, 또는 에뮬레이트된 프로세서 상에서 실행되도록 지정된 애플리케이션 프로그램이, 에뮬레이션 소프트웨어를 갖는 네이티브 프로세서상에서 실행될 수 있도록 한다.

[0514] 에뮬레이트되고 있는 특정 명령이 디코드되고, 서브루틴이 개별 명령의 기능을 수행하기 위해 호출(call)된다. 에뮬레이트된 프로세서의 기능을 에뮬레이트하는 에뮬레이션 소프트웨어 기능은, 예를 들어, "C" 서브루틴 또는 드라이버, 또는 특정 하드웨어를 위해 드라이브를 제공하는 몇몇 다른 방법들로 구현되며, 이는 바람직한 실시예의 설명을 이해하고 나면 이 기술 분야에서 통상의 지식을 가진 자들이 도출해 낼 수 있을 것이다. 여러 가지 소프트웨어 및 하드웨어 에뮬레이션 특허들은 — 예를 들어, Beausoleil 외 발명의 미국 특허증(Letters Patent) 제5,551,013호 "하드웨어 에뮬레이션을 위한 멀티프로세서(Multiprocessor for Hardware Emulation)"; Scalzi 외 발명의 미국 특허증 제6,009,261호 "타겟 프로세서 상에서 호환가능하지 않은 명령들을 에뮬레이트하기 위한 저장된 타겟 루틴들의 전처리(Preprocessing of Stored Target Routines for Emulating Incompatible Instructions on a Target Processor)"; Davidian 외 발명의 미국 특허증 제5,574,873호 "게스트 명령들을 에뮬레이트하는 직접 액세스 에뮬레이션 루틴들에 대한 게스트 명령을 디코드하는 것(Decoding Guest Instruction to Directly Access Emulation Routines that Emulate the Guest Instructions)"; Gorishek 외 발명의 미국 특허증 제6,308,255호 "시스템에서 논-네이티브 코드를 실행할 수 있도록 하는 코프로세서 지원에 사용되는 대칭형 다중 처리 버스 및 칩셋(Symmetrical Multiprocessing Bus and Chipset Used for Coprocessor Support Allowing Non-Native Code to Run in a System)"; Lethin 외 발명의 미국 특허증 제6,463,582호 "아키텍처 에뮬레이션을 위한 동적 최적화 객체 코드 변환 및 동적 최적화 객체 코드 변환 방법(Dynamic Optimizing Object Code Translator for Architecture Emulation and Dynamic Optimizing Object Code Translation Method)"; Eric Traut 발명의 미국 특허증 제5,790,825호 "호스트 명령들의 동적 리컴파일레이션을 통해 호스트 컴퓨터 상에서 게스트 명령들을 에뮬레이트하기 위한 방법(Method for Emulating Guest Instructions on a Host Computer Through Dynamic Recompilation of Host Instructions)" 등이 포함되나, 이러한 것들로 한정되는 것은 아님 — 이 기술 분야에서 통상의 지식을 가진 자들이 이용할 수 있는 목표 머신에 대한 다른 머신을 위해 아키텍처화된 명령 포맷의 에뮬레이션을 달성하는 알려진 여러 가지 방법들을 예시하고 있다.

[0515] 도 22에서는, 호스트 아키텍처의 호스트 컴퓨터 시스템(5000')을 에뮬레이트하는 에뮬레이트된 호스트 컴퓨터 시스템(5092)의 예가 제공된다. 에뮬레이트된 호스트 컴퓨터 시스템(5092)에서, 호스트 프로세서(CPU)(5091)는 에뮬레이트된 호스트 프로세서(또는 가상 호스트 프로세서)이고 호스트 컴퓨터(5000')의 프로세서(5091)의 네이티브 명령 세트 아키텍처(native instruction set architecture)와는 다른 네이티브 명령 세트 아키텍처를 갖는 에뮬레이션 프로세서(5093)를 포함한다. 에뮬레이트된 호스트 컴퓨터 시스템(5092)은 에뮬레이션 프로세서(5093)가 액세스 가능한 메모리(5094)를 갖는다. 상기 예시 실시예에서, 메모리(5094)는 호스트 컴퓨터 메모리(5096) 부분과 에뮬레이션 루틴들(5097) 부분으로 분할된다. 호스트 컴퓨터 메모리(5096)는 호스트 컴퓨터 아키텍처에 따른 에뮬레이트된 호스트 컴퓨터(5092)의 프로그램들이 이용할 수 있다. 에뮬레이션 프로세서(5093)는 에뮬레이트된 프로세서(5091)의 명령 이외의 아키텍처의 아키텍처화된 명령 세트의 네이티브 명령들, 즉 에뮬레이션 루틴들 메모리(5097)로부터 획득된 네이티브 명령들을 실행하며, 시퀀스 & 액세스/디코드 루틴—이는 액세스되는 호스트 명령의 기능을 에뮬레이트하기 위해 네이티브 명령 실행 루틴을 결정하기 위해 액세스되는 호스트 명령(들)을 디코드할 수 있음—에서 획득된 하나 또는 그 이상의 명령(들)을 채용함으로써 호스트 컴퓨터 메모리(5096) 내 프로그램으로부터 실행하기 위한 호스트 명령을 액세스할 수 있다. 호스트 컴퓨터 시스템(5000') 아키텍처에 대하여 정의된 다른 퍼실리티들이 아키텍처화된 퍼실리티들 루틴들(architected facilities

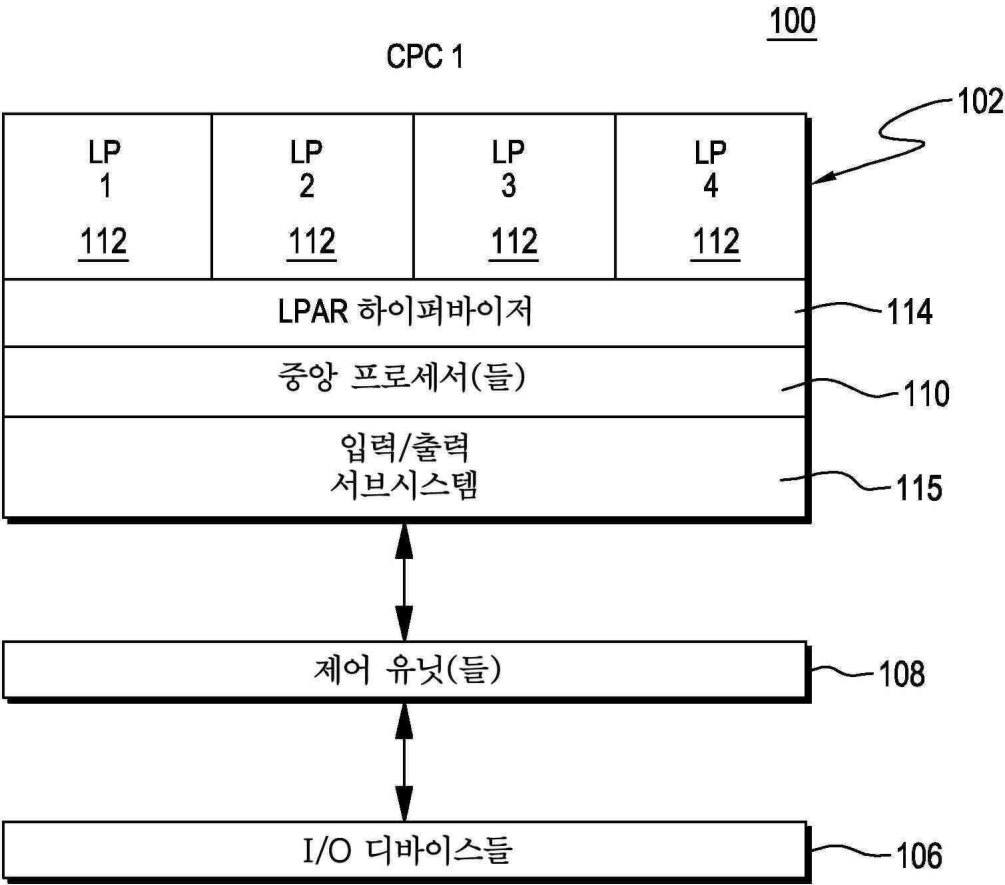
routines)에 의해 에뮬레이트될 수 있는데, 이러한 것들에는, 예를 들어, 범용 레지스터들, 제어 레지스터들(control registers), 동적 주소 변환(dynamic address translation) 및 I/O 서브시스템 지원 및 프로세서 캐시 등과 같은 퍼실리티들이 포함된다. 에뮬레이션 루틴들(emulation routines)은 또한 (범용 레지스터들 및 가상 주소들의 동적 변환 같은) 에뮬레이션 프로세서(5093)에서 이용 가능한 기능들을 이용하여 에뮬레이션 루틴들의 성능을 향상시킬 수 있다. 또한 특수 하드웨어(special hardware) 및 오프-로드 엔진들(off-load engines)이 제공되어 호스트 컴퓨터(5000')의 기능을 에뮬레이팅함에 있어서 프로세서(5093)를 보조할 수 있다.

[0516] 본 명세서 내에 사용되는 용어는 단지 특정 실시 예들을 기술할 목적으로 사용된 것이지 한정하려는 의도로 사용된 것은 아니다. 여기에서 사용할 때, 단수 형태인 "한", "일", 및 "하나" 등은 그 컨텍스트에서 그렇지 않은 것으로 명시되어 있지 않으면, 복수 형태도 또한 포함할 의도로 기술된 것이다. 또한, "포함한다" 및/또는 "포함하는"이라는 말들은 본 명세서에서 사용될 때, 언급되는 특징들, 정수들, 단계들, 동작들, 엘리먼트들, 및/또는 컴포넌트들의 존재를 명시하지만, 하나 또는 그 이상의 다른 특징들, 정수들, 단계들, 동작들, 엘리먼트들, 컴포넌트들 및/또는 이들의 그룹들의 존재 또는 추가를 배제하는 것은 아니라는 것을 이해할 수 있을 것이다.

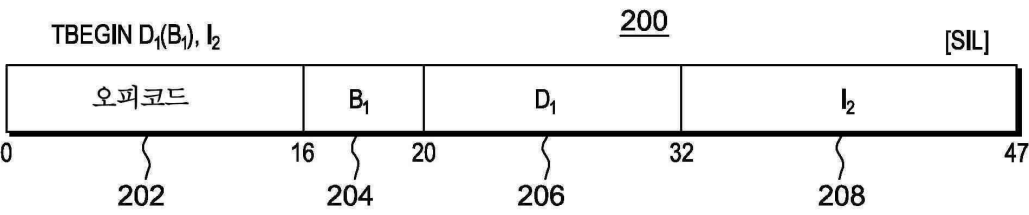
[0517] 이하의 청구항들에서, 구조들(structures), 재료들(materials), 동작들(acts), 및 모든 수단의 등가물들 또는 단계 플러스 기능 엘리먼트들은 구체적으로 청구되는 다른 청구된 엘리먼트들과 함께 그 기능을 수행하기 위한 구조, 재료, 또는 동작을 포함할 의도가 있다. 하나 또는 그 이상의 실시 예들에 대한 설명은 예시와 설명의 목적으로 제공되는 것이며, 개시되는 형태로 빠짐없이 총 망라하거나 한정하려는 의도가 있는 것은 아니다. 이 기술 분야에서 통상의 지식을 가진 자라면 많은 수정 예들 및 변형 예들이 있을 수 있다는 것을 알 수 있다. 실시 예에는 여러 특징들 및 실제 응용을 가장 잘 설명하기 위해 그리고 고려되는 구체적인 용도에 적합하게 여러 가지 수정 예들을 갖는 다양한 실시 예들을 이 기술 분야에서 통상의 지식을 가진 자들이 이해할 수 있도록 하기 위해, 선택되고 기술되었다.

도면

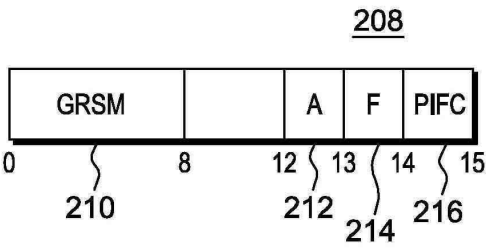
도면1



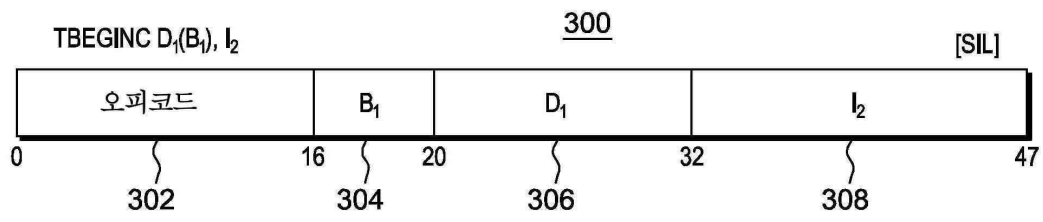
도면2a



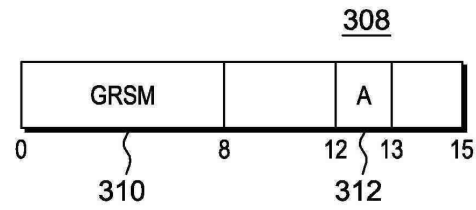
도면2b



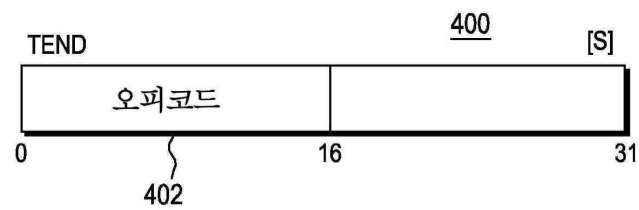
도면3a



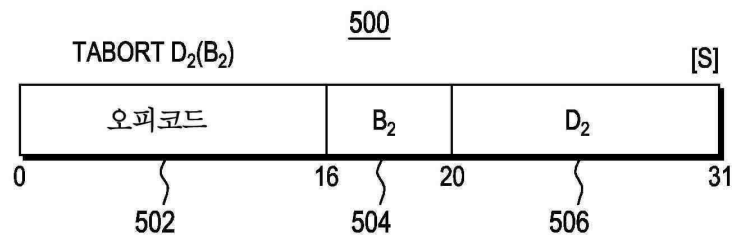
도면3b



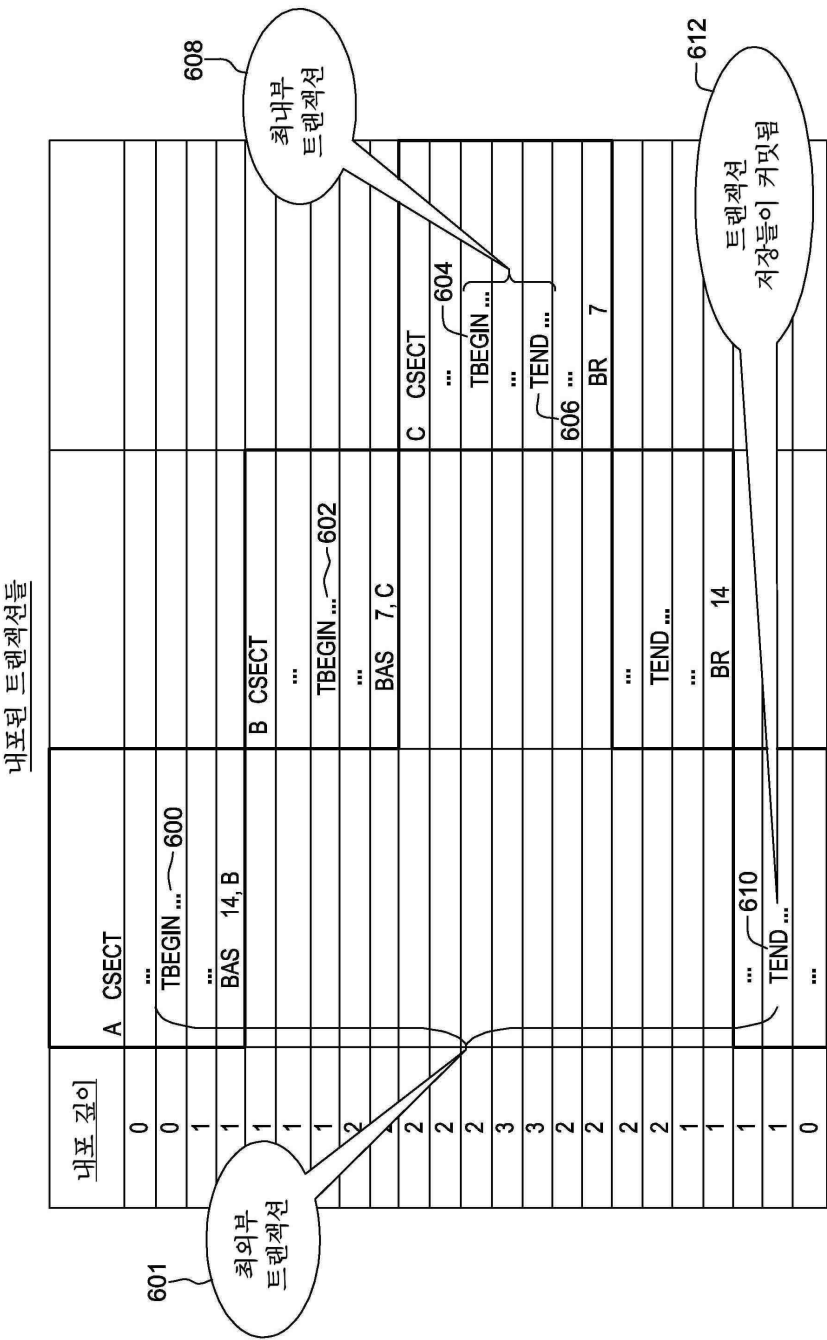
도면4



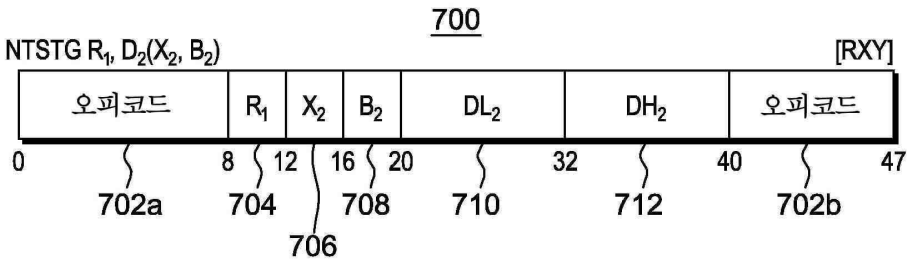
도면5



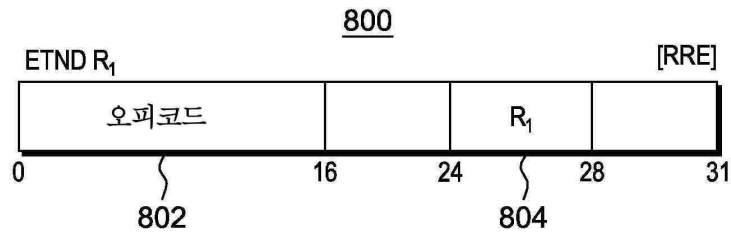
도면6



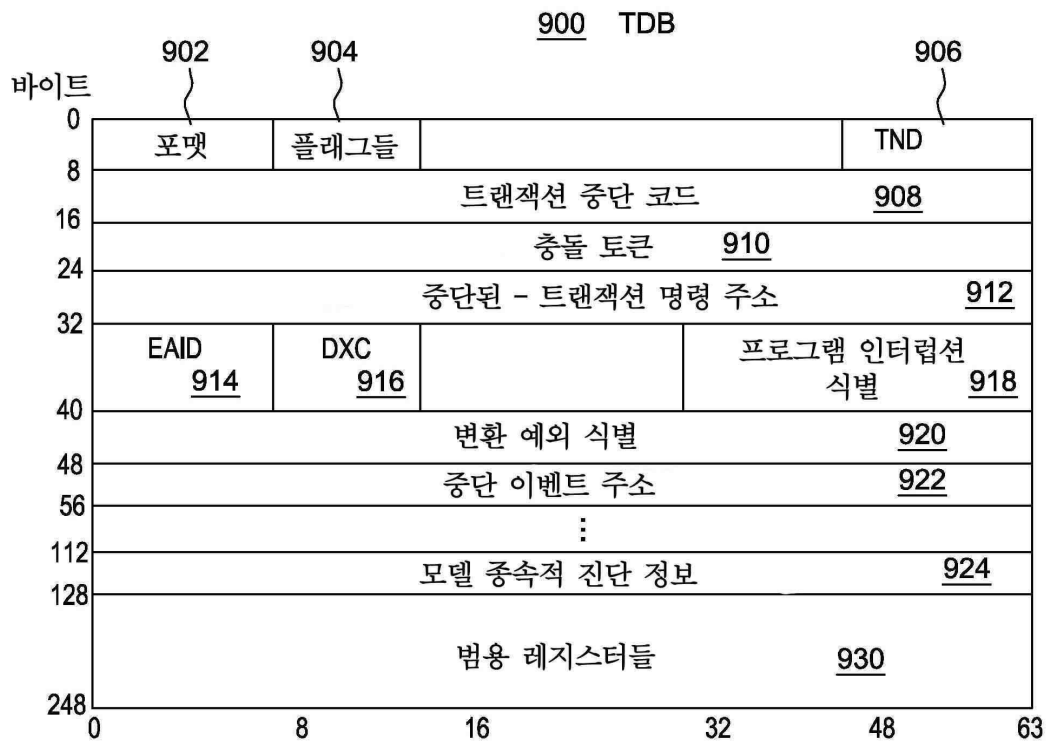
도면7



도면8



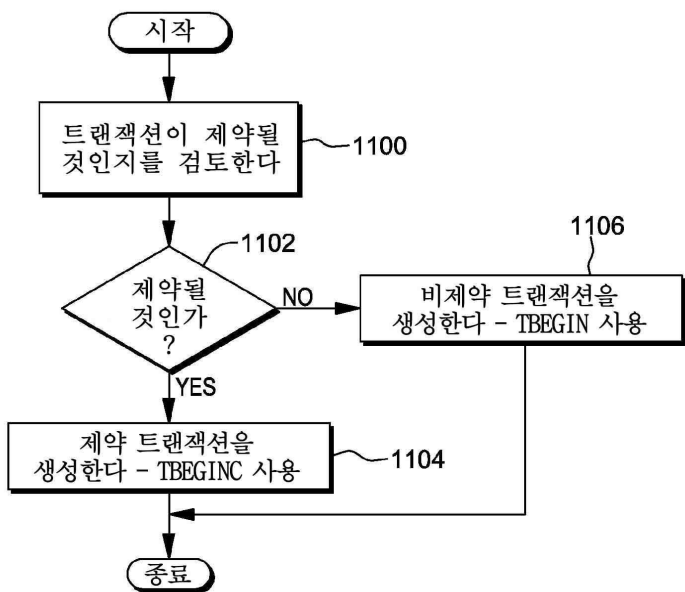
도면9



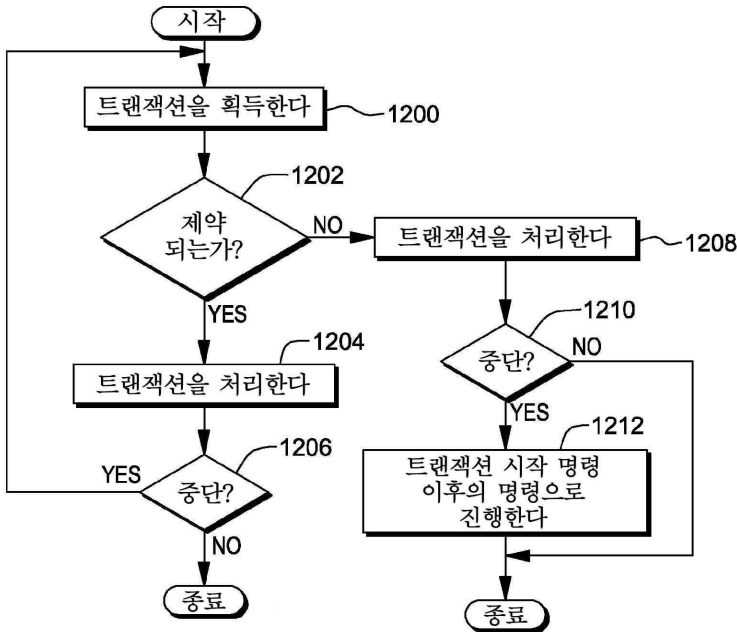
도면10

코드	중단 이유	CC세트
2	외부 인터럽션	2
4	프로그램 인터럽션(필터링 안됨)	2 OR 3 +
5	기계-검사 인터럽션	2
6	I/O 인터럽션	2
7	페치 오버플로	2 OR 3
8	저장 오버플로	2 OR 3
9	페치 충돌	2
10	저장 충돌	2
11	제한된 명령	3
12	프로그램 예외 조건(필터링 됨)	3
13	내포 깊이 초과	3
14	캐시 페치 관련	2 OR 3
15	캐시 저장 관련	2 OR 3
16	캐시 기타	2 OR 3
255	기타 조건	2 OR 3
>255	TABORT 명령	2 OR 3
‡	판정할 수 없음; TDB는 저장 안됨	1
설명: + 조건 코드는 인터럽션 코드에 기초한다 ‡ 이 상황은 트랜잭션이 중단될 때 발생하지만, TDB는 최외부 TBEGIN 명령의 성공적인 실행 이후로 액세스할 수 없게 된다. TBEGIN 명시 TDB는 저장되지 않고, 조건 코드는 1로 세트된다.		

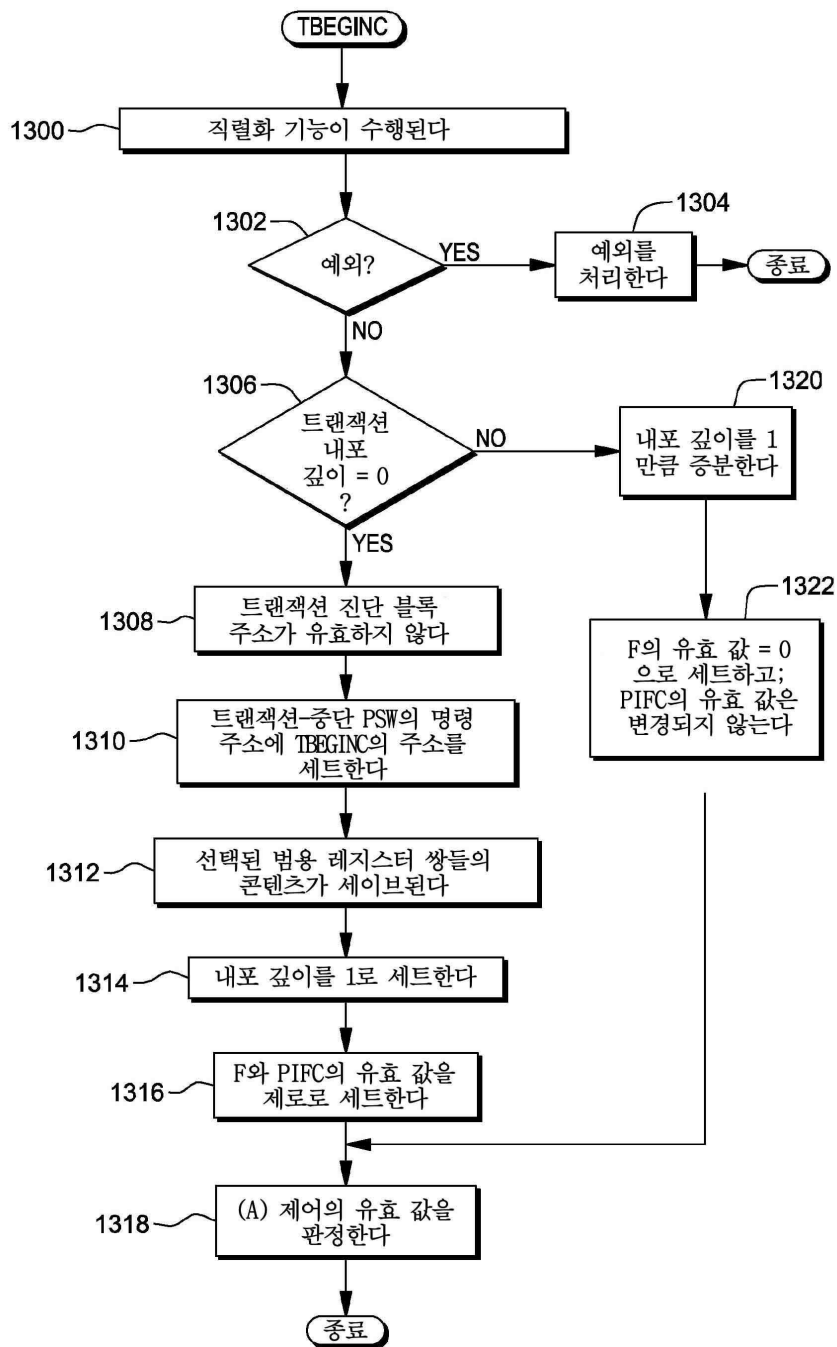
도면11



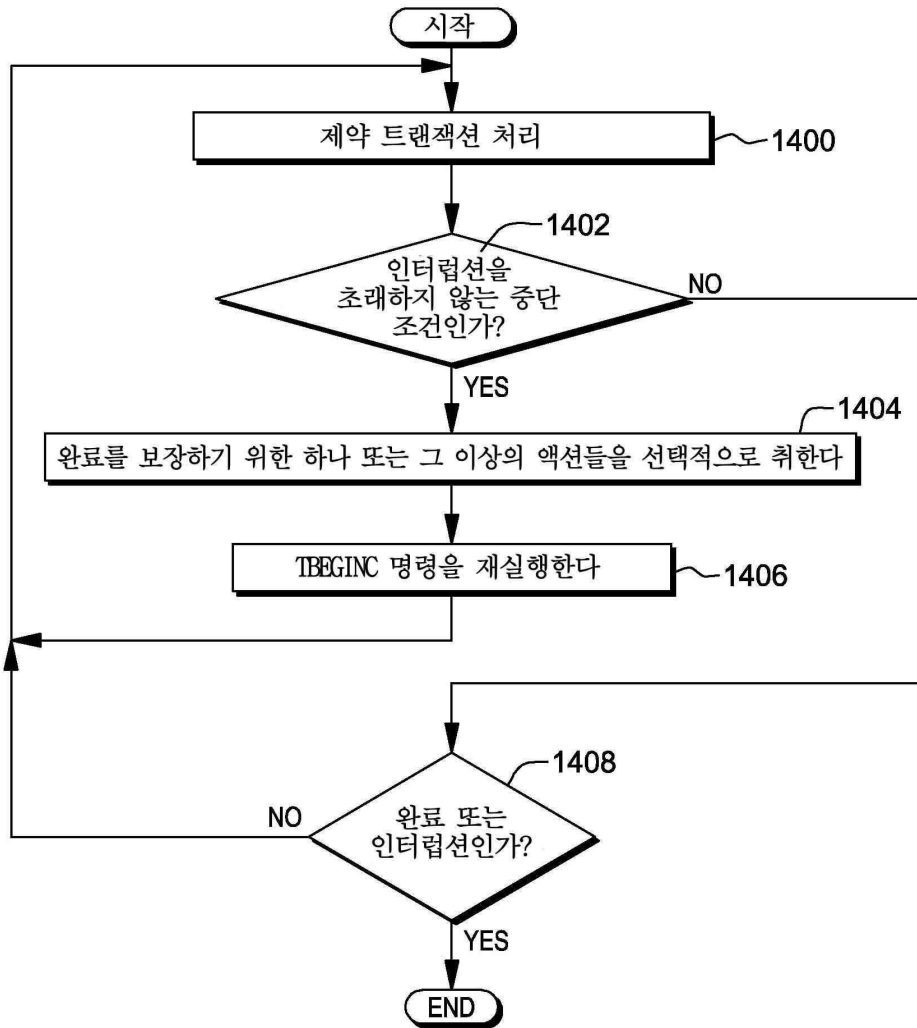
도면12



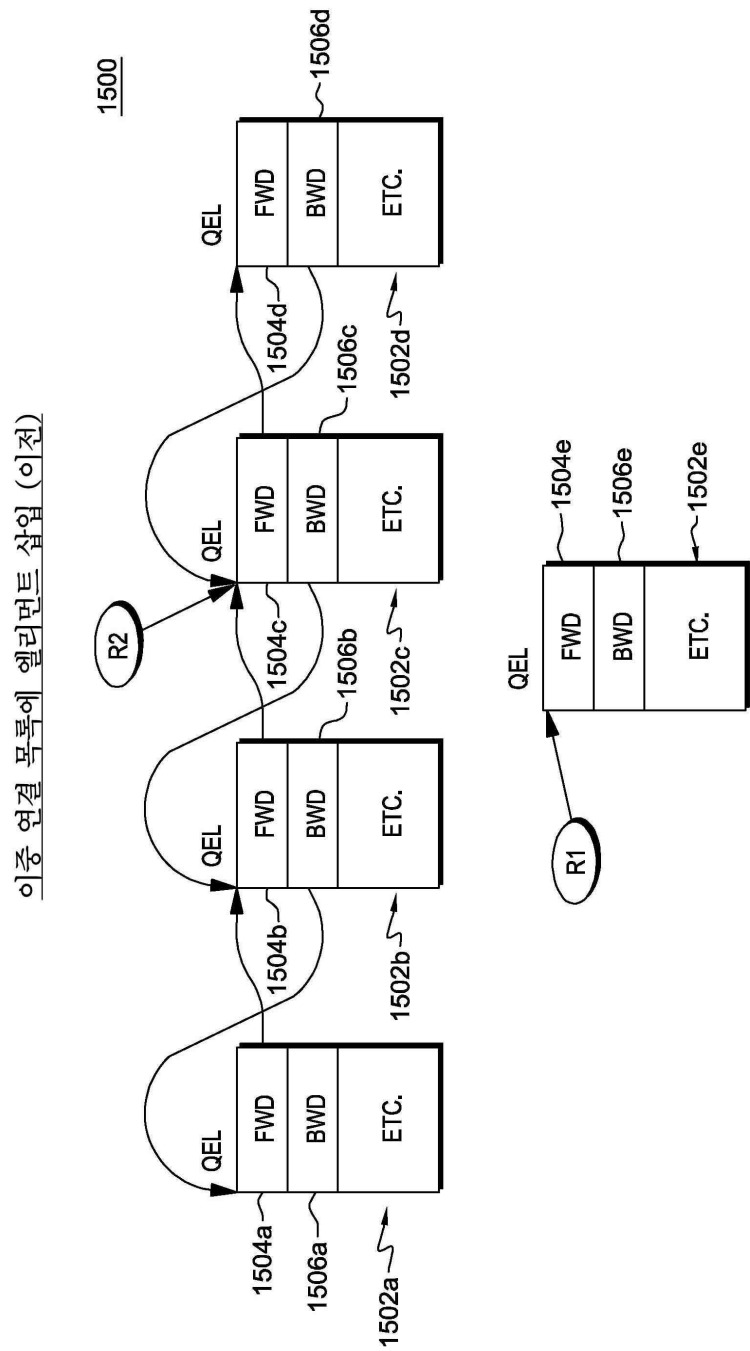
도면13



도면14

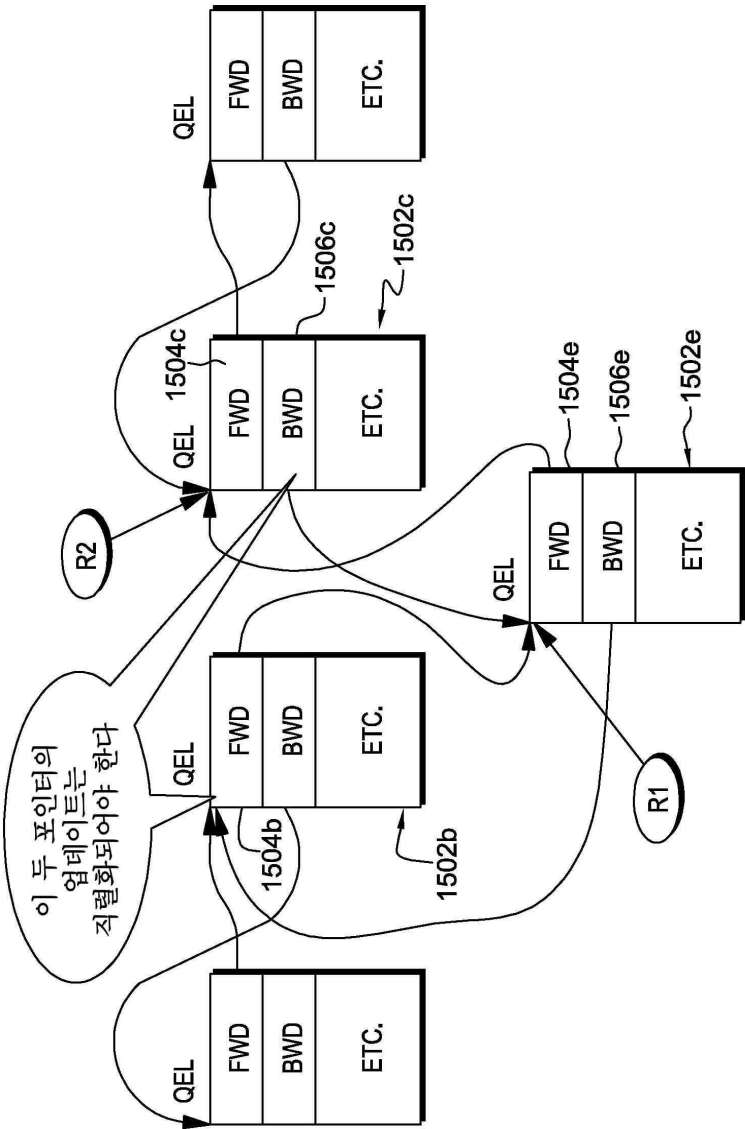


도면15a



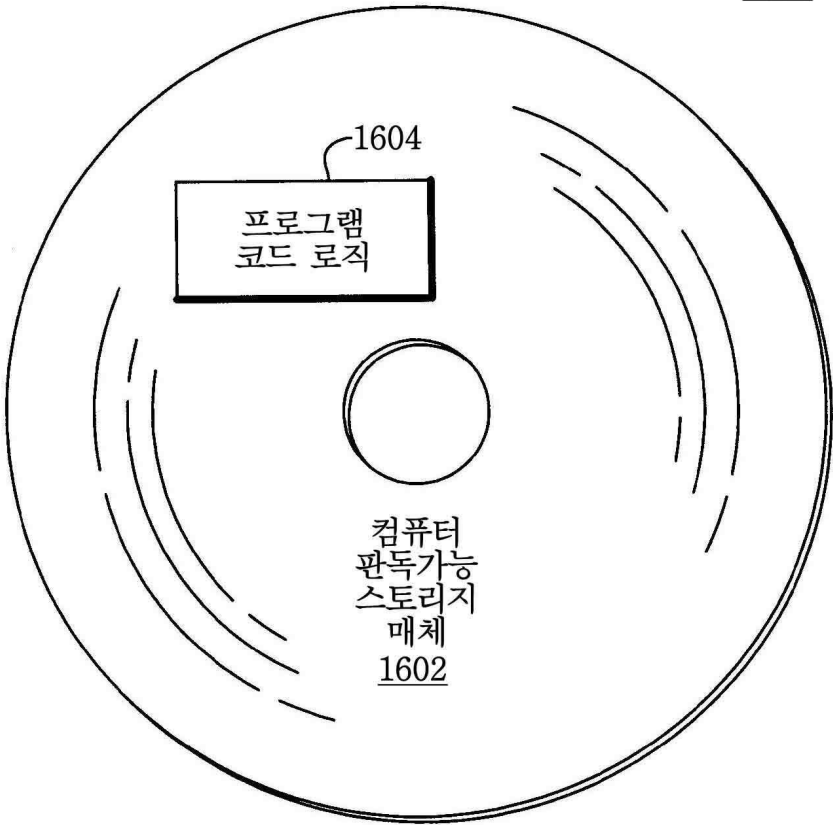
도면15b

이중 연결 목록에 엘리먼트 삽입 (이후)

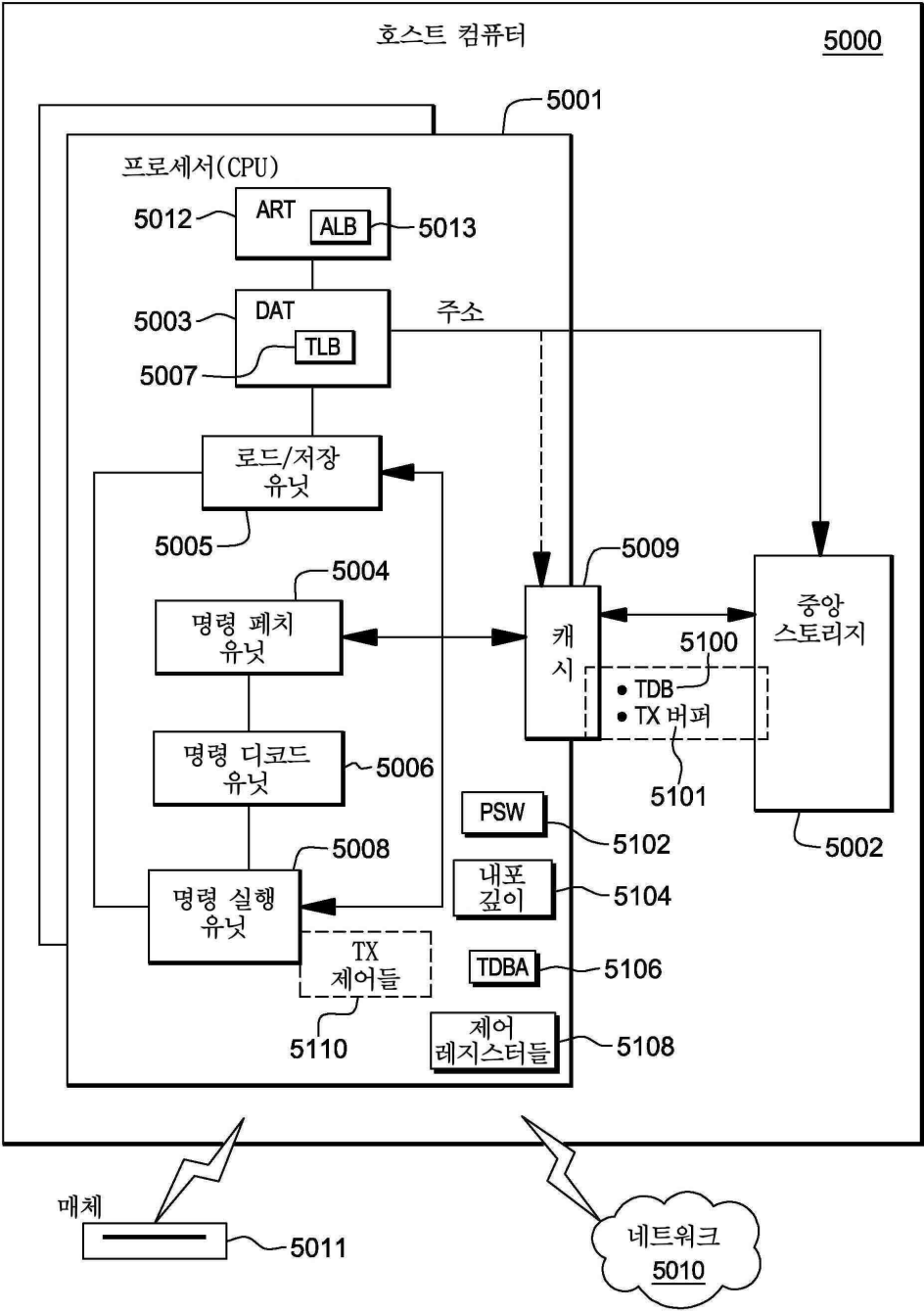


도면16

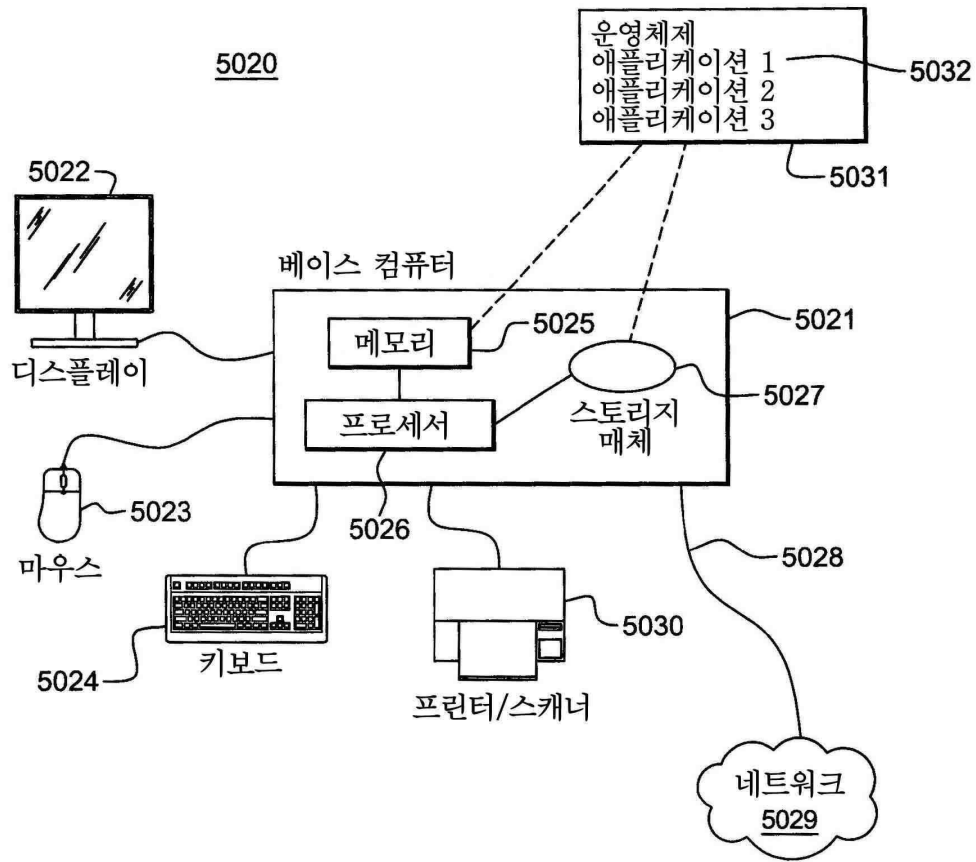
컴퓨터
프로그램
제품
1600



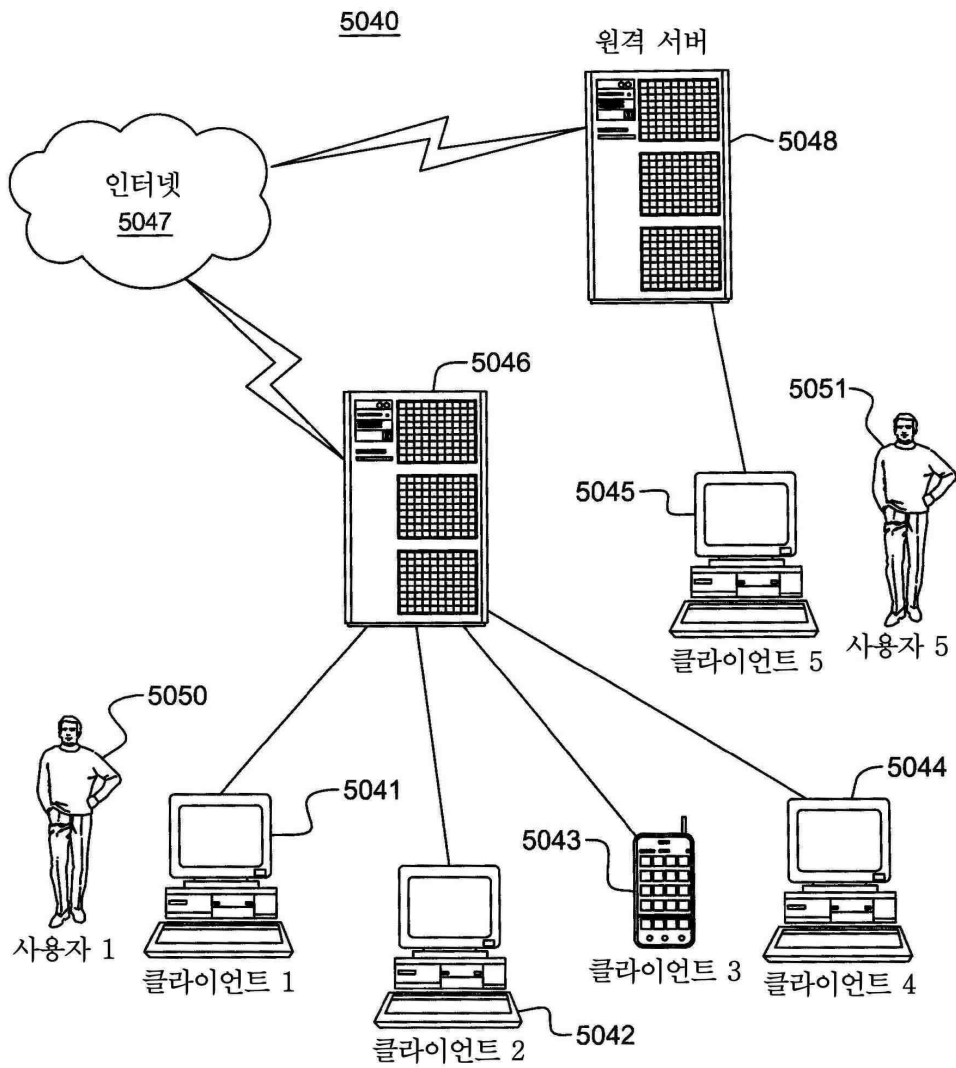
도면17



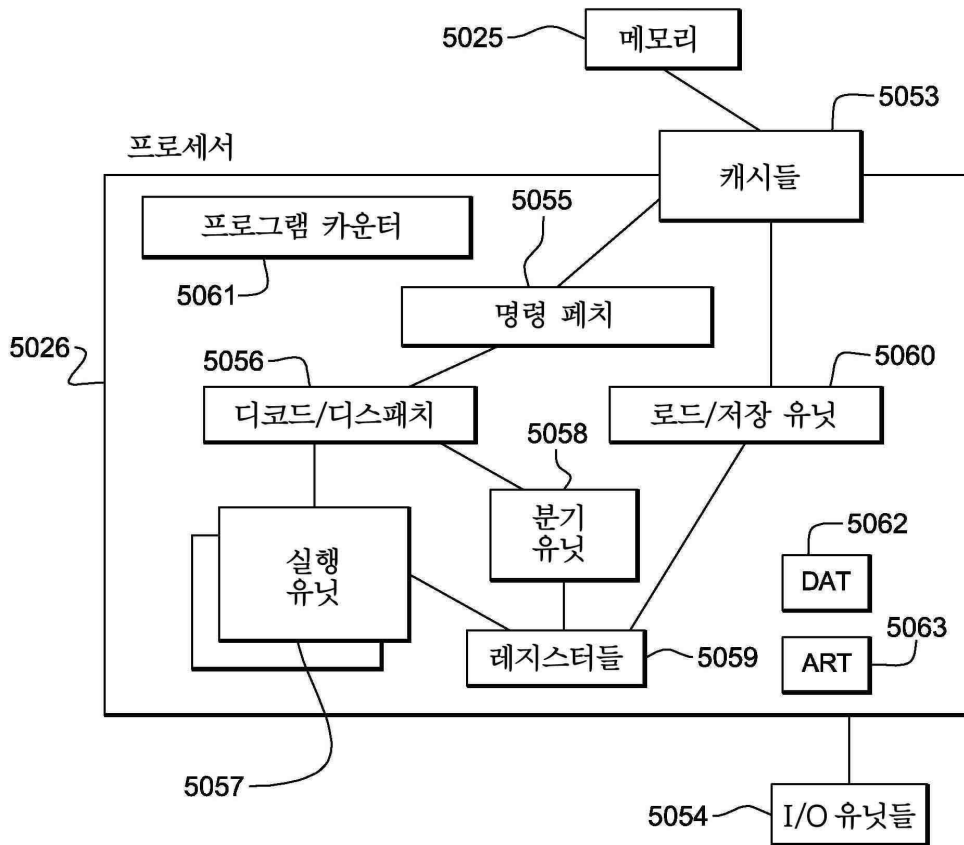
도면18



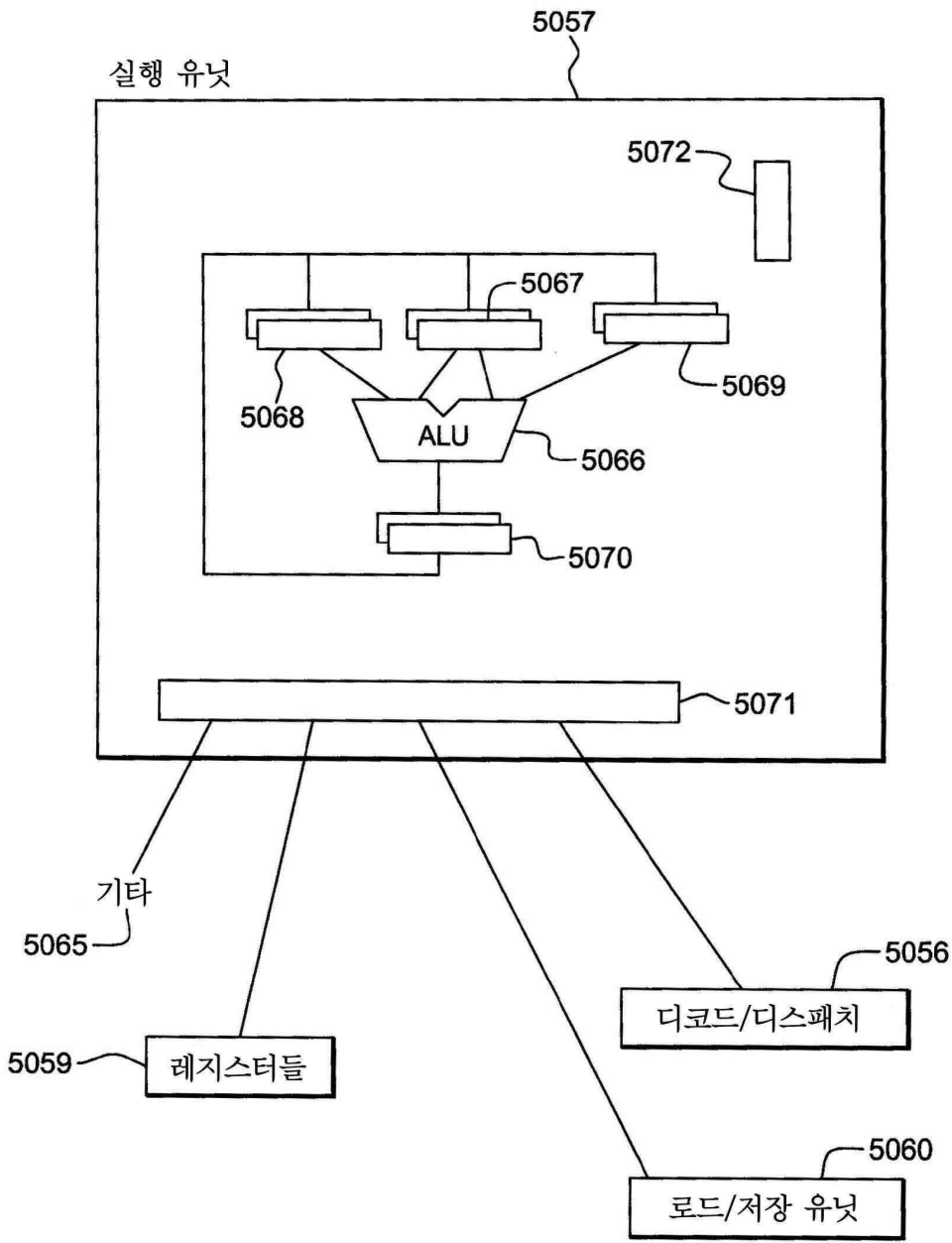
도면19



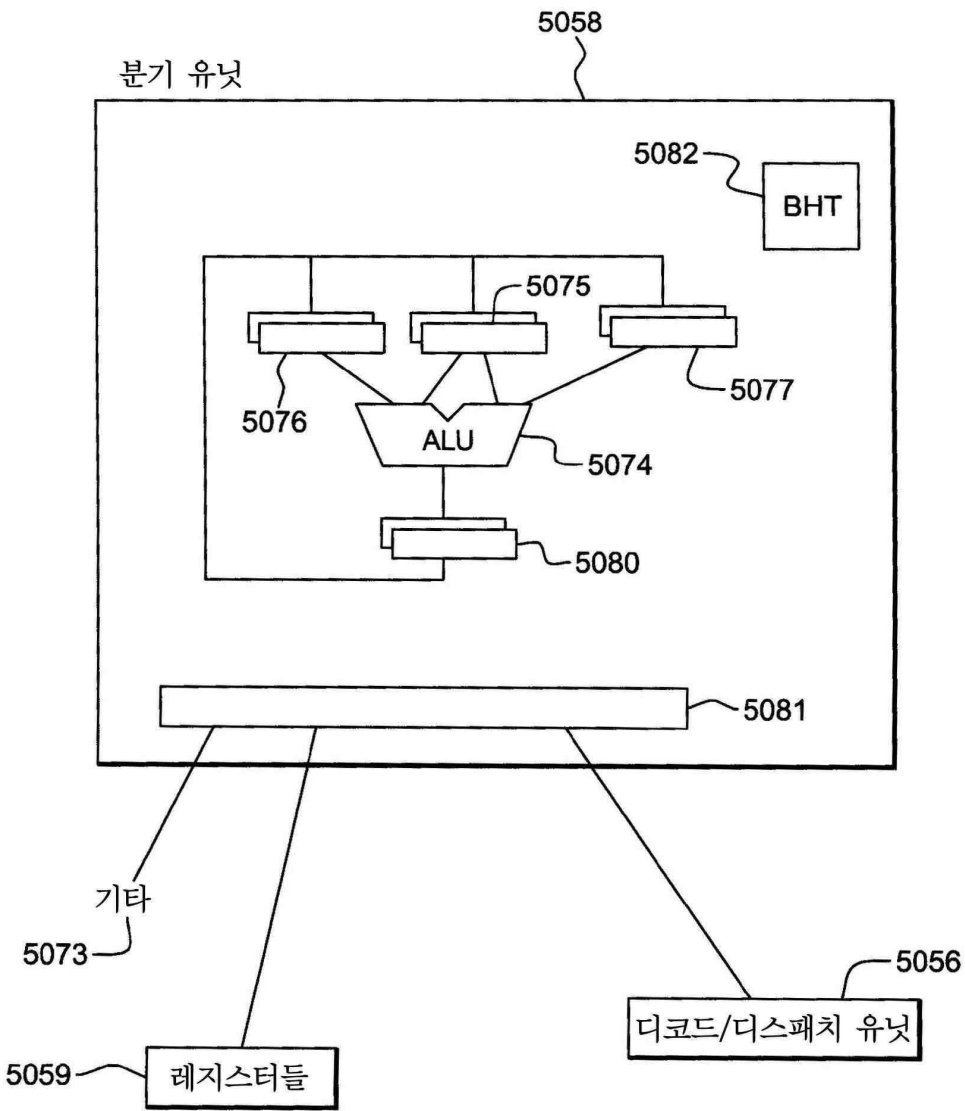
도면20



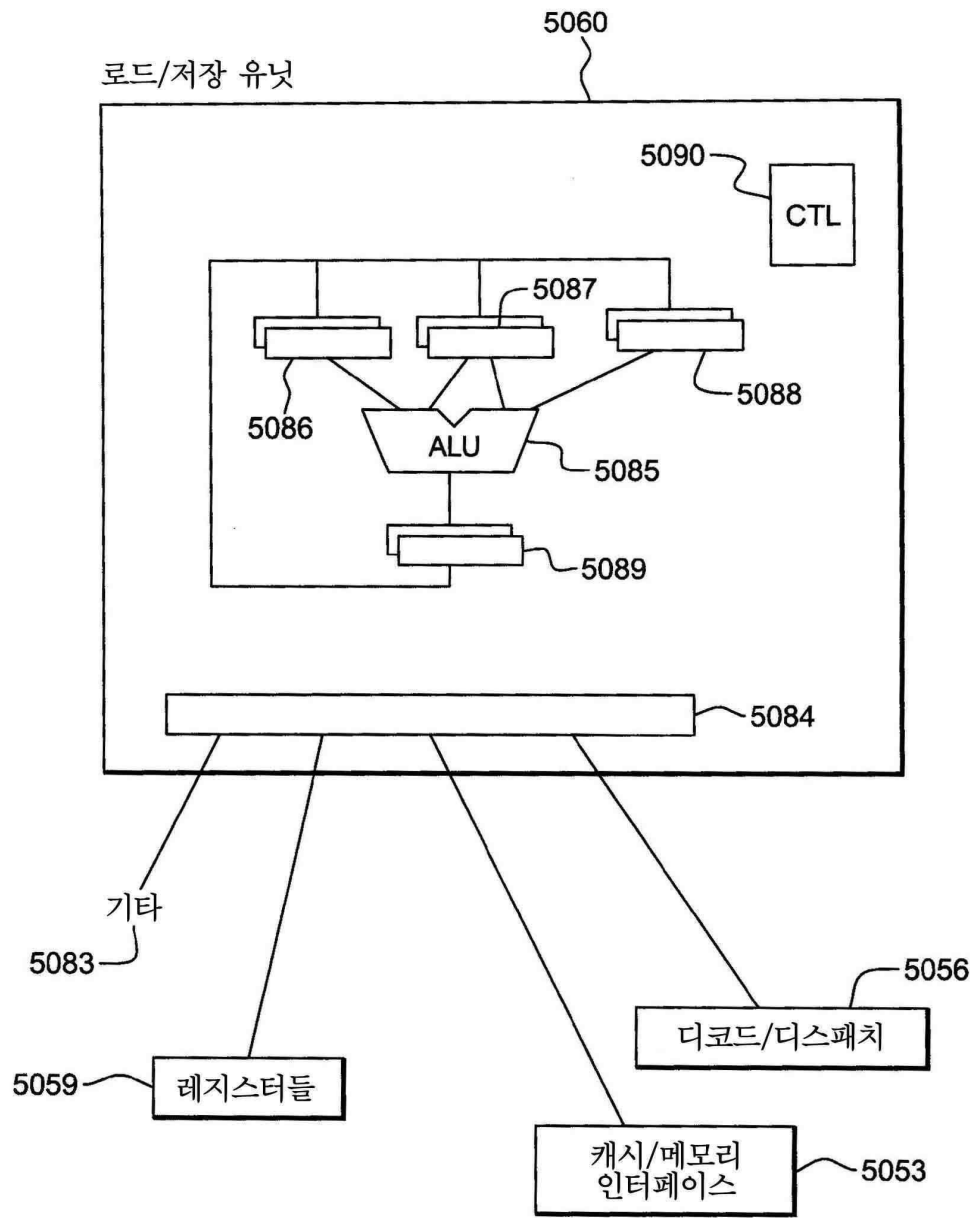
도면21a



도면21b



도면21c



도면22

